Máster en Desarrollo con Inteligencia Artificial

Big School

TRABAJO FIN DE MÁSTER

Accesibilidad 360

Plataforma web colaborativa para consultar y compartir información sobre la accesibilidad de establecimientos

Autor: Jorge Serna

Tutor: Brais Moure

Octubre de 2026

Resumen

Las personas con movilidad reducida, con discapacidad sensorial o con necesidades de apoyo suelen carecer de información fiable sobre las barreras de un establecimiento antes de desplazarse a él. Las plataformas existentes la ofrecen de forma incompleta, con un modelo de valoración poco detallado o con una cobertura limitada. Este trabajo diseña, implementa y despliega Accesibilidad 360, una plataforma web colaborativa en la que las personas registran establecimientos, los valoran con ocho criterios de accesibilidad puntuados de 0 a 5, comparten fotografías y los localizan en un mapa.

La aplicación está construida con Next.js 15, React 19, TypeScript, Prisma y PostgreSQL, con autenticación mediante Auth.js, almacenamiento de imágenes en Cloudinary y mapas de OpenStreetMap, y está desplegada en Vercel con la base de datos en Neon. Su arquitectura se organiza en módulos funcionales, con la autorización y las reglas de negocio concentradas en una capa de servicios. El modelo de datos incorpora la geografía administrativa oficial del INE, con 52 provincias y 8.192 municipios. El desarrollo siguió un método guiado por catorce especificaciones escritas antes del código y se apoyó en herramientas de inteligencia artificial como asistentes, con revisión, pruebas y aceptación manuales por parte del autor.

La evaluación se realizó con tres instrumentos propios. Una revisión estática de seguridad, tomando como marco el OWASP Top 10, no encontró hallazgos críticos ni altos, pero sí cuatro de severidad media (limitación de intentos, cabeceras de seguridad, configuración de sesión y protección en el middleware) y siete de severidad baja. Una medición de accesibilidad frente a WCAG 2.2, con axe-core y revisión manual, mostró 14 de 17 estados de página sin violaciones automáticas, pero identificó siete defectos, entre ellos marcadores del mapa inaccesibles con teclado y contrastes insuficientes, por lo que no se declara conformidad con ningún nivel. Una batería de 192 pruebas automáticas cubre en buena medida los servicios y la validación, aunque no la interfaz, la base de datos real ni los flujos completos de usuario. De los 36 requisitos no funcionales, 25 están cubiertos, 4 parcialmente y 7 no lo están, y los 23 requisitos funcionales están implementados.

Se concluye que el producto cumple su función y que su base de ingeniería es sólida, en especial la separación en capas, pero que sus garantías de seguridad, accesibilidad y pruebas son parciales: sus deficiencias están medidas, priorizadas y recogidas en una hoja de ruta de 21 acciones. La aplicación es apta como prototipo y no para uso abierto sin cerrar esas carencias. El trabajo no incluye pruebas con personas usuarias ni con lectores de pantalla, por lo que las conclusiones sobre usabilidad y utilidad de la información son provisionales.

Palabras clave: accesibilidad, WCAG, plataforma colaborativa, desarrollo con inteligencia artificial, Next.js, seguridad web.

Abstract

People with reduced mobility, sensory disabilities or support needs often lack reliable information about the barriers of a venue before travelling to it. Existing platforms provide it incompletely, with a coarse rating model or limited coverage. This work designs, implements and deploys Accesibilidad 360, a collaborative web platform where people register venues, rate them against eight accessibility criteria scored from 0 to 5, share photographs and locate them on a map.

The application is built with Next.js 15, React 19, TypeScript, Prisma and PostgreSQL, with authentication through Auth.js, image storage on Cloudinary and OpenStreetMap maps, and is deployed on Vercel with the database on Neon. Its architecture is organised into functional modules, with authorisation and business rules concentrated in a service layer. The data model incorporates the official administrative geography of Spain's national statistics institute (INE), with 52 provinces and 8,192 municipalities. Development followed a method guided by fourteen specifications written before the code and relied on artificial intelligence tools as assistants, with review, testing and acceptance performed manually by the author.

The evaluation used three instruments of its own. A static security review, based on the OWASP Top 10, found no critical or high findings, but four of medium severity (rate limiting, security headers, session configuration and middleware protection) and seven of low severity. An accessibility assessment against WCAG 2.2, using axe-core and manual review, showed 14 of 17 page states with no automated violations, but identified seven defects, including map markers unreachable by keyboard and insufficient colour contrast, so no conformance level is claimed. A suite of 192 automated tests covers the services and validation to a large extent, but not the user interface, a real database or complete user flows. Of the 36 non-functional requirements, 25 are met, 4 partially and 7 are not, and all 23 functional requirements are implemented.

The work concludes that the product fulfils its purpose and that its engineering base is sound, especially its layered separation, but that its security, accessibility and testing guarantees are partial: the shortcomings are measured, prioritised and gathered in a roadmap of 21 actions. The application is suitable as a prototype, and not for open use until those gaps are closed. The study includes no tests with end users or with screen readers, so conclusions on usability and on the usefulness of the information are provisional.

Keywords: accessibility, WCAG, collaborative platform, AI-assisted development, Next.js, web security.

Índice

Resumen **2**

Abstract **3**

1. Introducción **7**

1.1 Contexto 7
1.2 Motivación 7
1.3 Problema existente 8
1.4 Objetivos 8
1.5 Alcance 9
1.6 Limitaciones 10
1.7 Enfoque metodológico 11
1.8 Organización de la memoria 11

2. Estado del arte **12**

2.1 Planteamiento y criterios de análisis 12
2.2 Soluciones existentes 12
2.3 Comparación 15
2.4 Carencias identificadas 15
2.5 Aportación de Accesibilidad 360 16
2.6 Limitaciones frente a las soluciones existentes 17
2.7 Síntesis 18

3. Análisis de requisitos **19**

3.1 Alcance, método y actores 19
3.2 Requisitos funcionales 20
3.3 Requisitos no funcionales 24
3.4 Restricciones de diseño 27
3.5 Síntesis 28

4. Arquitectura del sistema **29**

4.1 Visión general y criterios de diseño 29
4.2 Pila tecnológica 30
4.3 Organización del código: arquitectura por funcionalidades 31
4.4 Arquitectura cliente-servidor 32
4.5 Capas y responsabilidades 33
4.6 Server Actions 34
4.7 Autenticación y sesiones 35
4.8 Autorización 37
4.9 Integración con servicios externos 37
4.10 Modelo de datos 38
4.11 Decisiones de arquitectura registradas 38
4.12 Limitaciones y síntesis 39

5. Diseño de la base de datos **40**

5.1 Criterios de diseño 40
5.2 Visión general del modelo 40
5.3 Relaciones, cardinalidades y política de borrado 42
5.4 Restricciones de integridad 43
5.5 Índices 44
5.6 Decisiones de modelado 45

5.7 Elementos del esquema sin uso en la versión actual....46
5.8 Migraciones y datos iniciales....46
5.9 Limitaciones y síntesis....47

6. Desarrollo de las funcionalidades....48
6.1 Planteamiento del capítulo....48
6.2 Registro de usuarios (RF-01)....48
6.3 Inicio y cierre de sesión (RF-02, RF-03)....48
6.4 Perfil de usuario (RF-05)....49
6.5 Alta de establecimientos (RF-06)....49
6.6 Edición y eliminación de establecimientos (RF-09, RF-10, RF-22)....50
6.7 Listado, búsqueda y filtros (RF-11)....51
6.8 Gestión de categorías y administración (RF-21)....51
6.9 Geolocalización y mapas (RF-07, RF-18, RF-19)....52
6.10 Valoraciones (RF-12 a RF-15)....53
6.11 Fotografías y carga de imágenes (RF-16, RF-17)....54
6.12 Panel principal (RF-20)....55
6.13 Validaciones....55
6.14 Dónde se aplica cada permiso....56
6.15 Páginas informativas (RF-23)....56
6.16 Síntesis....57

7. Seguridad....58
7.1 Enfoque....58
7.2 Contexto de amenaza....58
7.3 Controles implementados....59
7.4 Evaluación según el OWASP Top 10....60
7.5 Hallazgos....61
7.6 Dependencias y entrega continua....64
7.7 Protección de datos personales....64
7.8 Riesgo residual....65
7.9 Limitaciones de la evaluación y síntesis....65

8. Accesibilidad....67
8.1 Enfoque....67
8.2 Decisiones de diseño....67
8.3 Resultados de la auditoría automática....68
8.4 Revisión manual....69
8.5 Defectos detectados....70
8.6 Cobertura de las pruebas automáticas....72
8.7 Límites de la evaluación....72
8.8 Síntesis y mejoras propuestas....72

9. Uso de la inteligencia artificial en el desarrollo....74
9.1 Alcance y principio rector....74
9.2 Herramientas utilizadas....74
9.3 Tareas en las que se empleó....75
9.4 Método de trabajo: contexto escrito y desarrollo guiado por especificaciones....75
9.5 Control humano....76
9.6 Casos en que la IA no bastó....77
9.7 Riesgos de usar IA en el desarrollo y cómo se trataron....78
9.8 Limitaciones....78
9.9 Reflexión y recomendaciones....79

10. Pruebas....80
10.1 Estrategia....80
10.2 Herramientas y ejecución....80
10.3 Pruebas automáticas....80
10.4 Pruebas de extremo a extremo....83
10.5 Pruebas manuales....83
10.6 Incidencias detectadas....83
10.7 Análisis y mejoras propuestas....84
11. Despliegue....86
11.1 Entorno de producción....86
11.2 Configuración....86
11.3 Proceso de despliegue....87
11.4 Administración y operación....88
11.5 Riesgos y limitaciones del despliegue....88
11.6 Mejoras propuestas....89
12. Resultados y discusión....91
12.1 Qué se ha obtenido....91
12.2 Cumplimiento de los objetivos específicos....91
12.3 Cumplimiento de los requisitos....93
12.4 Valoración del objetivo general....93
12.5 Contraste con la aportación declarada....94
12.6 Qué funcionó y qué no....94
12.7 Limitaciones de la evaluación....95
12.8 Síntesis....95
13. Trabajo futuro....96
13.1 Criterio de ordenación....96
13.2 Fase 1: cerrar las deficiencias de uso real....96
13.3 Fase 2: consolidar la base técnica....97
13.4 Fase 3: evolución funcional....98
13.5 Fase 4: validación e investigación....99
13.6 Dependencias y camino mínimo....99
14. Conclusiones....101
14.1 Síntesis del trabajo....101
14.2 Conclusiones....101
14.3 Aportaciones....102
14.4 Lecciones aprendidas....102
14.5 Cierre....103
Bibliografía....104
Anexos....106

1. Introducción

1.1 Contexto

La Encuesta de Discapacidad, Autonomía personal y situaciones de Dependencia (EDAD) 2020 estimó que 4,38 millones de personas residentes en hogares españoles tenían alguna discapacidad o limitación, lo que equivale a 94,9 personas por cada mil habitantes (Instituto Nacional de Estadística [INE], 2022). Una parte de ese colectivo, junto con personas mayores, familiares y acompañantes, se enfrenta de forma cotidiana a una pregunta aparentemente sencilla: si podrá entrar y moverse en un lugar al que desea ir.

El marco normativo español reconoce que esa pregunta no debería depender de la suerte. El Real Decreto Legislativo 1/2013 define la accesibilidad universal como la condición que deben cumplir entornos, productos y servicios para ser comprensibles, utilizables y practicables por todas las personas, de la forma más autónoma y natural posible (Real Decreto Legislativo 1/2013, 2013). La Convención sobre los derechos de las personas con discapacidad, ratificada por España en 2008, incluye la accesibilidad al entorno físico y a la información entre los principios sobre los que se asienta el ejercicio de los demás derechos (Jefatura del Estado, 2008).

Existe, sin embargo, una distancia entre la obligación de que un establecimiento sea accesible y la posibilidad de saberlo antes de llegar a él. El cumplimiento normativo se evalúa en los edificios, pero la información que necesita una persona para decidir un desplazamiento es otra: si hay un escalón en la entrada, cuánto mide la puerta, si el aseo permite girar una silla de ruedas o si hay una plaza de aparcamiento reservada cerca. Es una información concreta, local y susceptible de cambiar, y suele estar repartida entre la experiencia de quienes han visitado el lugar.

Accesibilidad 360 nace en ese espacio. Es una aplicación web colaborativa en la que las personas usuarias registran establecimientos, los valoran según criterios de accesibilidad definidos, aportan fotografías y consultan lo que otras personas han publicado. La aplicación no sustituye a una certificación oficial; la complementa con información de uso cotidiano.

1.2 Motivación

La motivación del trabajo tiene dos vertientes, una de utilidad y otra de formación.

La primera es la utilidad del problema. La información de accesibilidad es especialmente valiosa para quien no puede resolver un imprevisto sobre la marcha: llegar a un local y descubrir que no se puede entrar tiene un coste mayor para una persona con movilidad reducida que para el resto. Un servicio que reduzca esa incertidumbre justifica el esfuerzo de desarrollo incluso en su forma más acotada.

La segunda es la formación en ingeniería del software. El problema exige resolver, en un único proyecto de tamaño contenido, cuestiones que aparecen en la mayoría de las aplicaciones web actuales: identificación y autorización de usuarios con distintos niveles de permiso, un modelo de datos relacional con integridad referencial, validación de entradas, gestión de archivos en un servicio externo, tratamiento de datos geográficos, pruebas automatizadas, integración continua y despliegue en producción. Aborda además un requisito poco habitual como criterio de calidad principal: la aplicación debe ser utilizable por las personas a las que se dirige, de modo que la accesibilidad es una propiedad del propio producto y no solo su tema.

Una tercera consideración atañe al marco del máster. El desarrollo se ha realizado con asistencia de herramientas de inteligencia artificial, y el trabajo es también una ocasión para documentar de forma rigurosa cómo se integra esa asistencia en un flujo de ingeniería en el que las decisiones, la revisión y la aceptación final corresponden a la persona que desarrolla (capítulo 9).

1.3 Problema existente

El planteamiento del producto identifica cinco carencias en la forma en que hoy se obtiene la información de accesibilidad de un establecimiento. La información disponible suele estar incompleta, con campos sin rellenar o limitada a indicaciones genéricas. No siempre está actualizada, porque las condiciones de un local cambian con obras, cambios de titularidad o decisiones de gestión. Con frecuencia carece de fotografías, que son la forma más directa de comprobar una entrada o un aseo. No permite valorar por separado los distintos aspectos de la accesibilidad, de modo que un único indicador oculta, por ejemplo, que un local tiene acceso sin escalones pero un aseo inutilizable. Y, en muchos casos, depende exclusivamente de lo que declara el propietario, que tiene un interés evidente en mostrar el local de la mejor forma posible.

La consecuencia práctica es que las personas con movilidad reducida deben desplazarse sin saber si podrán acceder, o descartar por prudencia lugares que sí lo serían. El estado de la técnica y la comparación con las plataformas existentes se desarrollan en el capítulo 2; aquí basta con fijar el problema que el proyecto pretende abordar: la ausencia de una fuente de información de accesibilidad que sea detallada, actualizada por la comunidad y apoyada en evidencia visual.

1.4 Objetivos

1.4.1 Objetivo general

Diseñar, implementar y desplegar una plataforma web colaborativa que permita registrar establecimientos, valorar su accesibilidad según criterios definidos, compartir fotografías y localizarlos en un mapa, aplicando prácticas de ingeniería del software que garanticen su mantenibilidad, su seguridad y su propia accesibilidad.

1.4.2 Objetivos específicos

Los objetivos específicos concretan el general en resultados verificables. La tabla 1.1 los enumera junto con el criterio con el que se considerará cumplido cada uno y el capítulo donde se analiza.

Tabla 1.1. *Objetivos específicos, criterio de cumplimiento y capítulo de referencia.*

Id.	Objetivo específico	Criterio de cumplimiento	Capítulo
OE1	Definir un modelo de datos relacional que represente usuarios, establecimientos, valoraciones por criterio, fotografías, categorías y geografía administrativa.	Esquema migrado y poblado con datos de referencia, con integridad garantizada por la base de datos.	5
OE2	Implementar identificación de usuarios y control de acceso basado en roles y en la titularidad de los recursos.	Toda operación sensible se autoriza en servidor, con pruebas automatizadas de los casos permitidos y denegados.	4, 6 y 7
OE3	Permitir valorar la accesibilidad mediante ocho criterios con escala numérica y opción «No aplicable» en los que lo admiten.	Una valoración por persona y establecimiento, con media calculada excluyendo los criterios no aplicables.	6
OE4	Gestionar fotografías con almacenamiento externo y validación de formato, peso y dimensiones.	Subida, selección de imagen principal y eliminación operativas, sin guardar el archivo en la base de datos.	6
OE5	Localizar los establecimientos mediante geocodificación y mostrarlos en un mapa.	Mapa por establecimiento y mapa global operativos, con comportamiento definido cuando no hay coordenadas.	6
OE6	Ofrecer búsqueda y filtrado por nombre, categoría, provincia y municipio, con varios órdenes.	Estado de la consulta reflejado en la URL y resultados coherentes con los filtros.	6
OE7	Construir una aplicación que sea ella misma accesible.	Uso de componentes y semántica accesibles, navegación por teclado y comprobación automática con axe-core.	8
OE8	Asegurar la calidad mediante pruebas automatizadas e integración continua.	Suite de pruebas unitarias, de integración y de extremo a extremo ejecutable en el proceso de integración.	10
OE9	Desplegar la aplicación en un entorno de producción.	Aplicación accesible públicamente con base de datos y almacenamiento de imágenes gestionados.	11
OE10	Analizar de forma crítica la incorporación de herramientas de IA al desarrollo.	Descripción del flujo de trabajo, de las tareas asistidas y del control humano aplicado.	9

1.5 Alcance

El trabajo cubre el diseño y la implementación de un producto mínimo viable completo, esto es, funcional de extremo a extremo, aunque limitado en funcionalidades. Dentro del alcance se

encuentran el registro e inicio de sesión de usuarios, la gestión del perfil (nombre y contraseña), el alta, la edición y la eliminación de establecimientos con las restricciones de permisos que se detallan en el capítulo 6, la gestión de categorías por parte de la administración, las valoraciones por criterio, las fotografías, el mapa de cada establecimiento y el mapa global, la búsqueda y el filtrado, un panel con estadísticas básicas y las páginas legales.

La información geográfica se limita a España. Las provincias y los municipios proceden del dataset oficial del INE, con 52 provincias y 8.192 municipios, y no se admite texto libre para la ubicación administrativa. La interfaz está disponible únicamente en español.

La aplicación distingue tres roles en su modelo de datos: usuario, moderador y administrador. En la versión desarrollada solo los roles de usuario y administrador tienen un comportamiento diferenciado; el rol de moderador queda reservado para una evolución posterior. La consulta de establecimientos, valoraciones y mapa requiere haber iniciado sesión; la parte pública se reduce a la página de presentación, el registro, el inicio de sesión y los documentos legales.

Quedan fuera del alcance la aplicación móvil nativa, la interfaz de programación pública, la internacionalización, los favoritos, la gamificación, las notificaciones, la moderación colaborativa y la recuperación de contraseña. La inteligencia artificial no forma parte del producto: interviene únicamente en el proceso de desarrollo.

1.6 Limitaciones

Las limitaciones del trabajo derivan del alcance elegido y de la naturaleza del producto, y se retoman en los capítulos correspondientes con mayor detalle.

La primera es de naturaleza epistemológica. Los datos de Accesibilidad 360 son testimonios de personas usuarias, no mediciones técnicas. La escala de 0 a 5 es una apreciación subjetiva, sin un protocolo de medida normalizado, y dos personas pueden puntuar de forma distinta el mismo local. La plataforma tampoco verifica la veracidad de lo publicado ni dispone de un mecanismo de moderación, de modo que la calidad de la información depende de la participación y de la buena fe de la comunidad. Por este motivo, la aplicación se presenta como complemento de las certificaciones oficiales y no como sustituto.

La segunda es de infraestructura. El producto se apoya en servicios externos de uso gratuito para el alojamiento, la base de datos, el almacenamiento de imágenes y la geocodificación. Esta elección reduce el coste y facilita la reproducción del proyecto, pero impone límites de capacidad y condiciones de uso que se analizan en el capítulo 11, y hace que la ubicación de un establecimiento pueda quedar sin determinar si el servicio de geocodificación no responde o no reconoce la dirección.

La tercera es de verificación. Aunque se dispone de pruebas automatizadas y de pruebas funcionales manuales, la memoria no recoge una evaluación de usabilidad con personas con

discapacidad. La comprobación de accesibilidad se ha apoyado en criterios de diseño, en el uso de componentes accesibles y en herramientas automáticas, que detectan una parte de los problemas y no sustituyen la valoración de las personas usuarias (capítulo 8).

La cuarta atañe a la seguridad. La revisión estática realizada sobre el código identificó mejoras pendientes de endurecimiento, que se exponen en el capítulo 7 junto con el riesgo residual que suponen para un despliegue de este alcance.

1.7 Enfoque metodológico

El desarrollo se ha guiado por un enfoque de desarrollo dirigido por especificaciones: cada funcionalidad se define antes de implementarse en un documento con su objetivo, requisitos, criterios de aceptación, casos límite y estrategia de pruebas, y la implementación se considera terminada cuando compila, supera el análisis estático y las pruebas, y cumple los criterios de aceptación. El código es la referencia última del comportamiento del sistema, y así se trata en esta memoria: cuando se describe una funcionalidad se describe la que el código implementa.

El trabajo se organizó en fases incrementales, una por especificación, de modo que el sistema fuera operativo en todo momento. El capítulo 4 describe la arquitectura que sostiene ese crecimiento y el capítulo 10 recoge cómo se verificó cada incremento.

1.8 Organización de la memoria

La memoria se organiza en catorce capítulos. Tras esta introducción, el capítulo 2 analiza las soluciones existentes y la aportación del proyecto, y el capítulo 3 formaliza los requisitos funcionales y no funcionales. Los capítulos 4 y 5 describen la arquitectura del sistema y el diseño de la base de datos. El capítulo 6 expone la implementación de cada funcionalidad, y los capítulos 7 y 8 se dedican a dos propiedades transversales: la seguridad y la accesibilidad. El capítulo 9 analiza el papel de la inteligencia artificial en el desarrollo y el capítulo 10 las pruebas realizadas. El capítulo 11 describe el despliegue. Los capítulos 12, 13 y 14 recogen, respectivamente, los resultados, el trabajo futuro y las conclusiones. La bibliografía y los anexos, que remiten al manual de usuario, a la auditoría de seguridad, a las especificaciones, al modelo de datos, a las capturas de pantalla y a la medición de accesibilidad, cierran el documento.

2. Estado del arte

2.1 Planteamiento y criterios de análisis

El problema descrito en el capítulo 1 no es nuevo, y existen iniciativas que lo abordan desde hace más de una década. Este capítulo examina las soluciones más próximas a Accesibilidad 360 para situar el proyecto, identificar qué resuelven y qué dejan sin resolver, y delimitar con precisión qué aporta el trabajo.

Se han seleccionado cuatro plataformas por su relación directa con la información de accesibilidad de lugares y por tratarse de ejemplos reconocibles de enfoques distintos: Wheelmap, basada en datos abiertos de OpenStreetMap; AXS Map, de valoraciones por usuarios; Mapcesible, la iniciativa más cercana por ámbito geográfico y destinatarios; y Google Maps, como referencia de servicio generalista que ha incorporado atributos de accesibilidad.

La comparación se organiza en torno a cuatro preguntas. La primera es de dónde proceden los lugares que se muestran, es decir, si la plataforma importa un directorio existente o depende de que alguien lo dé de alta. La segunda es cómo se expresa la accesibilidad, ya sea con una etiqueta de pocos niveles, con una puntuación o con respuestas a un cuestionario. La tercera es quién contribuye y con qué barreras de entrada. La cuarta es si los datos pueden reutilizarse fuera de la plataforma.

Conviene precisar el alcance de la comparación. Se basa en publicaciones y documentación accesibles públicamente, cuyas fechas se indican en cada caso y que no siempre coinciden con el estado actual de los servicios. No se ha realizado una prueba de uso sistemática de cada plataforma, de modo que la descripción se limita a lo que las fuentes afirman y no se atribuyen funciones que no consten en ellas.

2.2 Soluciones existentes

2.2.1 Wheelmap y el etiquetado en OpenStreetMap

Wheelmap es un mapa de lugares accesibles para personas que utilizan silla de ruedas, promovido por Sozialhelden e. V., una asociación sin ánimo de lucro con sede en Berlín. Dispone de sitio web y de aplicaciones para dispositivos móviles, y permite tanto consultar la accesibilidad de un lugar como etiquetarlo (OpenStreetMap Wiki, s. f.-b).

Su característica técnica más relevante es que no mantiene una base de datos propia aislada, sino que escribe la información en OpenStreetMap. La accesibilidad se codifica con la clave `wheelchair`, que toma tres valores principales: `yes`, para un acceso sin restricciones; `limited`, para un acceso parcial, definido en la documentación de OpenStreetMap como una entrada con un escalón de hasta siete centímetros y con las salas principales sin escalones; y `no`, para los lugares sin acceso sin restricciones (OpenStreetMap Wiki, s. f.-a). Una clave adicional,

`toilets:wheelchair`, indica si el lugar dispone de un aseo accesible, y otra, `wheelchair:description`, permite añadir información en texto libre (OpenStreetMap Wiki, s. f.-a, s. f.-b). En la interfaz de Wheelmap estos valores se representan con un código de color de semáforo (Mobasheri et al., 2017).

Mobasheri et al. (2017) describen Wheelmap como una plataforma de captación colaborativa de datos con más de 800.000 lugares registrados en la fecha del estudio, concentrados mayoritariamente en Alemania aunque la plataforma funciona en todo el mundo por apoyarse en OpenStreetMap. Los autores señalan dos limitaciones que resultan relevantes para este trabajo. La primera es que Wheelmap cubre puntos de interés y no rutas. La segunda es la calidad de la información voluntaria: modificar una etiqueta es muy sencillo, y no exige registro cuando solo se cambia el color, lo que obliga a vigilar las tasas de cambio inusuales por zona o por persona.

Desde el punto de vista del modelo de información, la clasificación en tres niveles tiene la ventaja de la simplicidad y de que la contribución exige muy poco esfuerzo. Su coste es la granularidad: un lugar con entrada plana y aseo inutilizable y otro con entrada plana y aseo adaptado pueden recibir el mismo valor intermedio, y la distinción solo se recupera si alguien ha completado las claves complementarias.

2.2.2 AXS Map

AXS Map es una aplicación gratuita de valoración de la accesibilidad de negocios, creada por Jason DaSilva y Alice Cook. Según el artículo que la presentó en 2013, nació de la constatación de que muchos negocios se declaran accesibles sin serlo, y permite a los usuarios buscar un negocio en el mapa y dejar comentarios sobre los elementos que pueden resultar difíciles para personas con necesidades especiales (Dale, 2013).

Un artículo de 2012 describe el planteamiento de la plataforma como un mapa de accesibilidad construido con las aportaciones de la comunidad, en el que las reseñas se basan en criterios como las entradas, las rampas, los ascensores y los aseos, y cada lugar recibe una valoración de una a cinco estrellas (Nemani, 2012). Esa misma fuente indica que, para poblar su base de negocios y ubicaciones, la aplicación utilizaba la API de Places de Google. El enfoque es, por tanto, el opuesto al de Wheelmap en cuanto al origen de los datos: no depende de que los usuarios den de alta los lugares, sino que parte de un directorio existente al que superpone la información de accesibilidad.

La ficha del proyecto en el Zero Project, publicada en 2018, sitúa su inicio en 2010 y recoge que a mediados de 2017 contaba con 100.000 usuarios en 200 ciudades y que había ampliado sus datos a más de 600.000 ubicaciones mediante otras bases de datos de accesibilidad (Zero Project, 2018). Las fuentes consultadas son, por tanto, de entre 2012 y 2018, y no se ha podido verificar el estado actual del servicio. Tampoco precisan la escala utilizada por criterio ni si los

datos pueden reutilizarse fuera de la aplicación, por lo que no se establecen conclusiones sobre esos aspectos.

2.2.3 Mapcesible

Mapcesible es una plataforma colaborativa española con el lema «Haz visible lo accesible». Permite mapear la accesibilidad de lugares, consultar valoraciones y planificar desplazamientos. Está impulsada por Fundación Telefónica, con el apoyo de COCEMFE y el respaldo técnico de ILUNION, y está disponible para Android, iOS y web (datos.gob.es, 2019).

Combina dos fuentes de información. Por un lado, incorpora 14 conjuntos de datos de organismos oficiales, entre ellos varios ayuntamientos. Por otro, cualquier persona, tenga o no movilidad reducida, puede mapear los lugares que visita tras registrarse con su correo electrónico y responder a preguntas sencillas sobre la accesibilidad del sitio. Los lugares analizados incluyen cines, teatros, restaurantes, alojamientos, playas, plazas de aparcamiento y polideportivos, y la fuente indica que a finales de 2018 había más de 3.500 puntos analizados (datos.gob.es, 2019).

Es la iniciativa más próxima a Accesibilidad 360 en ámbito geográfico y en destinatarios. Su diferencia principal está en la fuente de los datos, ya que combina datos institucionales con aportaciones de la comunidad, y en la forma de captar la información, mediante un cuestionario de preguntas sencillas. La fuente consultada no detalla la estructura de ese cuestionario ni cómo se agregan las respuestas, por lo que no se compara a ese nivel.

2.2.4 Google Maps

Google Maps es un servicio generalista de mapas que ha incorporado información de accesibilidad a su directorio de lugares. Según la publicación de su programa Local Guides de mayo de 2021, la plataforma admite atributos como entrada accesible para silla de ruedas, aseo accesible, aparcamiento accesible y ascensores, además de un atributo que indica la existencia de bucle de inducción para personas con audífonos. Admite también detalles de accesibilidad en las reseñas, como caminos táctiles, rampas, iluminación, braille o asientos adecuados, y listas públicas de lugares accesibles (Chomsky, 2021).

La contribución puede hacerse escribiendo reseñas con una plantilla de accesibilidad, respondiendo a preguntas sobre un negocio después de visitarlo, o, en el caso de propietarios o gestores con un perfil de empresa verificado, añadiendo los atributos directamente (Chomsky, 2021). La ventaja evidente es la cobertura: la información de accesibilidad se asocia a un directorio de lugares que ya existe y que la población utiliza para otros fines. Desde la perspectiva de este trabajo, interesa destacar dos rasgos de su modelo. Los atributos indican la presencia de una característica, y la fuente no describe una escala graduada. Y los propietarios pueden declararlos, lo que reintroduce la dependencia del declarante que el planteamiento del

producto identifica como una de las carencias del problema (apartado 1.3), aunque coexista con las aportaciones de usuarios.

2.3 Comparación

La tabla 2.1 resume las características de las plataformas analizadas en las cuatro dimensiones definidas en el apartado 2.1 e incluye Accesibilidad 360 para facilitar la comparación. Cuando una fuente no aporta información sobre un aspecto, la celda no se rellena con suposiciones.

Tabla 2.1. *Comparación de plataformas de información de accesibilidad de lugares.*

Plataforma	Origen de los lugares	Expresión de la accesibilidad	Contribución	Reutilización de los datos
Wheelmap	Puntos de interés de OpenStreetMap; más de 800.000 lugares en 2017, mayoría en Alemania	Tres niveles (accesible, parcial, no accesible), aseo accesible como dato aparte	Etiquetado abierto; cambiar el color no exige registro	Los datos se escriben en OpenStreetMap
AXS Map	Directorio de Google (API de Places) según la fuente de 2012; más de 600.000 ubicaciones en 2017	Valoración de una a cinco estrellas con criterios como entradas, rampas, ascensores y aseos	Reseñas de usuarios	No consta en las fuentes
Mapcesible	14 conjuntos de datos oficiales más aportaciones de la comunidad; más de 3.500 puntos a finales de 2018	Respuestas a preguntas sencillas sobre el lugar; valoraciones consultables	Registro por correo y cuestionario	No consta en las fuentes
Google Maps	Directorio general de lugares	Atributos de presencia por lugar y detalles de accesibilidad en reseñas	Reseñas, respuestas a preguntas y perfil de empresa verificado	No consta en las fuentes
Accesibilidad 360	Alta manual por personas usuarias; geografía normalizada con datos del INE	Ocho criterios con puntuación de 0 a 5 y opción «No aplicable»; media por establecimiento	Cuenta obligatoria; una valoración por persona y establecimiento; fotografías	Sin API pública ni exportación

2.4 Carencias identificadas

El análisis permite extraer cuatro observaciones que orientan las decisiones de diseño del proyecto.

La primera es el compromiso entre simplicidad y detalle. Los modelos de pocos niveles favorecen la participación y la cobertura, como muestra el volumen de Wheelmap, pero pierden la capacidad de distinguir qué aspecto concreto de la accesibilidad falla. Los modelos basados en atributos de presencia plantean una limitación análoga cuando no incorporan grados. La respuesta a la pregunta práctica de una persona con movilidad reducida suele requerir un desglose: un lugar puede ser accesible en la entrada y no en el aseo, y esa información es la que decide si se va o no.

La segunda es la dependencia del declarante. Cuando la información procede del propietario, existe un incentivo a presentar el local de la mejor forma posible, circunstancia que motivó la creación de AXS Map según Dale (2013). Las plataformas colaborativas mitigan el problema con aportaciones de terceros, pero ninguna de las fuentes consultadas la elimina.

La tercera es la calidad de la información voluntaria. Mobasheri et al. (2017) la identifican como una preocupación general en Wheelmap, y la facilidad de modificación que favorece la participación es la misma que facilita el error y el uso indebido. Es una tensión intrínseca a cualquier plataforma abierta, no un defecto de una de ellas.

La cuarta es el papel de la evidencia visual. Las fuentes consultadas describen sistemas de clasificación, cuestionarios y reseñas, pero no presentan la fotografía como elemento central del modelo de datos de accesibilidad. Para una persona que debe decidir un desplazamiento, una imagen de la entrada o del aseo aporta una verificación directa que ninguna etiqueta sustituye.

2.5 Aportación de Accesibilidad 360

Accesibilidad 360 no introduce un concepto nuevo en el campo, y sería inexacto presentarlo como tal. Su aportación es de diseño e integración: combina elementos que las plataformas analizadas resuelven por separado, con un alcance deliberadamente acotado, y lo hace en un producto que puede reproducirse y evaluarse como proyecto académico. Se concreta en cuatro decisiones, cada una de las cuales responde a una de las carencias anteriores.

Primera, un modelo de valoración con ocho criterios puntuados de forma independiente en una escala de 0 a 5. La separación por criterios responde a la primera carencia: permite expresar que un establecimiento tiene acceso sin escalones y un aseo deficiente, y la escala común permite calcular una media comparable entre establecimientos. Dos criterios, el ascensor y el aparcamiento para personas con movilidad reducida, admiten la opción «No aplicable», que se excluye de la media y no se confunde con un cero, porque la ausencia de ascensor en un local de una planta no es un fallo de accesibilidad. El modelo se justifica en el apartado 3.2 y se detalla en el capítulo 5.

Segunda, la limitación a una valoración por persona y establecimiento, que reduce la influencia de una sola persona sobre la media y responde de forma parcial a la tercera carencia. No sustituye a un sistema de moderación, que el proyecto no incluye.

Tercera, la fotografía como parte del establecimiento. Cada ficha admite varias imágenes, con una principal, de modo que la evidencia visual acompaña a las puntuaciones, en respuesta a la cuarta carencia. Las restricciones de formato, peso y dimensiones y la elección de un almacenamiento externo se tratan en el capítulo 6.

Cuarta, una geografía administrativa normalizada. Las provincias y los municipios se toman del dataset oficial del INE y se seleccionan de una lista, sin texto libre, con lo que se evitan las variantes de escritura de un mismo municipio y se facilita el filtrado y el cruce posterior de datos. Es una decisión propia del contexto español que las plataformas de ámbito mundial no pueden adoptar.

2.6 Limitaciones frente a las soluciones existentes

La comparación debe leerse también en sentido contrario, porque en varios aspectos Accesibilidad 360 queda por debajo de las plataformas analizadas.

El primero es la cobertura. Al no importar un directorio de lugares ni conectarse a fuentes institucionales, cada establecimiento debe darlo de alta una persona usuaria. Esto significa que la plataforma solo es útil donde ya hay contribuciones, el problema que se conoce como arranque en frío de los sistemas colaborativos, y que su utilidad crece con la participación. Frente a los más de 800.000 lugares de Wheelmap en 2017 o los 3.500 puntos de Mapcesible a finales de 2018, el alcance del proyecto es el de un producto mínimo viable.

El segundo es la apertura de los datos. Wheelmap escribe en OpenStreetMap y por ello los datos son reutilizables por terceros; Accesibilidad 360 no ofrece interfaz de programación pública ni exportación, lo que limita su integración con otros servicios. La interfaz de programación pública se considera trabajo futuro (capítulo 13).

El tercero es el acceso. La consulta de la información exige haber iniciado sesión, de modo que, a diferencia de Wheelmap, que admite cambios de color sin registro, existe una barrera de entrada que reduce la visibilidad del contenido. La decisión simplifica el modelo de permisos del producto mínimo viable, y se analiza en el capítulo 4.

El cuarto es la ausencia de moderación y de verificación de las aportaciones, que se traduce en que la fiabilidad depende de la comunidad. El rol de moderador existe en el modelo de datos pero no tiene funcionalidad asociada en la versión presentada.

El quinto es de ámbito. La plataforma cubre exclusivamente España y está en español, y no contempla rutas accesibles, un límite que comparte con Wheelmap según Mobasheri et al. (2017).

2.7 Síntesis

Las plataformas analizadas muestran tres estrategias de captación de la información: etiquetado abierto sobre datos libres, reseñas sobre un directorio de lugares ya existente e integración de datos institucionales con la comunidad. Cada una resuelve una parte del problema y acepta un compromiso, normalmente entre cobertura y detalle. Accesibilidad 360 ocupa un lugar distinto en ese espacio: sacrifica cobertura y apertura para ofrecer un modelo de información detallado, desglosado por criterios y acompañado de fotografías, sobre una geografía normalizada. Los capítulos siguientes describen cómo se materializa esa elección, empezando por los requisitos que se derivan de ella.

3. Análisis de requisitos

3.1 Alcance, método y actores

Este capítulo formaliza lo que el sistema debe hacer y las cualidades que debe tener. Los requisitos no se redactaron al margen del desarrollo: proceden de las especificaciones que guiaron cada entrega y se han contrastado después con el código, de modo que lo que aquí se afirma como requisito cubierto es comportamiento comprobable en la aplicación desplegada. Cuando un requisito no está cubierto, se indica así de forma expresa. El capítulo 12 evalúa el conjunto y los capítulos 7 y 8 profundizan en seguridad y accesibilidad.

Cada requisito lleva un identificador (RF para los funcionales y RNF para los no funcionales), una descripción, su origen en las especificaciones y, en los no funcionales, su estado de cumplimiento. El estado puede ser «Cubierto», «Parcial» o «No cubierto». El texto del requisito describe el comportamiento exigido; el estado describe lo que hace la versión entregada.

El alcance es el de un producto mínimo viable. Quedan fuera, por decisión de diseño, los favoritos, la moderación colaborativa, las estadísticas avanzadas, la aplicación móvil, la API pública y el multidioma. El capítulo 13 retoma estas líneas como trabajo futuro.

3.1.1 Actores

La aplicación distingue cuatro situaciones de acceso. Las tres primeras son actores propiamente dichos y la cuarta es una condición relativa a cada recurso.

- **Visitante**: persona sin sesión iniciada. Puede ver la página de presentación, las páginas legales, de privacidad y de cookies, y acceder al registro y al inicio de sesión. No puede consultar establecimientos, mapa ni panel.
- **Usuario registrado**: persona con sesión iniciada. Puede consultar el listado, las fichas y el mapa, crear establecimientos, valorar, subir fotografías y gestionar su perfil.
- **Administrador**: usuario con rol de administración. Además de lo anterior, gestiona las categorías y puede editar o eliminar establecimientos, valoraciones y fotografías de otras personas.
- **Autor del recurso**: condición, no rol. El creador de un establecimiento, el autor de una valoración o quien subió una fotografía puede gestionar ese recurso concreto, y no los de los demás.

El modelo de datos define un tercer rol, el de moderador, previsto para una futura moderación colaborativa. En la versión entregada no tiene funcionalidad propia y sus titulares se comportan como usuarios registrados.

La decisión de exigir sesión para consultar contenido se tomó para esta versión y se justifica en el capítulo 4. Implica que la aplicación no funciona hoy como directorio abierto, y esa limitación se recoge en el apartado 2.6.

3.2 Requisitos funcionales

3.2.1 Justificación del modelo de valoración

La mayor parte de los requisitos funcionales son operaciones habituales sobre datos, pero la valoración de accesibilidad es el elemento que da identidad al proyecto, y su modelo requiere una justificación previa. Se decidió sobre cuatro cuestiones.

Qué se puntúa. Se utilizan ocho criterios: acceso sin escalones, puerta accesible, anchura de paso, espacio de giro, aseo adaptado, ascensor accesible, aparcamiento para personas con movilidad reducida y señalización accesible. Corresponden a los elementos que una persona con movilidad reducida encuentra, en ese orden aproximado, desde que llega a un local hasta que lo utiliza. Es una selección de diseño pensada para el destinatario principal descrito en el capítulo 1, no una transcripción de una norma técnica, y por ello la valoración colaborativa no sustituye a una auditoría de accesibilidad realizada por un técnico.

Cómo se puntúa. Cada criterio recibe un entero de 0 a 5. Una escala común permite comparar establecimientos y calcular una media; seis niveles dan más matiz que un sí o no, que es la limitación de las etiquetas de pocos niveles señalada en el capítulo 2, sin exigir al usuario una precisión que no puede tener. Separar los criterios evita que un establecimiento con entrada accesible y aseo inutilizable reciba una nota intermedia que no informa de ninguna de las dos cosas: la ficha muestra la puntuación por criterio, no solo el resultado agregado.

Qué ocurre cuando un criterio no procede. Un local de una sola planta no tiene ascensor, y puntuarlo con cero lo penalizaría por algo que no es una deficiencia. Para el ascensor y el aparcamiento se admite la opción «No aplicable», que se almacena como ausencia de puntuación y se excluye de la media. Los otros seis criterios la rechazan, porque son elementos que todo local abierto al público ha de tener o que siempre pueden evaluarse. La regla se aplica en el servidor y no solo en el formulario.

Cómo se agrega. La media de un establecimiento es la media aritmética, redondeada a un decimal, de todas las puntuaciones numéricas de todas sus valoraciones. No hay ponderación entre criterios, ya que cualquier peso sería una decisión arbitraria sin respaldo empírico en este trabajo. La media se calcula en el momento de la consulta y no se almacena, de modo que eliminar una valoración la actualiza sin pasos adicionales. Cada usuario puede emitir una única valoración por establecimiento, para que una persona no pese más que las demás, y las valoraciones no se editan; quien quiera cambiar su opinión elimina la suya y crea otra.

Estas decisiones tienen un coste que se discute en el capítulo 12: la puntuación es subjetiva, no hay verificación de que quien valora haya visitado el local y la media sin ponderar trata igual un escalón en la entrada que una señalización deficiente.

3.2.2 Catálogo de requisitos funcionales

Los requisitos se agrupan por área. La columna «Origen» remite a la especificación de cada entrega, recogida en el anexo correspondiente.

Tabla 3.1. *Requisitos funcionales de cuentas, sesión y perfil.*

ID	Requisito	Origen
RF-01	El sistema permitirá registrarse con nombre, correo electrónico, contraseña y confirmación. La contraseña tendrá al menos 12 caracteres, con una mayúscula, una minúscula y un número. El correo será único y se normalizará a minúsculas.	SPEC-010
RF-02	El sistema permitirá iniciar sesión con correo y contraseña. Ante credenciales incorrectas mostrará un mensaje genérico que no indique cuál de los dos datos falla.	SPEC-010
RF-03	El sistema permitirá cerrar la sesión.	SPEC-010
RF-04	El contenido de la aplicación (listado, fichas, mapa, panel, perfil y administración) solo será accesible con sesión; sin ella se redirigirá al inicio de sesión.	SPEC-010
RF-05	El usuario podrá consultar su perfil y su actividad, modificar su nombre y cambiar su contraseña, para lo cual deberá indicar la actual. La nueva seguirá las reglas del registro.	SPEC-020

Tabla 3.2. *Requisitos funcionales de establecimientos.*

ID	Requisito	Origen
RF-06	El usuario registrado podrá dar de alta un establecimiento con nombre (3 a 120 caracteres), categoría, dirección, provincia, municipio y descripción opcional. Provincia y municipio se elegirán de las entidades oficiales y el municipio deberá pertenecer a la provincia.	SPEC-030
RF-07	Al crear un establecimiento, el sistema calculará sus coordenadas a partir de la dirección. Si el cálculo falla, el establecimiento se guardará sin coordenadas y la ficha lo indicará.	SPEC-060
RF-08	La ficha mostrará los datos del establecimiento, su ubicación, la valoración media con el número de valoraciones, la puntuación por criterio, la galería y las valoraciones individuales.	SPEC-030, 040, 050, 060
RF-09	El creador o un administrador podrán editar el establecimiento con las mismas validaciones que el alta. Las coordenadas solo se recalcularán si cambia la dirección, el municipio o la provincia.	SPEC-080

ID	Requisito	Origen
RF-10	El creador o un administrador podrán eliminar el establecimiento, previa confirmación. Se eliminarán también sus fotografías, en el servicio de almacenamiento y en la base de datos, y sus valoraciones.	SPEC-080
RF-11	El listado permitirá buscar por nombre (coincidencia parcial, sin distinguir mayúsculas), filtrar por categoría, provincia y municipio, y ordenar por fecha ascendente o descendente, nombre ascendente o descendente y valoración. Los filtros viajarán en la dirección de la página.	SPEC-070

Tabla 3.3. *Requisitos funcionales de valoraciones y fotografías.*

ID	Requisito	Origen
RF-12	El usuario registrado podrá valorar un establecimiento puntuando los ocho criterios de 0 a 5 y añadiendo un comentario opcional. La valoración se guardará completa o no se guardará.	SPEC-040
RF-13	Los criterios de ascensor y aparcamiento admitirán «No aplicable». Esa opción no se permitirá en los demás y no contará en la media.	Ampliación de SPEC-040
RF-14	Un usuario solo podrá emitir una valoración por establecimiento.	SPEC-040
RF-15	El autor de una valoración o un administrador podrán eliminarla. Las valoraciones no se editan.	SPEC-040 y ampliación posterior
RF-16	El usuario registrado podrá subir fotografías a cualquier establecimiento en formato JPEG, PNG o WebP, de hasta 5 MB y con un mínimo de 800 × 600 píxeles. Cada establecimiento tendrá como máximo diez fotografías.	SPEC-050
RF-17	La primera fotografía de un establecimiento será la principal. Quien subió una fotografía, o un administrador, podrá designarla como principal o eliminarla. Si se elimina la principal, pasará a serlo la más antigua de las restantes.	SPEC-050 y ampliación posterior

Tabla 3.4. *Requisitos funcionales de mapa, panel, administración e información legal.*

ID	Requisito	Origen
RF-18	La ficha mostrará un mapa con la ubicación del establecimiento o, si no hay coordenadas, un aviso. Los datos de ubicación existirán también en texto.	SPEC-060
RF-19	Una página de mapa global mostrará los establecimientos con coordenadas, con un resumen textual de su número. Cada marcador abrirá una ventana con nombre, categoría, municipio, valoración, fotografía principal y enlace a la ficha.	SPEC-090

ID	Requisito	Origen
RF-20	El panel principal mostrará los totales de establecimientos, usuarios, valoraciones y fotografías, los seis últimos establecimientos, los seis mejor valorados y accesos rápidos.	SPEC-100
RF-21	El administrador podrá crear, renombrar y eliminar categorías. El nombre será único y no se podrá eliminar una categoría con establecimientos asociados.	Panel de administración
RF-22	El administrador podrá editar o eliminar los establecimientos, valoraciones y fotografías de otros usuarios.	SPEC-080 y panel de administración
RF-23	La aplicación ofrecerá, sin necesidad de sesión, una página de presentación y las páginas de aviso legal, política de privacidad y política de cookies.	Páginas informativas

El catálogo no incluye la recuperación de contraseña. Se consideró durante el planteamiento inicial y no llegó a implementarse en esta versión; sus consecuencias de seguridad se tratan en el capítulo 7. La consulta anónima de fichas y mapa, tampoco incluida, se recoge en el capítulo 13.

3.2.3 Reglas de autorización

Los permisos que aparecen dispersos en los requisitos anteriores siguen una regla única, que se resume en la tabla 3.5. Su implementación se describe en el capítulo 4.

Tabla 3.5. *Operaciones y actores autorizados.*

Operación	Visitante	Usuario registrado	Autor del recurso	Administrador
Ver presentación y páginas legales	Sí	Sí	Sí	Sí
Consultar fichas, listado, mapa y panel	No	Sí	Sí	Sí
Crear establecimiento, valoración o fotografía	No	Sí	Sí	Sí
Editar o eliminar establecimiento	No	No	Sí	Sí
Eliminar valoración	No	No	Sí	Sí
Eliminar o designar como principal una fotografía	No	No	Sí	Sí
Gestionar categorías	No	No	No	Sí

Dos detalles de la tabla son decisiones deliberadas. La primera es que el creador de un establecimiento no puede eliminar las fotografías o valoraciones de otros usuarios sobre su ficha: cada recurso pertenece a quien lo aportó, de modo que el creador de un local no puede

borrar una valoración negativa. La segunda es que cualquier usuario registrado puede subir fotografías a un establecimiento ajeno, ya que la información útil para otros usuarios no debería depender de que el creador de la ficha tenga tiempo de completarla.

3.3 Requisitos no funcionales

Las especificaciones base fijaron seis cualidades transversales: seguridad, accesibilidad, mantenibilidad, facilidad de despliegue y de prueba, y adaptación a distintos tamaños de pantalla. Las especificaciones no cuantificaron esos objetivos, y por ello este apartado los concreta en requisitos verificables y declara en cada caso cuánto cubre la versión entregada. Facilidad de despliegue y de prueba se tratan en los apartados de mantenibilidad y en los capítulos 10 y 11.

3.3.1 Seguridad

Tabla 3.6. *Requisitos no funcionales de seguridad.*

ID	Requisito	Estado	Capítulo
RNF-S1	Las contraseñas se almacenarán con una función de derivación adaptativa y nunca en claro.	Cubierto	Cap. 7
RNF-S2	Toda entrada se validará en el servidor, con independencia de la validación del cliente.	Parcial	Cap. 7
RNF-S3	Las operaciones sobre recursos se autorizarán en el servidor, comparando con el propietario persistido y no con datos enviados por el cliente.	Cubierto	Cap. 7
RNF-S4	La sesión estará firmada y tendrá una duración limitada y definida.	Parcial	Cap. 7
RNF-S5	La aplicación se protegerá frente a inyección de consultas y a peticiones falsificadas desde otros orígenes.	Cubierto	Cap. 7
RNF-S6	Se limitará el número de intentos de autenticación por origen o por cuenta.	No cubierto	Cap. 7
RNF-S7	Las respuestas incluirán cabeceras HTTP de seguridad.	No cubierto	Cap. 7
RNF-S8	Se registrarán los eventos relevantes de seguridad.	No cubierto	Cap. 7

La validación es parcial porque todos los datos se comprueban en el servidor con Zod, pero varios campos de texto libre, como la dirección, la descripción y el comentario, no tienen longitud máxima. La sesión es parcial porque está firmada, pero su duración es la que establece por defecto la biblioteca de autenticación, treinta días, sin una política propia. El capítulo 7 desarrolla el detalle, los controles pendientes y el riesgo residual de cada uno.

3.3.2 Rendimiento

Las especificaciones base solo exigen aprovechar las capacidades nativas del framework y evitar optimizaciones prematuras. No se fijaron tiempos de respuesta ni umbrales de carga, y tampoco se han realizado pruebas de carga, de modo que el rendimiento no puede afirmarse como una propiedad medida. Lo que se verifica son decisiones de diseño con efecto sobre el rendimiento.

Tabla 3.7. *Requisitos no funcionales de rendimiento.*

ID	Requisito	Estado
RNF-R1	Las consultas leerán solo los campos que necesitan las vistas, sin cargar relaciones completas.	Cubierto
RNF-R2	El mapa global se obtendrá con una única lectura de datos.	Cubierto
RNF-R3	Las bibliotecas de mapas se cargarán de forma diferida para no penalizar las páginas que no las usan.	Cubierto
RNF-R4	Las fotografías no se almacenarán en el servidor de la aplicación, sino en un servicio externo especializado.	Cubierto
RNF-R5	Se definirán y medirán objetivos cuantitativos de tiempo de respuesta.	No cubierto

3.3.3 Accesibilidad

La accesibilidad es el requisito transversal más coherente con el propósito del producto: una aplicación que informa sobre barreras no puede ponerlas. Las especificaciones lo formulan como cumplimiento de «buenas prácticas WCAG», sin fijar un nivel de conformidad. Esta memoria no declara conformidad con un nivel concreto, porque no se ha realizado una auditoría completa con tecnologías de apoyo; el capítulo 8 evalúa la aplicación frente a los criterios de las Pautas de Accesibilidad para el Contenido Web 2.2 (World Wide Web Consortium [W3C], 2023) e indica qué se ha comprobado y cómo.

Tabla 3.8. *Requisitos no funcionales de accesibilidad.*

ID	Requisito	Estado
RNF-A1	La estructura de las páginas será semántica y el idioma del documento estará declarado.	Cubierto
RNF-A2	Habrá un enlace para saltar al contenido principal y los controles serán elementos nativos o componentes con soporte de teclado.	Cubierto
RNF-A3	Los errores de formulario se asociarán al campo y se anunciarán a tecnologías de apoyo.	Cubierto
RNF-A4	Ninguna información se transmitirá solo por el mapa: los datos de ubicación existirán también en texto.	Cubierto
RNF-A5	La interfaz del mapa no capturará el desplazamiento de la página y la ventana de cada marcador será alcanzable con teclado.	Parcial

ID	Requisito	Estado
RNF-A6	Las pruebas automáticas incluirán una comprobación de accesibilidad.	Parcial

La comprobación automática es parcial porque la herramienta axe-core se ejecuta solo sobre la página de inicio y la de aviso legal. Las páginas autenticadas no están cubiertas por esa prueba. RNF-A5 es parcial porque el mapa no captura el desplazamiento de la página, pero los marcadores no reciben el foco del teclado, como se comprueba en el capítulo 8.

3.3.4 Mantenibilidad

Tabla 3.9. *Requisitos no funcionales de mantenibilidad.*

ID	Requisito	Estado	Capítulo
RNF-M1	El código estará escrito en TypeScript con comprobación estricta de tipos.	Cubierto	Cap. 4
RNF-M2	El código se organizará por funcionalidades, con cada módulo independiente en acciones, servicios, esquemas y componentes.	Cubierto	Cap. 4
RNF-M3	Las reglas de negocio residirán en servicios independientes de la interfaz y de las acciones.	Cubierto	Cap. 4
RNF-M4	Un conjunto de pruebas automáticas se ejecutará antes de cada integración.	Cubierto	Cap. 10
RNF-M5	El estilo se verificará de forma automática con analizador estático y formateador.	Cubierto	Cap. 11
RNF-M6	Las decisiones de diseño con consecuencias estructurales quedarán documentadas.	Cubierto	Cap. 4
RNF-M7	Los umbrales de cobertura de pruebas se fijarán y harán cumplir en la integración continua.	No cubierto	Cap. 10

3.3.5 Escalabilidad

El producto se concibió para una comunidad pequeña y no se diseñó para un volumen alto de datos. Los requisitos de escalabilidad se formulan, por ello, como condiciones que no cierren el camino a un crecimiento posterior, y no como capacidad demostrada.

Tabla 3.10. *Requisitos no funcionales de escalabilidad.*

ID	Requisito	Estado
RNF-E1	La aplicación no mantendrá estado en el servidor, de modo que pueda ejecutarse en varias instancias.	Cubierto
RNF-E2	La base de datos evolucionará mediante migraciones versionadas.	Cubierto
RNF-E3	La arquitectura admitirá nuevas funcionalidades sin modificar los módulos existentes.	Cubierto

ID	Requisito	Estado
RNF-E4	El listado soportará grandes volúmenes de establecimientos mediante paginación.	No cubierto
RNF-E5	La ordenación por valoración se resolverá en la base de datos.	No cubierto

Los dos últimos requisitos son limitaciones conocidas. El listado devuelve todos los establecimientos que cumplen el filtro, porque la paginación se excluyó expresamente del alcance, y la ordenación por valoración se realiza en memoria tras leer las valoraciones, al ser la media un valor calculado y no almacenado. Ambas decisiones simplifican la primera versión y son correctas con el volumen de datos esperado; dejarían de serlo al crecer la comunidad. La evolución prevista, una media materializada o una agregación en base de datos junto con paginación, se recoge en el capítulo 13.

3.3.6 Experiencia de usuario

Tabla 3.11. *Requisitos no funcionales de experiencia de usuario.*

ID	Requisito	Estado
RNF-U1	La interfaz se adaptará a pantallas de distintos tamaños, desde teléfonos hasta equipos de escritorio.	Cubierto
RNF-U2	Todos los textos de la interfaz y los mensajes de error estarán en español y serán comprensibles sin conocimientos técnicos.	Cubierto
RNF-U3	Las acciones destructivas pedirán confirmación.	Cubierto
RNF-U4	Toda lista vacía explicará la situación y ofrecerá una acción siguiente.	Cubierto
RNF-U5	Los resultados de búsqueda serán enlazables y compatibles con el historial del navegador.	Cubierto

El requisito RNF-U1 se verificó mediante pruebas manuales en distintos tamaños de pantalla, sin una matriz formal de dispositivos. Los criterios de usabilidad no se contrastaron con usuarios reales, limitación que el capítulo 12 recoge.

3.4 Restricciones de diseño

Además de los requisitos, el proyecto se desarrolló bajo restricciones que acotaron las soluciones posibles. Se fijaron al comienzo y se respetaron durante todo el desarrollo.

- Todo el código se escribe en TypeScript; no se usa JavaScript sin tipos.
- No se emplean librerías de gestión global del estado ni CSS tradicional: el estilo se resuelve con utilidades de Tailwind.
- No se implementa funcionalidad que no esté especificada, y no se incorporan dependencias sin necesidad.

- La lógica no se duplica: las reglas de permisos y el cálculo de medias residen cada uno en un único lugar y se reutilizan.
- Ante varias soluciones válidas se elige la más sencilla, la más mantenible y la más alineada con el framework.
- La inteligencia artificial es una herramienta de desarrollo y no forma parte del producto. Su uso se trata en el capítulo 9.

3.5 Síntesis

El catálogo comprende 23 requisitos funcionales, todos implementados, y 36 no funcionales, de los que 25 están cubiertos, 4 lo están parcialmente y 7 no lo están. Los requisitos funcionales se corresponden con el alcance del producto mínimo viable. Los no funcionales muestran el perfil de la versión entregada: sólida en autorización en servidor, validación, organización del código y accesibilidad estructural, y con carencias en endurecimiento de seguridad, medición del rendimiento y escalabilidad del listado. Esas carencias se tratan como deuda técnica conocida y se desarrollan en los capítulos 7, 12 y 13, no como incumplimientos ocultos.

4. Arquitectura del sistema

4.1 Visión general y criterios de diseño

Accesibilidad 360 es una aplicación web monolítica, desarrollada con Next.js en su modalidad App Router, que ejecuta en el mismo proyecto la interfaz, la lógica de negocio y el acceso a datos. La decisión de no separar una API y un cliente independientes se tomó por la escala del proyecto: un único desarrollador, un conjunto reducido de funcionalidades y la prioridad de las especificaciones base, que ordenan elegir ante varias soluciones válidas la más sencilla, la más mantenible y la más alineada con el framework. Un monolito modular ofrece esa simplicidad sin cerrar la evolución posterior, porque la organización interna ya separa las responsabilidades que habría que dividir en un futuro.

Los criterios que han guiado el diseño son cuatro. El primero es que toda regla de negocio resida en un lugar único y reutilizable, de modo que la interfaz no decida nada que importe para la integridad de los datos. El segundo es que la validación y la autorización se ejecuten siempre en el servidor, y que las comprobaciones del cliente se limiten a mejorar la experiencia. El tercero es que las dependencias apunten en una sola dirección, de la interfaz hacia los servicios y de estos hacia los datos. El cuarto es que cada módulo funcional pueda leerse, probarse y modificarse sin recorrer el resto del código.

La figura 4.1 resume los componentes y sus relaciones. Los apartados siguientes los describen de fuera hacia dentro.

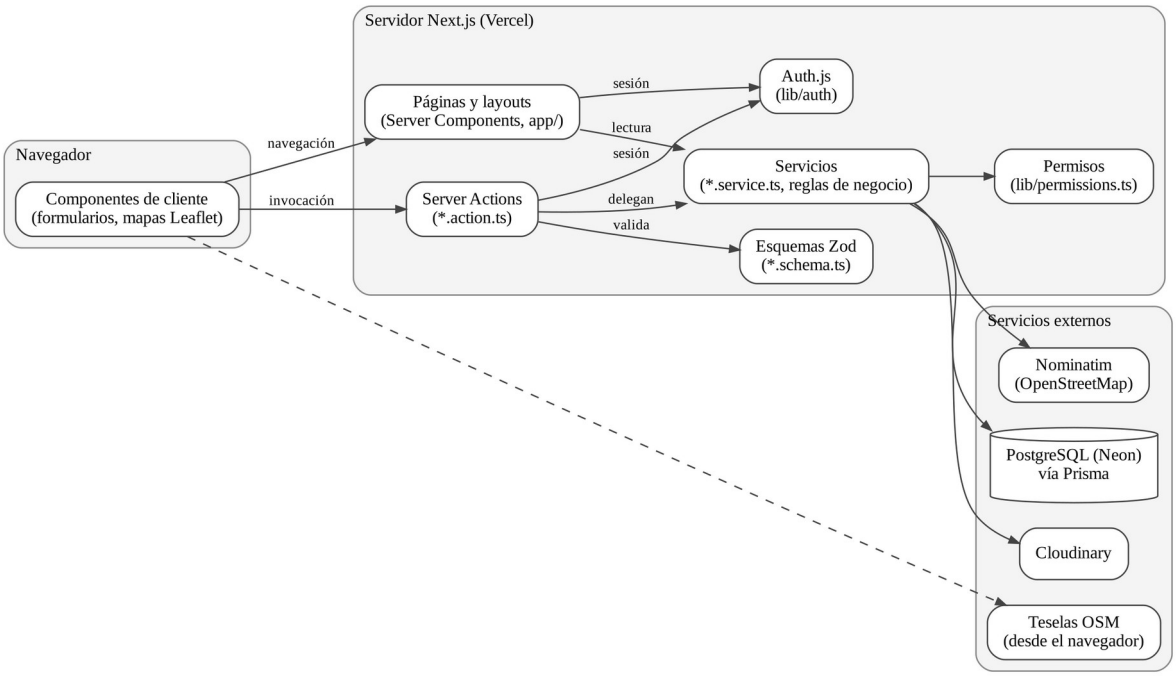


Figura 4.1. Componentes del sistema y dirección de las dependencias.

4.2 Pila tecnológica

La tabla 4.1 recoge las tecnologías principales y el motivo de su elección. Las versiones proceden del manifiesto del proyecto.

Tabla 4.1. *Tecnologías principales y justificación.*

Capa	Tecnología	Motivo de la elección
Framework	Next.js 15 (App Router) y React 19	Unifica interfaz, servidor y rutas en un proyecto. Los componentes de servidor y las Server Actions evitan construir una API REST propia para las operaciones de la aplicación.
Lenguaje	TypeScript en modo estricto	Los tipos detectan en compilación errores de contrato entre capas, un riesgo mayor en un proyecto con asistencia de IA donde el código se revisa pero no se escribe línea a línea.
Estilos y componentes	Tailwind CSS 4, shadcn/ui, Lucide	Utilidades en lugar de hojas de estilo propias, y componentes de código abierto copiados al proyecto, sin dependencia de una biblioteca cerrada.
Formularios y validación	React Hook Form y Zod 3	Un mismo esquema de Zod sirve para validar en el navegador y, de nuevo, en el servidor.
Datos	PostgreSQL con Prisma 6	Modelo relacional con integridad referencial para entidades fuertemente relacionadas. Prisma ofrece consultas tipadas y parametrizadas y migraciones versionadas.
Autenticación	Auth.js v5 con proveedor de credenciales, bcrypt	Estándar del ecosistema Next.js. Evita implementar firma de sesiones y gestión de cookies a mano.
Imágenes	Cloudinary (SDK oficial v2)	El sistema de archivos del alojamiento es efímero y no sirve para almacenar fotografías.
Mapas	Leaflet, react-leaflet y teselas de OpenStreetMap; Nominatim	Alternativa abierta y sin clave de pago a los mapas comerciales, coherente con el origen colaborativo de los datos.
Alojamiento	Vercel y Neon	Despliegue directo desde el repositorio, con PostgreSQL gestionado. Se describen en el capítulo 11.

Dos decisiones merecen comentario. La primera es el uso de Auth.js en lugar de una solución propia de sesiones: la firma y verificación de tokens y el manejo de cookies son un terreno donde un error de implementación tiene consecuencias de seguridad, y la biblioteca los resuelve con revisión de una comunidad amplia. La segunda es que la versión de Auth.js utilizada (5.0.0-

beta.32) es una versión beta. Es una dependencia que debe vigilarse hasta que exista una versión estable, y se retoma en el capítulo 7.

4.3 Organización del código: arquitectura por funcionalidades

4.3.1 Estructura de directorios

El código se organiza según el patrón conocido como arquitectura basada en funcionalidades (feature-based). En lugar de agrupar los ficheros por tipo técnico, de modo que todos los componentes estén juntos y todos los servicios en otro lugar, se agrupan por área funcional. Cada área contiene sus propios componentes, acciones, servicios, esquemas y pruebas. La estructura de primer nivel es la siguiente.

- `app/`: rutas y layouts de Next.js. Contiene solo composición de páginas y delega la lógica.
- `features/`: módulos funcionales (`auth`, `establishments`, `reviews`, `photos`, `profile`, `dashboard`).
- `components/`: componentes compartidos sin lógica de negocio, como la interfaz base, el diseño general, la navegación y las tarjetas de estadísticas.
- `lib/`: infraestructura transversal: autenticación, cliente de base de datos, permisos y utilidades.
- `prisma/`: esquema, migraciones y datos iniciales.
- `tests/`: pruebas que no pertenecen a un módulo concreto, incluidas las de extremo a extremo.

La razón de preferir esta organización es la cohesión: modificar la valoración de establecimientos afecta a los ficheros de `features/reviews`, y no obliga a tocar cinco directorios técnicos. Dentro de cada módulo se repite la misma estructura, lo que reduce el esfuerzo de orientarse en un módulo desconocido y permite aplicar reglas uniformes.

4.3.2 Módulos

Tabla 4.2. *Módulos funcionales y responsabilidades.*

Módulo	Responsabilidad	Rutas asociadas
`auth`	Registro, inicio y cierre de sesión, verificación de credenciales.	`/register`, `/login`
`establishments`	Alta, edición, eliminación, listado y filtros, geocodificación, mapas y gestión de categorías.	`/establishments`, `/establishments/[id]`, `/establishments/new`, `/establishments/[id]/edit`, `/map`, `/admin`
`reviews`	Valoraciones por criterios y catálogo de criterios.	Integrado en la ficha

Módulo	Responsabilidad	Rutas asociadas
`photos`	Subida, galería, fotografía principal y almacenamiento externo.	Integrado en la ficha
`profile`	Edición del nombre y cambio de contraseña.	`/profile`
`dashboard`	Agregados del panel principal.	`/dashboard`

El mapa y la administración de categorías no tienen módulo propio y residen en `establishments`. La decisión es deliberada: ambos operan sobre el mismo agregado de datos y no tienen reglas independientes. Crear módulos separados habría duplicado conceptos para obtener una apariencia de organización sin ganancia funcional. Si el mapa creciera, por ejemplo con filtros o rutas, la separación pasaría a estar justificada.

4.3.3 Dependencias entre módulos

Las dependencias entre módulos son pocas y forman un grafo sin ciclos. `dashboard` reutiliza el cálculo de la media y el formato de puntuaciones de `establishments`, para no duplicar la fórmula, y `establishments` utiliza el servicio de almacenamiento de `photos` para eliminar las imágenes de un establecimiento al borrarlo. Ningún otro módulo importa de otro. La única dependencia desde `lib` hacia `features` es la de la autenticación, que se explica en el apartado 4.7.

4.4 Arquitectura cliente-servidor

La aplicación sigue el modelo de componentes de servidor y de cliente de React 19. Por defecto, páginas y layouts son componentes de servidor: se ejecutan en el servidor, leen datos llamando directamente a los servicios y envían al navegador HTML ya generado. Esto significa que no hay una capa de API intermedia para las lecturas, ni peticiones del navegador a un servicio de datos para mostrar un listado o una ficha.

Los componentes de cliente se reservan para lo que necesita interactividad: formularios, botones con confirmación, la galería y los mapas. De los ficheros de interfaz, 22 declaran `"use client"`. Los mapas son el caso más delicado: Leaflet necesita el DOM del navegador, de modo que se cargan de forma diferida y sin renderizado en servidor, mostrando un mensaje de carga mientras tanto. Esta carga diferida evita además descargar la biblioteca en las páginas que no muestran mapa.

Las escrituras se realizan con Server Actions, funciones de servidor que el cliente invoca como si fueran locales y que Next.js expone mediante peticiones POST gestionadas por el framework. La aplicación no define ninguna ruta de API propia. El único manejador de rutas es el de Auth.js, que implementa el protocolo de autenticación y no contiene lógica de producto.

El grupo de rutas se estructura en tres partes, que no afectan a la dirección web. `(auth)` agrupa registro e inicio de sesión. `(protected)` agrupa todo el contenido que exige sesión: panel, establecimientos, mapa, perfil y administración. Las páginas de presentación, aviso legal, privacidad y cookies están fuera de ambos grupos y son públicas.

4.5 Capas y responsabilidades

Dentro de cada módulo se distinguen cinco responsabilidades, cada una con una convención de nombre de fichero que facilita localizarla y comprobarla de forma automática.

Tabla 4.3. *Capas, ficheros y reglas.*

Capa	Fichero	Responsabilidad	Prohibido
Presentación	`components/*.tsx`, páginas	Mostrar datos y recoger entrada.	Acceder a la base de datos; decidir permisos.
Acciones	`*.action.ts`	Punto de entrada de las mutaciones: obtiene la sesión, valida la entrada, llama al servicio y traduce errores a mensajes.	Contener reglas de negocio; exportar algo que no sea una función asíncrona.
Esquemas	`*.schema.ts`	Definir la forma y límites de los datos de entrada con Zod.	Depender de la interfaz.
Servicios	`*.service.ts`	Reglas de negocio, autorización contra datos persistidos y acceso a datos.	Conocer la interfaz o la sesión de la petición.
Datos	`lib/db/prisma.ts`	Cliente único de Prisma.	Ser usado fuera de los servicios.

Las dependencias permitidas van de arriba abajo: las páginas y acciones usan servicios y esquemas, y los servicios usan Prisma y los demás servicios de su módulo. Esta regla se cumple en el código actual, que se ha comprobado mediante la búsqueda de importaciones: el cliente de Prisma solo se importa en diez ficheros de servicio, y ningún componente, página ni acción accede directamente a la base de datos. La regla no la hace cumplir una herramienta automática, como una restricción de importaciones en el analizador estático, sino la revisión del código; esa es una mejora posible.

4.5.1 Reparto de responsabilidades

El reparto responde a una pregunta: qué parte del sistema es la última responsable de una decisión. La respuesta, en este diseño, es siempre el servicio. La interfaz puede ocultar el botón «Eliminar» a quien no tenga permiso, pero quien decide si se elimina es el servicio, comparando el actor con el propietario almacenado en la base de datos. Del mismo modo, la interfaz puede avisar de que una contraseña es corta, pero es el esquema, ejecutado en el servidor, el que la

rechaza. Esta jerarquía hace que la ocultación de controles sea una cuestión de usabilidad y no de seguridad.

Un ejemplo ilustra la cadena completa. Para crear una valoración, el formulario recoge las puntuaciones y llama a la acción. La acción obtiene el identificador de usuario de la sesión, nunca del formulario, y valida los datos con el esquema. El servicio comprueba que el establecimiento exista, que el usuario no haya valorado ya, que se hayan puntuado todos los criterios y que «No aplicable» solo se use donde se permite, y guarda la valoración y sus puntuaciones en una transacción. Cada comprobación tiene su sitio, y ninguna depende de que la anterior se haya hecho bien en el cliente.

4.5.2 Acoplamiento e inversión de dependencias

Los servicios dependen directamente de Prisma. No se ha introducido una capa de repositorios con interfaces que invierta esa dependencia, como pediría una aplicación estricta del principio de inversión de dependencias. Es una decisión consciente. Con un único almacén de datos, una única implementación y un equipo de una persona, una capa de repositorios añadiría ficheros e indirecciones sin un caso de uso que las justifique, y contradiría el criterio de simplicidad fijado al inicio. El coste se paga en las pruebas, que deben simular el cliente de Prisma en lugar de sustituir una interfaz pequeña, y en que cambiar de base de datos exigiría tocar todos los servicios. Se asume ese coste porque el cambio de base de datos no figura entre los escenarios previstos.

Sí hay inversión de responsabilidades en dos puntos concretos. El primero es la autorización: la regla de quién puede gestionar un recurso se define una sola vez, en `lib/permissions.ts`, y cada servicio la aplica contra su propio recurso. El segundo es la autenticación, descrita a continuación, en la que el marco de autenticación llama a un servicio del dominio y no al revés.

4.6 Server Actions

Las 16 Server Actions de la aplicación siguen el mismo patrón, que se muestra en la tabla 4.4 y que permite razonar sobre cualquiera de ellas sin leer su código.

Tabla 4.4. *Secuencia común de una Server Action.*

Paso	Descripción
1	Obtener la sesión con `auth()` y rechazar si no existe. Las 13 acciones que operan sobre datos de un usuario autenticado lo hacen; las tres restantes son el registro, el inicio y el cierre de sesión.
2	Validar la entrada, tipada como desconocida, con el esquema de Zod. Si falla, devolver errores por campo.
3	Llamar al servicio con los datos validados y los datos de la sesión, nunca los del cliente, para identificar al actor.

Paso	Descripción
4	Traducir los errores tipados del servicio a mensajes comprensibles para el usuario, asociados a un campo cuando procede.
5	Revalidar las rutas cuyos datos han cambiado y devolver un resultado tipado.

El resultado es siempre un objeto con el campo `success` y, si falla, un mensaje, de modo que el cliente trata los errores esperados como valores y no como excepciones. Los servicios, por su parte, lanzan errores tipados con un código, por ejemplo `DUPLICATE_REVIEW` o `FORBIDDEN_PHOTO`, de forma que la acción puede distinguir un fallo de permisos de uno de validación sin analizar texto.

4.6.1 Convención de ficheros

Next.js trata como función de servidor cada exportación de un fichero marcado con `"use server"`. Una exportación que no sea una función asíncrona, como un tipo, una constante o un esquema, provoca errores que pueden no manifestarse en desarrollo. Por ello se adoptó una regla única, registrada como decisión de arquitectura: los ficheros `*.action.ts` exportan exclusivamente funciones asíncronas, y los tipos, constantes y esquemas residen en ficheros hermanos (`*.types.ts`, `*.schema.ts`, `*.constants.ts`). La regla se derivó de un fallo real ocurrido durante el despliegue, que se describe en el capítulo 11, y se ha aplicado de manera uniforme a las 16 acciones.

4.7 Autenticación y sesiones

4.7.1 Diseño

Se utiliza Auth.js con proveedor de credenciales y sesiones basadas en token JWT. La alternativa, sesiones almacenadas en base de datos, habría añadido una consulta por petición y una tabla de sesiones que gestionar. El token JWT se firma en el servidor, se transporta en una cookie y contiene el identificador del usuario, su nombre, su correo y su rol. El rol se incluye en el token para evitar consultar la base de datos en cada comprobación de administración; el coste es que un cambio de rol no se aplica hasta que la sesión se renueva. No existe actualmente una función para cambiar el rol de un usuario, por lo que esa situación no se presenta.

La configuración se divide en dos ficheros por una restricción técnica. El código de `middleware.ts` se ejecuta en un entorno de ejecución restringido (Edge), que no puede cargar Prisma ni bcrypt. Por ello `auth.config.ts` contiene solo lo compatible con ese entorno, y `auth.ts` lo amplía con el proveedor de credenciales, que sí necesita acceso a la base de datos.

4.7.2 Flujo de autenticación

La figura 4.2 muestra los dos flujos principales: el inicio de sesión y el acceso a rutas protegidas.

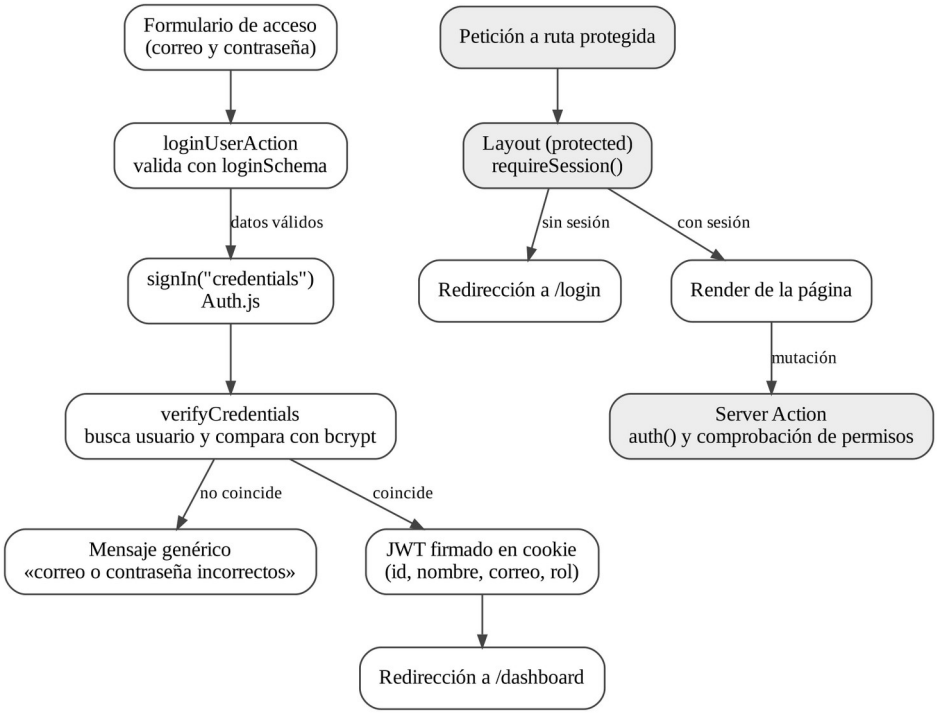


Figura 4.2. Flujo de inicio de sesión y de acceso a rutas protegidas.

En el inicio de sesión, el formulario envía el correo y la contraseña a la acción de login, que los valida con el esquema, deliberadamente mínimo: la contraseña solo se exige presente, de modo que cualquier contraseña incorrecta reciba la misma respuesta genérica. A continuación llama a `signIn` de Auth.js, que invoca `authorize`, y este delega en el servicio `verifyCredentials`. El servicio busca el usuario por correo y compara la contraseña con el hash almacenado mediante bcrypt. Devuelve nulo tanto si el correo no existe como si la contraseña es incorrecta, y la acción responde con un único mensaje que no distingue ambos casos. Si las credenciales son válidas, se emite la cookie con el token y se redirige al panel.

Que Auth.js exija definir `authorize` dentro de su propia inicialización crea una dependencia desde `lib/auth` hacia `features/auth`, contraria al sentido normal de las dependencias. Se resolvió limitando `lib/auth` a cablear: el fichero no contiene reglas, importa el servicio de verificación y el esquema, y estos quedan aislados y probados dentro de su módulo. Es la única excepción a la dirección de dependencias y consta como decisión documentada.

El registro es independiente del inicio de sesión. Crea el usuario con rol de usuario, almacena únicamente el hash con coste 12 y no inicia sesión automáticamente. La unicidad del correo se garantiza dos veces: con una comprobación previa para dar un mensaje claro y con la restricción de unicidad de la base de datos, que cubre la carrera entre dos registros simultáneos.

4.7.3 Protección de rutas

Todo el contenido que exige sesión cuelga del layout del grupo `(protected)`, que llama a `requireSession()` y redirige a `/login` si no hay sesión. Es el único punto de protección de

páginas: ninguna página lo reimplementa. El middleware se limita a refrescar la sesión y no redirige.

La consecuencia es que las páginas están protegidas como conjunto y las acciones no. Una Server Action no pasa por el layout, ya que se invoca por una petición propia, y por esa razón cada acción que opera sobre datos del usuario vuelve a obtener la sesión y verifica los permisos por sí misma. Esta duplicación deliberada es una defensa en profundidad, y también el motivo de que las 13 acciones con datos de usuario llamen a `auth()`. El capítulo 7 evalúa las implicaciones de depender del layout y no del middleware.

4.7.4 Actualización de la sesión

El nombre del usuario forma parte del token. Cuando el usuario lo cambia desde el perfil, la acción guarda el valor en la base de datos y pide a Auth.js que actualice el token, de modo que la interfaz muestra el nombre nuevo sin cerrar la sesión y sin leer la base de datos en cada petición.

4.8 Autorización

La autorización se basa en una regla simple y única, que se aplica a establecimientos, valoraciones y fotografías: puede gestionar un recurso quien lo creó o un administrador. La función `canManageOwnerOrAdmin` en `lib/permissions.ts` implementa la regla y `isAdmin` la comprobación de administración. Los permisos de cada operación están en la tabla 3.5.

Tres decisiones de diseño sustentan el esquema. La primera es que el propietario se compara siempre con el valor persistido, que el servicio lee en la base de datos antes de decidir, y nunca con un identificador enviado por el cliente. La segunda es que el identificador del actor procede de la sesión del servidor. La tercera es que la página de edición no revela información: tanto un establecimiento inexistente como uno ajeno producen la misma respuesta de recurso no encontrado, de modo que un tercero no puede averiguar qué identificadores existen. Las acciones de eliminación y edición, en cambio, devuelven un mensaje de falta de permiso cuando el actor no está autorizado, ya que quien las invoca ha llegado hasta la ficha y el recurso es visible.

La aplicación no incluye ninguna función para asignar el rol de administrador, por lo que se establece directamente en la base de datos. Ello reduce la superficie de ataque, ya que no existe un flujo de elevación de privilegios que proteger, y es suficiente para el alcance del proyecto, aunque no escalaría a una organización con varios administradores.

4.9 Integración con servicios externos

La aplicación depende de tres servicios externos, y para cada uno se tomó una decisión sobre qué ocurre cuando falla.

Cloudinary almacena las fotografías. La subida se realiza siempre desde el servidor, con el secreto de la API en variables de entorno, y en la base de datos solo se guarda la dirección y el identificador público que permite eliminar la imagen después. Una fotografía se sube primero a Cloudinary y solo se registra si cumple las dimensiones mínimas; en caso contrario se elimina del servicio externo. En la eliminación se actúa a la inversa: se borra primero en Cloudinary y después en la base de datos, de modo que un fallo del servicio externo deja el registro intacto y la operación puede repetirse, en lugar de dejar una imagen huérfana sin referencia. El mismo orden se sigue al eliminar un establecimiento. El detalle de este proceso se expone en el capítulo 6.

Nominatim convierte la dirección en coordenadas. La consulta se hace desde el servidor, con un plazo máximo de cinco segundos, un identificador de aplicación en la cabecera y la dirección construida con los nombres oficiales de municipio y provincia, no con texto libre. Cualquier fallo, sea de red, de plazo o por falta de resultados, devuelve nulo y el establecimiento se guarda sin coordenadas. Se eligió que la geocodificación no sea imprescindible para crear una ficha porque el servicio público de Nominatim no ofrece garantías de disponibilidad, y una caída del mismo no debe impedir contribuir.

OpenStreetMap sirve las teselas del mapa, que el navegador solicita directamente. La atribución es obligatoria y se muestra en el mapa.

El acceso a la base de datos se hace a través de un cliente único de Prisma, creado una vez y conservado en el objeto global durante el desarrollo, para no agotar las conexiones con la recarga en caliente.

4.10 Modelo de datos

El modelo de datos se diseñó junto con la arquitectura y condiciona varias de sus decisiones, como la media calculada en consulta o las eliminaciones en cascada. Su descripción completa, con entidades, relaciones, restricciones e índices, se presenta en el capítulo 5.

4.11 Decisiones de arquitectura registradas

Las decisiones con consecuencias estructurales se documentaron en el momento de tomarlas. La tabla 4.5 resume las que afectan a la arquitectura. Las políticas de eliminación en cascada se tratan en el capítulo 5, por afectar al modelo de datos.

Tabla 4.5. *Decisiones de arquitectura.*

Decisión	Contexto	Resolución	Consecuencia
Verificación de credenciales fuera de `lib/auth`	Auth.js exige definir `authorize` en su inicialización, y la lógica de credenciales es de dominio.	`lib/auth` solo cablea y delega en un servicio de `features/auth`.	Excepción única a la dirección de dependencias.

Decisión	Contexto	Resolución	Consecuencia
Almacenamiento externo de fotografías	El sistema de archivos del alojamiento es efímero.	Cloudinary, subida desde el servidor, y se guardan solo la dirección y el identificador.	Dependencia de un servicio externo y de su plan gratuito.
Regla única para Server Actions	Las exportaciones no asíncronas de un fichero `"use server"` provocan fallos de despliegue.	Los ficheros de acción solo exportan funciones asíncronas.	Más ficheros pequeños por módulo, con estructura uniforme.

4.12 Limitaciones y síntesis

La arquitectura resuelve las necesidades del producto mínimo viable con pocas piezas y reglas claras, pero tiene limitaciones que conviene declarar.

- La protección de páginas descansa en un layout y no en el middleware, por lo que cada nueva ruta protegida debe colocarse dentro del grupo correspondiente. Un error humano al crearla fuera del grupo la dejaría abierta, y ninguna comprobación automática lo detectaría.
- La dirección de las dependencias se respeta por disciplina y revisión, no mediante reglas automáticas.
- Los servicios están acoplados a Prisma, lo que encarece las pruebas unitarias y fija el almacén de datos.
- La ordenación por valoración y la media se calculan en memoria, solución adecuada para el volumen previsto y no para uno grande (apartado 3.3.5).
- No existe una capa de registro ni de observabilidad. Los errores inesperados no se registran de forma estructurada.

Como contrapartida, la separación en capas, la comprobación de permisos en el servicio y el patrón uniforme de las acciones hacen que cada comportamiento sea localizable y verificable, propiedad que el capítulo 10 aprovecha para probar las reglas de negocio sin base de datos real.

5. Diseño de la base de datos

5.1 Criterios de diseño

El esquema íntegro, con el diagrama y la relación de migraciones, se recoge en el anexo D.

El dominio de la aplicación es fuertemente relacional: los establecimientos pertenecen a una categoría y a un municipio, las valoraciones relacionan usuarios con establecimientos y cada valoración se descompone en puntuaciones por criterio. Esas relaciones deben ser consistentes, de modo que no existan valoraciones sin establecimiento ni puntuaciones sin valoración, y las consultas de la aplicación cruzan datos de varias entidades con frecuencia, por ejemplo para calcular la media de un establecimiento o el ranking del panel. Por ambas razones se eligió una base de datos relacional, PostgreSQL, frente a una base documental, en la que esas garantías recaerían en el código de la aplicación.

El esquema se define en Prisma, que actúa como única fuente de verdad: de él se generan los tipos de TypeScript que usan los servicios y las migraciones SQL que se aplican a la base de datos. Esto evita que el modelo del código y el de la base diverjan. En producción la base de datos es un PostgreSQL gestionado mediante Neon (capítulo 11).

Se aplicaron cinco criterios. Las claves primarias son identificadores opacos y no secuenciales (cuid), que no revelan el número de registros ni permiten recorrer los recursos por enumeración. Los datos de referencia, como categorías, criterios, provincias y municipios, se normalizan en tablas propias en lugar de repetirse como texto. La integridad se delega en la base de datos siempre que el motor pueda garantizarla, mediante claves foráneas, restricciones de unicidad y de obligatoriedad, y solo cuando no puede, como en algunas reglas de cardinalidad, se garantiza en los servicios y se declara. Las fechas de creación y modificación se mantienen automáticamente. Por último, el esquema no almacena datos derivables, como la media de valoraciones, que se calculan en la consulta.

5.2 Visión general del modelo

El modelo comprende nueve entidades de dominio, relacionadas como muestra la figura 5.1, y tres tablas auxiliares que exige Auth.js. Las tablas auxiliares no se representan en la figura.

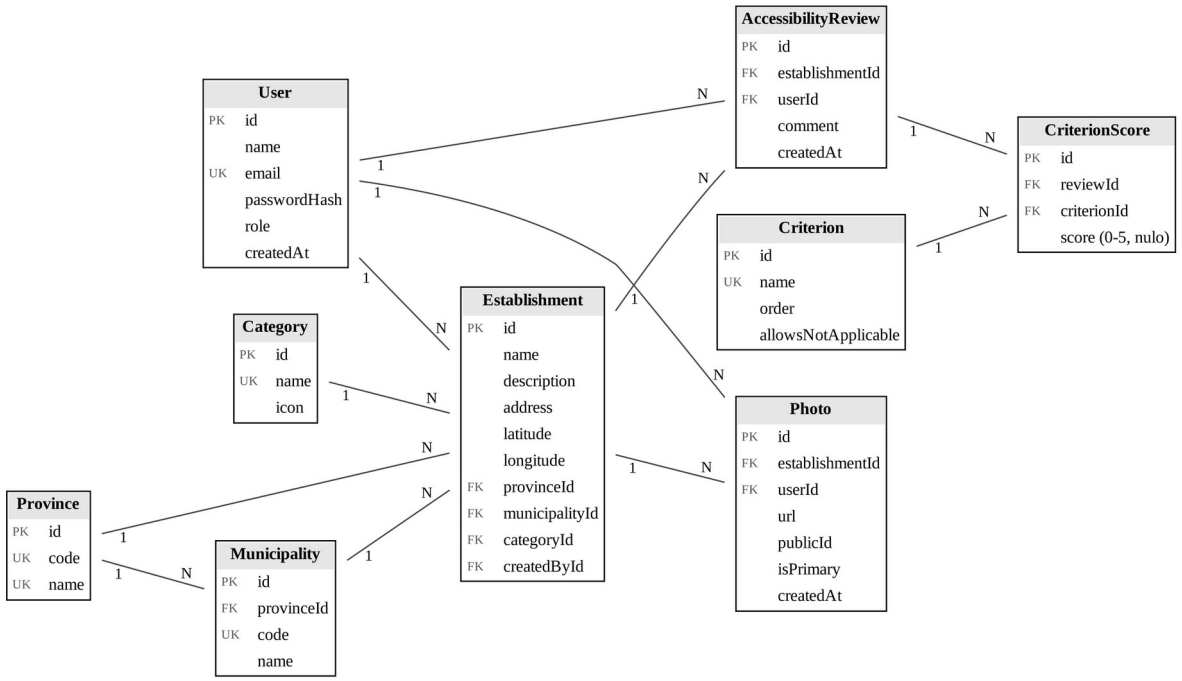


Figura 5.1. Modelo entidad-relación de las entidades de dominio.

Tabla 5.1. *Entidades de dominio y motivo de cada una.*

Entidad	Contenido	Motivo de su existencia
User	Datos de la cuenta: nombre, correo único, hash de contraseña y rol.	Identifica a quien aporta cada dato y sustenta la autenticación y los permisos.
Establishment	Nombre, descripción, dirección, coordenadas anulables, categoría, provincia, municipio y creador.	Es el objeto sobre el que gira todo el producto: lo que se localiza y se valora.
Category	Nombre único.	Clasifica los establecimientos y permite filtrar. Es una tabla y no un tipo enumerado para que el administrador pueda gestionarla sin modificar el código.
Province y Municipality	Código oficial del INE y denominación oficial.	Normalizan la ubicación para evitar variantes de escritura, permitir filtros fiables y construir la consulta de geocodificación con nombres oficiales.
Criterion	Nombre único, orden de presentación e indicador de si admite «No aplicable».	Define los ocho aspectos valorados. Es una tabla y no un tipo enumerado para que la lista y sus reglas sean datos, no código.
AccessibilityReview	Valoración de un usuario sobre un establecimiento, con comentario opcional.	Es la entidad de asociación entre usuarios y establecimientos. Existe como entidad propia porque la relación tiene atributos y una restricción de unicidad.

Entidad	Contenido	Motivo de su existencia
CriterionScore	Puntuación de 0 a 5, o nula, de un criterio en una valoración.	Resuelve la relación de muchos a muchos entre valoraciones y criterios y almacena el valor puntuado.
Photo	Dirección y identificador del archivo en Cloudinary, indicador de principal, autor y establecimiento.	Registra las fotografías sin almacenar los archivos en la base de datos.

Los registros de referencia se cargan mediante un script de datos iniciales: 10 categorías, 8 criterios, las 52 provincias y 8.192 municipios del INE. El script es idempotente, de modo que puede ejecutarse varias veces sin duplicar datos.

5.3 Relaciones, cardinalidades y política de borrado

La tabla 5.2 recoge todas las relaciones del modelo. La política de borrado indica qué ocurre con el registro dependiente cuando se elimina el de referencia. «Cascada» lo elimina también. «Restringir» impide la eliminación mientras existan dependientes.

Tabla 5.2. *Relaciones del modelo.*

Relación	Cardinalidad	Al eliminar el registro de referencia	Justificación
User – Establishment	1 a N	Cascada	Un establecimiento sin creador carece de propietario a efectos de permisos.
User – AccessibilityReview	1 a N	Cascada	Las valoraciones pertenecen a su autor.
User – Photo	1 a N	Cascada	Igual que las valoraciones.
Category – Establishment	1 a N	Restringir	Una categoría en uso no puede desaparecer sin dejar fichas sin clasificar.
Province – Municipality	1 a N	Restringir	Los municipios son datos de referencia fijos.
Province – Establishment	1 a N	Restringir	Ídem.
Municipality – Establishment	1 a N	Restringir	Ídem.
Establishment – AccessibilityReview	1 a N	Cascada	Una valoración carece de sentido sin su establecimiento.
Establishment – Photo	1 a N	Cascada	Idem.
AccessibilityReview – CriterionScore	1 a N	Cascada	Las puntuaciones son parte de la valoración.
Criterion – CriterionScore	1 a N	Restringir	Impide eliminar un criterio que ya ha sido puntuado.

Relación	Cardinalidad	Al eliminar el registro de referencia	Justificación
User – Establishment (a través de AccessibilityReview)	N a M	No procede	Un usuario valora muchos establecimientos y un establecimiento recibe valoraciones de muchos usuarios.
Valoración – Criterion (a través de CriterionScore)	N a M	No procede	Cada valoración puntúa los ocho criterios.

La regla de fondo es que los datos de referencia, es decir, categorías, criterios, provincias y municipios, no pueden eliminarse mientras estén en uso, y los datos que aportan los usuarios dependen de su autor y de su establecimiento. La regla de las categorías se refuerza en el servicio, que comprueba si hay establecimientos asociados antes de eliminar y devuelve un mensaje comprensible en lugar de dejar que falle la restricción.

5.3.1 Consecuencias del borrado en cascada

El borrado en cascada desde el usuario fue una decisión de simplicidad adoptada para el producto mínimo viable. Elimina a la vez la cuenta y todo lo que la persona aportó, y no distingue entre una cuenta eliminada y una aportación que se quisiera conservar. Mientras la aplicación no ofrezca eliminar cuentas, la política no se ejercita, ya que ninguna operación elimina usuarios. Sí se ejercita la cascada desde el establecimiento, cuando se elimina una ficha con sus valoraciones y fotografías.

Hay una consecuencia que la base de datos no puede resolver por sí sola: la cascada elimina los registros de fotografías, pero no los archivos que Cloudinary conserva. Por eso la eliminación de un establecimiento borra primero las imágenes en el servicio externo desde el código (apartado 4.9). Si se incorporara la eliminación de cuentas, habría que repetir ese paso para las fotografías de la persona y decidir si sus valoraciones y establecimientos se conservan de forma anónima, con la política «poner a nulo» en lugar de la cascada. Se recoge en el capítulo 13.

5.4 Restricciones de integridad

No todas las reglas del dominio pueden expresarse como restricciones de PostgreSQL. La tabla 5.3 distingue dónde se garantiza cada una, porque de ello depende su robustez frente a accesos concurrentes y frente a errores de programación.

Tabla 5.3. *Reglas de integridad y lugar donde se garantizan.*

Regla	Garantizada en
El correo de un usuario es único.	Base de datos (restricción de unicidad) y servicio, que comprueba antes de insertar para dar un mensaje claro.

Regla	Garantizada en
Un usuario solo valora una vez un establecimiento.	Base de datos (unicidad sobre establecimiento y usuario).
Cada criterio aparece una sola vez por valoración.	Base de datos (unicidad sobre valoración y criterio).
Los nombres de categoría y de criterio son únicos.	Base de datos.
Los códigos y nombres de provincia, y los códigos de municipio, son únicos; no se repite un nombre dentro de una provincia.	Base de datos.
Todo establecimiento tiene categoría, provincia, municipio y creador.	Base de datos (claves foráneas obligatorias).
El municipio de un establecimiento pertenece a su provincia.	Servicio.
Una valoración puntúa todos los criterios.	Servicio, dentro de una transacción.
La puntuación es un entero de 0 a 5.	Esquema de validación y servicio.
Solo los criterios marcados admiten puntuación nula.	Servicio.
Un establecimiento tiene como máximo diez fotografías.	Servicio.
Un establecimiento tiene una sola fotografía principal.	Servicio, mediante una transacción que desmarca la actual al marcar la nueva.
Las coordenadas son ambas nulas o ambas informadas.	Servicio, que las obtiene siempre de un mismo resultado de geocodificación.

Las reglas garantizadas solo en el servicio son las más débiles. El límite de diez fotografías se comprueba contando antes de insertar, de modo que dos subidas simultáneas al mismo establecimiento podrían superarlo en una unidad. La unicidad de la fotografía principal podría reforzarse con un índice parcial único en PostgreSQL, y el rango de las puntuaciones con una restricción de comprobación (CHECK); ninguna de las dos se ha incorporado. Con el volumen de uso previsto el riesgo es bajo y la corrección es sencilla, por lo que se documentan como mejora en el capítulo 13.

5.5 Índices

PostgreSQL crea automáticamente un índice por cada clave primaria y cada restricción de unicidad, pero no por cada clave foránea. Los índices del esquema son, por tanto, los de las claves primarias y las restricciones de unicidad, y uno explícito sobre el establecimiento de cada fotografía.

Algunos de los índices de unicidad cumplen una segunda función. El índice sobre establecimiento y usuario de las valoraciones sirve también para buscar las valoraciones de un establecimiento, que es la consulta de la ficha, porque el establecimiento es su primera columna. Del mismo modo, el de valoración y criterio sirve para recuperar las puntuaciones de una

valoración, y el de provincia y nombre de los municipios para listar los municipios de una provincia, que alimenta el filtro del listado.

No existen índices sobre las claves foráneas de los establecimientos hacia categoría, provincia, municipio y creador, ni sobre el autor de las valoraciones y de las fotografías, aunque la aplicación los consulta al filtrar el listado, calcular las estadísticas del perfil o eliminar contenidos. La búsqueda por nombre, de coincidencia parcial sin distinguir mayúsculas, tampoco puede apoyarse en un índice convencional. Con el volumen de datos previsto, el motor recorre las tablas enteras sin coste apreciable, y por eso no se añadieron. Si la comunidad creciera, serían los primeros cambios de rendimiento a considerar, junto con un índice específico para la búsqueda de texto.

5.6 Decisiones de modelado

Geografía normalizada. La primera versión del establecimiento guardaba provincia y municipio como texto libre. Esa elección admite variantes de escritura de un mismo lugar, como el uso o no de mayúsculas o de la denominación en otra lengua cooficial, lo que degrada los filtros y obliga a geocodificar con datos no verificados. Se sustituyó por dos tablas con los datos oficiales del INE, de modo que se pasó de un texto sin garantías a una referencia verificable. El cambio se hizo en dos migraciones: la primera añadió las tablas y las referencias como anulables, manteniendo las columnas de texto, y la segunda eliminó el texto e impuso las referencias obligatorias. Este procedimiento de dos pasos evita dejar la base en un estado inconsistente en mitad de la transición.

Provincia y municipio en el establecimiento. El establecimiento almacena ambas referencias aunque la provincia podría deducirse del municipio. Es una redundancia deliberada: simplifica el filtro por provincia, que no necesita unir tablas, y la construcción de la dirección. Su coste es que la coherencia entre ambas no la garantiza la base de datos, sino el servicio, que rechaza un municipio que no pertenece a la provincia indicada. Una clave foránea compuesta la haría cumplir en la propia base de datos, pero complicaría el esquema para un riesgo que, con el acceso únicamente mediante la aplicación, queda cubierto.

Puntuación nula. El valor «No aplicable» se representa como ausencia de puntuación y no con un número reservado, como −1. Un valor mágico obligaría a recordar en cada cálculo que ese número no es una puntuación, y bastaría un olvido para que contaminara una media. Con el valor nulo, las funciones de agregación de SQL y el código TypeScript ya lo tratan como ausencia de dato.

Media no almacenada. La media de un establecimiento no es una columna: se calcula en cada consulta a partir de las puntuaciones. Almacenarla habría exigido mantenerla sincronizada al crear y eliminar valoraciones, con riesgo de inconsistencia. El precio es el cálculo en cada

lectura, asumible con el volumen actual y que el capítulo 13 propone sustituir por una media materializada si crece.

Valoración como entidad de asociación. La relación entre usuarios y establecimientos no es una simple tabla intermedia. La valoración tiene comentario y fecha, y la restricción de unicidad sobre el par establecimiento y usuario es la que impide valorar dos veces.

Fotografías por referencia. Solo se almacenan la dirección y el identificador público de la imagen. La dirección permite mostrarla y el identificador permite eliminarla en Cloudinary. El archivo no se guarda en la base de datos, lo que la mantiene pequeña y evita servir imágenes desde ella.

Contraseña opcional. El hash de la contraseña es anulable. En el registro por correo siempre existe, pero permitir su ausencia admite incorporar más adelante inicios de sesión con proveedores externos sin cambiar el esquema.

Rol como enumeración. Los roles son un tipo enumerado con tres valores: usuario, moderador y administrador. El de moderador no tiene funcionalidad en la versión entregada y se reservó para una futura moderación colaborativa, de modo que su incorporación no exija una migración.

5.7 Elementos del esquema sin uso en la versión actual

Algunos elementos del esquema se previeron para ampliaciones y no se utilizan por ahora. Las tablas de cuentas externas, de sesiones y de tokens de verificación son las que define el adaptador de bases de datos de Auth.js; la aplicación usa sesiones basadas en token, sin adaptador, y no ofrece proveedores externos ni verificación de correo, de modo que permanecen vacías. Lo mismo ocurre con los campos del código postal y de la imagen de perfil, el pie de foto de las fotografías y la fecha de verificación del correo. Mantenerlos tiene un coste bajo y evita migraciones cuando se incorporen las funciones correspondientes, aunque supone cierto ruido en el modelo y debería revisarse si esas funciones no llegan a implementarse.

5.8 Migraciones y datos iniciales

La base de datos evoluciona mediante ocho migraciones versionadas, aplicadas en orden y almacenadas en el repositorio. Cada una se generó junto con la funcionalidad que la requería, de modo que el historial de migraciones refleja el orden en que creció el modelo.

Tabla 5.4. *Migraciones del esquema.*

Migración	Contenido
`init_auth`	Usuarios, rol y tablas auxiliares de Auth.js.
`add_establishments`	Categorías y establecimientos, con ubicación como texto.
`add_accessibility_reviews`	Criterios, valoraciones y puntuaciones.
`add_photos`	Fotografías.

Migración	Contenido
`add_establishment_coordinates`	Latitud y longitud anulables.
`add_geo_entities`	Provincias y municipios; referencias anulables convivientes con el texto.
`drop_geo_text_columns`	Eliminación del texto y referencias obligatorias.
`add_criterion_not_applicable`	Indicador de «No aplicable» y puntuación anulable.

Los datos de referencia no forman parte de las migraciones, sino de un script de datos iniciales. Los municipios proceden del conjunto oficial «Relación de municipios y sus códigos por provincias» del INE, consultado a través de su servicio de datos y convertido a un fichero JSON incluido en el repositorio. La carga de provincias y categorías usa operaciones de inserción o actualización por clave única y la de municipios omite duplicados, y por eso puede repetirse sin efectos secundarios, requisito para desplegar en un entorno nuevo y para restablecer el entorno de desarrollo.

5.9 Limitaciones y síntesis

El esquema cubre las necesidades del producto con restricciones razonables, pero deja abiertas varias cuestiones que se han ido señalando: la ausencia de restricciones de comprobación y de índices parciales para algunas reglas, la falta de índices sobre las claves foráneas más consultadas, la política de borrado en cascada sin tratamiento de los archivos externos, y los elementos previstos que todavía no se usan. Ninguna compromete la integridad con el volumen y el patrón de uso actuales, que es el de un acceso exclusivamente a través de los servicios de la aplicación. Si se abriera otro punto de entrada a los datos, como una API pública o una importación masiva, las reglas garantizadas solo en el servicio dejarían de estar protegidas y habría que trasladarlas a la base de datos.

6. Desarrollo de las funcionalidades

6.1 Planteamiento del capítulo

El uso de la aplicación se describe paso a paso en el manual de usuario (anexo A), y las capturas de las pantallas principales están en el anexo E.

Este capítulo describe cómo se implementó cada funcionalidad del catálogo del capítulo 3. Para cada una se explica su funcionamiento desde el punto de vista del usuario, los elementos del código que la sustentan y las decisiones que condicionaron la solución. No se repite lo que ya se ha tratado en los capítulos anteriores: el patrón común de las Server Actions, el flujo de autenticación y la regla de permisos se describieron en el capítulo 4, y las entidades y restricciones en el capítulo 5. Aquí se citan por referencia.

Los requisitos se identifican con los códigos del capítulo 3. Las validaciones se recogen de forma conjunta en el apartado 6.13, y no dentro de cada funcionalidad, para poder compararlas.

6.2 Registro de usuarios (RF-01)

Funcionamiento. La persona introduce nombre, correo, contraseña y confirmación. Si los datos son válidos, se crea la cuenta y se la lleva a la pantalla de inicio de sesión con un aviso de registro completado. El registro no inicia sesión automáticamente.

Implementación. El formulario valida en el navegador con el esquema de Zod y envía los datos a la acción de registro, que repite la validación en el servidor y llama al servicio. El servicio normaliza el correo, comprueba que no exista, calcula el hash con bcrypt con coste 12 y crea el usuario con rol de usuario. Los errores esperados, como un correo ya registrado, se devuelven asociados al campo correspondiente y se muestran junto a él.

Decisiones. No iniciar la sesión al registrarse simplifica el flujo y mantiene separadas las dos operaciones, lo que facilitó construir y probar cada una por su lado. Se eligió una longitud mínima de contraseña de 12 caracteres, con mayúscula, minúscula y número, y no un mínimo menor con reglas de composición más estrictas: es un compromiso entre robustez y facilidad para la persona usuaria, que no puede recuperar la contraseña si la olvida (apartado 3.2.2). La unicidad del correo y la protección frente a registros simultáneos se explicaron en el apartado 4.7.2.

6.3 Inicio y cierre de sesión (RF-02, RF-03)

Funcionamiento. Con credenciales correctas se accede al panel principal. Con credenciales incorrectas se muestra el mensaje «Correo electrónico o contraseña incorrectos», sin indicar cuál de los dos datos falla. El cierre de sesión está disponible en la barra superior.

Implementación. El flujo completo se describió en el apartado 4.7.2. Desde el punto de vista de la interfaz, la acción de inicio de sesión nunca devuelve éxito, porque cuando las credenciales son válidas Auth.js redirige directamente; por eso su tipo de resultado solo modela el fallo. Cualquier error que no sea de credenciales se relanza, para no ocultar la redirección que Next.js implementa mediante una excepción interna.

Decisiones. El formulario de inicio de sesión valida únicamente que el correo tenga formato válido y que la contraseña no esté vacía, y no aplica las reglas de composición del registro. Una contraseña que no las cumpla recibe la misma respuesta genérica que cualquier otra incorrecta.

6.4 Perfil de usuario (RF-05)

Funcionamiento. La página de perfil muestra el nombre, el correo, el rol, la fecha de alta y tres contadores de actividad: establecimientos creados, valoraciones y fotografías aportadas. Permite cambiar el nombre y la contraseña.

Implementación. Los datos del perfil se obtienen en una única consulta que incluye los recuentos mediante agregación de la base de datos, sin traer los registros. El hash de la contraseña nunca sale del servicio: este solo informa de si la cuenta tiene contraseña local. El cambio de nombre guarda el valor y actualiza el token de sesión (apartado 4.7.4). El cambio de contraseña exige la actual, comprueba que coincida con el hash almacenado y aplica a la nueva las mismas reglas que el registro. Las tarjetas de estadísticas reutilizan el componente que también muestra el panel principal.

Decisiones. Pedir la contraseña actual protege frente a quien aproveche una sesión abierta en un equipo ajeno. El servicio contempla cuentas sin contraseña local, como las de un futuro proveedor externo, y en ese caso no ofrece el formulario; hoy no existe ninguna cuenta de ese tipo. Como la sesión es un token firmado que no se almacena en el servidor, cambiar la contraseña no invalida los tokens ya emitidos, hasta que caducan. Es una consecuencia del diseño de sesión que se evalúa en el capítulo 7.

6.5 Alta de establecimientos (RF-06)

Funcionamiento. El formulario pide nombre, categoría, dirección, provincia, municipio y una descripción opcional. Al elegir una provincia, el desplegable de municipios muestra solo los de esa provincia. Si los datos son válidos se crea la ficha y se navega al listado.

Implementación. El formulario recibe del servidor las categorías, las provincias y la lista completa de municipios. El filtrado de municipios por provincia se hace en el navegador, y al cambiar la provincia se reinicia el municipio elegido para no conservar uno de otra provincia. En el servidor, la acción obtiene el usuario de la sesión y valida con el esquema, y el servicio verifica que existan la categoría y la provincia, y que el municipio pertenezca a la provincia.

Solo entonces geocodifica y guarda. El creador del establecimiento proviene siempre de la sesión y no del formulario, de modo que no se puede crear una ficha a nombre de otra persona manipulando la petición.

Decisiones. Cargar los 8.192 municipios en el servidor y filtrarlos en el navegador evita una petición adicional cada vez que cambia la provincia y simplifica el componente, pero tiene un coste: la lista viaja completa con la página, del orden de 1 MB sin comprimir según una estimación a partir de los datos. La compresión del transporte lo reduce de forma notable, y la página de listado, que incluye el mismo filtro, comparte la carga. Es una decisión razonable para la primera versión que sería la primera a revisar si se midiera un problema de carga en conexiones lentas, por ejemplo cargando los municipios bajo demanda al elegir una provincia.

6.6 Edición y eliminación de establecimientos (RF-09, RF-10, RF-22)

Funcionamiento. En la ficha, el creador y los administradores ven las acciones «Editar» y «Eliminar». La edición abre el mismo formulario del alta con los valores actuales. La eliminación pide confirmación y, al aceptarla, borra el establecimiento con sus fotografías y valoraciones.

Implementación. La página de edición comprueba en el servidor que el usuario pueda gestionar el establecimiento y, si no, responde como si no existiera (apartado 4.8). La acción de actualización repite la comprobación mediante el servicio, que lee el propietario de la base de datos. Las coordenadas se recalculan solo si cambia alguno de los tres datos que determinan la ubicación; en caso contrario se conservan, sin llamar al servicio externo. Al guardar se revalidan la ficha y el listado, para que el contenido en caché se actualice.

La eliminación lee primero las fotografías del establecimiento, borra cada una en Cloudinary y después elimina el registro, cuyas dependencias caen en cascada (apartado 5.3). El orden evita dejar imágenes huérfanas en el servicio externo.

Decisiones. Reutilizar el formulario del alta en modo edición garantiza que las reglas de validación sean idénticas y que no puedan divergir. La confirmación se resuelve con el diálogo nativo del navegador, que es accesible por defecto y no requiere un componente propio; el permiso real se comprueba en el servidor y la confirmación solo previene errores del usuario. Una limitación que conviene señalar es que, si el borrado de una imagen en Cloudinary falla a mitad del bucle, las anteriores ya se han eliminado y el establecimiento sigue existiendo con fotografías cuyo archivo ya no está. No se trata mediante una compensación: la operación se puede repetir, pero no se ha verificado cómo responde el servicio externo al borrar de nuevo una imagen ya eliminada.

6.7 Listado, búsqueda y filtros (RF-11)

Funcionamiento. El listado muestra los establecimientos con su fotografía, categoría, ubicación y valoración media. Un formulario permite buscar por nombre, filtrar por categoría, provincia y municipio, y ordenar por fecha, nombre o valoración. Los filtros aparecen en la dirección de la página, de modo que se pueden compartir y se conservan al volver con el historial del navegador. Sin resultados, la página lo indica y ofrece limpiar los filtros.

Implementación. El formulario es un formulario HTML con método GET, sin JavaScript para enviar los filtros. La página de servidor lee los parámetros de la dirección y los pasa al servicio. Este normaliza cada filtro, descarta los vacíos y construye la condición de la consulta con un único constructor, en el que solo participan los filtros con valor. Un valor de ordenación desconocido equivale al orden por defecto, por lo que una dirección manipulada no provoca error. La búsqueda por nombre es de coincidencia parcial sin distinguir mayúsculas. El municipio solo se habilita cuando hay provincia elegida.

Decisiones. El formulario GET aporta enlaces compartibles, historial del navegador y funcionamiento sin JavaScript, que se obtienen sin código adicional. La ordenación por valoración se hace en memoria porque la media no es una columna (apartado 5.6): el servicio lee los establecimientos con sus puntuaciones, calcula las medias y ordena, con desempate por fecha. Es una limitación que se declaró desde el principio, junto con la ausencia de paginación, y que se evalúa en el apartado 3.3.5.

6.8 Gestión de categorías y administración (RF-21)

Funcionamiento. Los administradores disponen de un enlace «Administración» en la barra superior que abre una página en la que pueden crear, renombrar y eliminar categorías. Cada categoría muestra cuántos establecimientos la usan.

Implementación. La página comprueba en el servidor que el usuario sea administrador y, si no lo es, responde como recurso inexistente. Las tres acciones de categoría repiten la comprobación. El nombre se valida con un esquema de entre 2 y 100 caracteres. El servicio traduce el error de unicidad de la base de datos a un mensaje claro y rechaza la eliminación de una categoría con establecimientos asociados, para lo cual comprueba el recuento antes de borrar.

Decisiones. El enlace solo se muestra a administradores, pero ocultarlo es una cuestión de comodidad: la protección real está en el servidor. La administración se limita a las categorías porque es el único dato de referencia que los usuarios necesitan ampliar: los criterios, las provincias y los municipios son conjuntos cerrados. El rol de administrador se asigna directamente en la base de datos, de modo que la aplicación no contiene un flujo de elevación de privilegios.

6.9 Geolocalización y mapas (RF-07, RF-18, RF-19)

El proyecto distingue dos operaciones que a menudo se confunden. La geocodificación convierte la dirección de un establecimiento en coordenadas y se realiza en el servidor, y la visualización de los mapas ocurre en el navegador. No se utiliza la geolocalización del dispositivo de la persona usuaria, que se excluyó expresamente del alcance.

6.9.1 Geocodificación

Funcionamiento. Al crear o editar un establecimiento, el sistema consulta a Nominatim con la dirección escrita, el municipio y la provincia oficiales. Si obtiene un resultado, guarda sus coordenadas. Si la consulta falla, no hay resultado o la respuesta no es válida, el establecimiento se guarda igualmente sin coordenadas y la ficha muestra un aviso en lugar del mapa.

Implementación. La función de geocodificación construye la consulta, la envía con un plazo máximo de cinco segundos y un identificador de aplicación, tal como exigen las condiciones de uso del servicio, y valida que las coordenadas recibidas sean números finitos. Cualquier excepción devuelve nulo. El municipio y la provincia que se envían son los nombres oficiales leídos de la base de datos y no texto libre del formulario; la dirección, en cambio, es el texto que escribió la persona.

Decisiones. El fallo silencioso fue deliberado: crear una ficha es la contribución principal del producto y no debe depender de la disponibilidad de un servicio público. El coste es que el usuario no sabe por qué un establecimiento carece de mapa; el aviso de la ficha lo señala sin explicar la causa. No existe una opción para repetir la geocodificación ni para corregir las coordenadas a mano, porque se excluyeron del alcance. La única forma de volver a intentarlo es editar la dirección, lo que dispara una nueva consulta. La precisión depende de que la dirección figure en OpenStreetMap, y no se verifica que el punto devuelto corresponda al establecimiento correcto, sino únicamente que exista un resultado.

6.9.2 Mapa de la ficha y mapa global

Funcionamiento. La ficha incluye una sección de ubicación con un mapa centrado en el establecimiento con un marcador. La página de mapa global muestra todos los establecimientos con coordenadas, indica su número en texto y ajusta la vista para que se vean todos. Al pulsar un marcador se abre una ventana con el nombre, la categoría, el municipio, la valoración, la fotografía principal y un enlace a la ficha.

Implementación. Ambos mapas comparten un marco común que fija las teselas de OpenStreetMap, la atribución obligatoria y el desplazamiento con rueda desactivado, para que el mapa no capture el desplazamiento de la página. Los marcadores son círculos dibujados con vectores, no imágenes, lo que evita los problemas de rutas de iconos habituales en Leaflet con los empaquetadores y no añade recursos que descargar. Como Leaflet necesita el DOM del

navegador, cada mapa se carga mediante un componente intermedio de carga diferida, sin renderizado en servidor (apartado 4.4). El mapa global recibe sus datos de una única consulta, con la primera fotografía de cada establecimiento y sus puntuaciones, y la ventana de cada marcador es un componente aparte con enlaces navegables por teclado. Los propios marcadores, sin embargo, no reciben el foco del teclado, de modo que esa ventana solo se abre con el ratón o el tacto; el capítulo 8 lo analiza como defecto de accesibilidad.

Decisiones. Se consideró desde el diseño que el mapa debía ser complementario y no el único acceso a la información: cada dato de ubicación existe también en texto, tanto en la ficha como en el resumen de la página de mapa. Esa decisión responde a un requisito de accesibilidad, que el capítulo 8 evalúa. Una limitación del mapa global es que carga todos los establecimientos con coordenadas, sin agrupación de marcadores ni filtros, lo que se vuelve poco manejable con muchos puntos.

6.10 Valoraciones (RF-12 a RF-15)

Funcionamiento. Debajo de la ficha, el formulario de valoración muestra un campo de comentario opcional, una leyenda con el significado de la escala y una tarjeta por criterio con las puntuaciones de 0 a 5 como botones de radio. Los dos criterios que lo admiten ofrecen además «No aplicable». Una vez enviada, la valoración aparece en la lista de la ficha, donde su autor y los administradores pueden eliminarla.

Implementación. El formulario se construye a partir de los criterios leídos de la base de datos, y no de una lista fija en el código: añadir un criterio no exige modificar la interfaz. Los valores de los botones de radio llegan al servidor como texto; el esquema convierte «na» en valor nulo y el resto en números enteros, y rechaza lo que no esté entre 0 y 5. Si el envío no es válido, el formulario desplaza la vista hasta el primer criterio sin puntuar y le da el foco, con el DOM nativo. En el servicio, la creación comprueba la existencia del establecimiento, la ausencia de una valoración previa del usuario, la cobertura completa de los criterios y el uso permitido de «No aplicable», y guarda la valoración y las puntuaciones en una transacción, de forma que no puede quedar una valoración incompleta.

Decisiones. El modelo de valoración se justificó en el apartado 3.2.1. En la implementación, la decisión más relevante es la de no permitir la edición: obliga a eliminar y volver a crear, pero evita tener que gestionar versiones o un historial de cambios y deja la fecha de la valoración como la de la última opinión vigente. El comentario no tiene longitud máxima definida, ni se sanea de forma específica: el sistema de plantillas de React escapa su contenido al mostrarlo, lo que impide que se ejecute como código, pero la ausencia de límite es una carencia que recoge el capítulo 7. Los comentarios no se moderan; solo el autor y los administradores pueden retirarlos.

6.11 Fotografías y carga de imágenes (RF-16, RF-17)

Funcionamiento. La ficha muestra una galería con la fotografía principal en grande y las demás como miniaturas. Cualquier usuario registrado puede subir una fotografía por vez. Quien subió una fotografía, y los administradores, pueden eliminarla y, pulsando una miniatura, convertirla en la principal. Si se elimina la principal, la más antigua de las restantes pasa a serlo.

Implementación. El formulario de subida usa el mecanismo de acciones de formulario de React, con un único archivo y los atributos de tipo aceptado y de ayuda asociados. La acción recibe el formulario, obtiene la sesión, valida el establecimiento y el archivo con el esquema (que no esté vacío, tenga como máximo 5 MB y su tipo declarado sea JPEG, PNG o WebP) y llama al servicio. El servicio realiza esta secuencia: comprueba que el establecimiento exista, cuenta las fotografías y rechaza la subida si ya hay diez, envía el archivo a Cloudinary mediante un flujo desde el servidor, y comprueba las dimensiones que el servicio externo devuelve. Si son inferiores a 800 × 600 píxeles, elimina la imagen recién subida y rechaza la subida. Si todo es correcto, guarda la dirección y el identificador público, marcando como principal la primera fotografía del establecimiento.

La eliminación borra primero el archivo en Cloudinary y después el registro. Designar una fotografía como principal desmarca la actual y marca la nueva en una única transacción.

Decisiones. Las dimensiones mínimas se comprueban después de subir el archivo porque se obtienen de la respuesta de Cloudinary y no se leen en el servidor, lo que evita incorporar una biblioteca de procesamiento de imágenes. El coste es que una imagen demasiado pequeña llega a subirse antes de rechazarse. El tipo de archivo se valida a partir del tipo que declara el navegador, sin inspeccionar el contenido real del archivo, de modo que un fichero manipulado podría presentarse como imagen; Cloudinary previsiblemente rechaza lo que no sea una imagen, pero la comprobación propia es limitada. Se recoge en el capítulo 7. El límite de 5 MB del esquema coincide en cifra con el que se configuró para el tamaño máximo del cuerpo de las acciones. Como el cuerpo incluye además la sobrecarga del formulario, una imagen muy cercana a los 5 MB podría ser rechazada por el framework antes de llegar a la validación propia y con un mensaje menos claro; este caso límite no se ha comprobado.

Esta funcionalidad fue la más problemática del desarrollo. Una convención del framework sobre los ficheros de acciones, que no se manifestaba en el entorno de desarrollo, provocó fallos en el entorno desplegado y requirió una serie de correcciones consecutivas hasta formalizar la regla descrita en el apartado 4.6.1. El episodio se analiza en el capítulo 11.

Las fotografías muestran como texto alternativo el nombre del establecimiento. No se solicita una descripción al subirlas, lo que limita la información que reciben quienes dependen de un lector de pantalla; el campo de pie de foto existe en el modelo pero no se utiliza. Este punto se trata en el capítulo 8.

6.12 Panel principal (RF-20)

Funcionamiento. El panel saluda a la persona por su nombre y muestra cuatro totales, los seis últimos establecimientos, los seis mejor valorados con medallas para los tres primeros, y cuatro accesos rápidos.

Implementación. La página no calcula nada: recibe de un servicio un objeto ya preparado para la vista. El servicio lanza en paralelo las cuatro cuentas, la lectura de establecimientos y la de las últimas valoraciones, de modo que el tiempo total es el de la más lenta y no la suma de todas, y evita el patrón de una consulta por cada elemento de una lista. La media de cada establecimiento usa la misma función que el listado, sin duplicar la fórmula. El servicio también prepara las cinco últimas valoraciones, que la página no muestra por ahora.

Decisiones. Se prefirió una única función que prepare todo el objeto de la página para que esta quede reducida a la presentación y para que la lógica se pueda probar sin interfaz. Como en otras partes, el servicio lee todos los establecimientos con sus puntuaciones y se queda con seis, de modo que el coste crece con el volumen; es la misma limitación que la ordenación por valoración. El ranking ordena por media y, solo cuando las medias coinciden, por número de valoraciones y después por fecha. No pondera la media por el número de valoraciones, de modo que un establecimiento con una única valoración perfecta se sitúa por encima de otro con muchas y una media algo inferior. Es una limitación del indicador, que no mide la fiabilidad de la media, y una mejora posible sería exigir un número mínimo de valoraciones para entrar en el ranking.

6.13 Validaciones

La tabla 6.1 reúne las reglas que aplican los esquemas de validación. Todas se ejecutan en el servidor y las del formulario del navegador son las mismas, por compartir esquema.

Tabla 6.1. *Reglas de validación de entrada.*

Formulario	Campo	Regla
Registro	Nombre	Obligatorio, de 2 a 100 caracteres, sin espacios sobrantes.
	Correo	Obligatorio, formato de correo, normalizado a minúsculas.
	Contraseña	Mínimo 12 caracteres, con mayúscula, minúscula y número.
	Confirmación	Debe coincidir con la contraseña.
Inicio de sesión	Correo y contraseña	Correo con formato válido y contraseña no vacía.
Perfil	Nombre	Mismas reglas que en el registro.
	Nueva contraseña	Mismas reglas que en el registro, más contraseña actual obligatoria.

Formulario	Campo	Regla
Establecimiento	Nombre	Obligatorio, de 3 a 120 caracteres.
	Categoría, provincia, municipio	Obligatorias; en el servidor, deben existir y el municipio pertenecer a la provincia.
	Dirección	Obligatoria, sin longitud máxima.
	Descripción	Opcional, sin longitud máxima.
Categoría	Nombre	De 2 a 100 caracteres, único.
Valoración	Puntuaciones	Una por criterio, entero de 0 a 5; nulo solo en los criterios que lo admiten.
	Comentario	Opcional, sin longitud máxima.
Fotografía	Archivo	No vacío, hasta 5 MB, tipo JPEG, PNG o WebP; mínimo 800 × 600 píxeles; máximo diez por establecimiento.

Las tres filas sin límite máximo son la carencia más visible del conjunto y se tratan como hallazgo en el capítulo 7.

6.14 Dónde se aplica cada permiso

La regla de permisos se describió en el apartado 4.8 y la matriz de actores en la tabla 3.5. La tabla 6.2 indica, para cada operación, en qué punto del código se hace cumplir, con el fin de que la protección sea comprobable.

Tabla 6.2. *Puntos de aplicación de los permisos.*

Operación	Sesión	Permiso sobre el recurso
Ver páginas de contenido	Layout del grupo protegido	No procede
Crear establecimiento, valoración o fotografía	Acción	No procede; el autor procede de la sesión
Editar establecimiento	Página y acción	Servicio, contra el creador almacenado
Eliminar establecimiento	Acción	Servicio, contra el creador almacenado
Eliminar valoración	Acción	Servicio, contra el autor almacenado
Eliminar o fijar fotografía	Acción	Servicio, contra quien la subió
Gestionar categorías	Página y acción	Comprobación de rol de administrador

6.15 Páginas informativas (RF-23)

La página de presentación explica el propósito del producto, resume los criterios de valoración y ofrece los accesos al registro y al inicio de sesión. Es una página estática y su contenido,

incluidas las tarjetas de beneficios y el resumen de los criterios, se compone de componentes sin lógica. Las páginas de aviso legal, política de privacidad y política de cookies son de contenido estático, con un enlace para volver a la aplicación que depende de si hay sesión. Se enlazan desde el pie de página, que forma parte del diseño general de la aplicación.

6.16 Síntesis

Las funcionalidades comparten una estructura que permite entender cualquiera de ellas con el patrón del capítulo 4: la acción valida y delega, el servicio decide y la interfaz presenta. Las decisiones que más marcaron la implementación fueron mantener la geocodificación fuera del camino crítico, calcular la media en la consulta y no almacenarla, reutilizar el formulario de alta en la edición y trasladar al servidor todo permiso y toda validación. Entre las limitaciones que se han ido señalando, las más relevantes son la carga completa de municipios en las páginas con filtros, la ausencia de paginación y de agrupación en el mapa, la validación del tipo de archivo basada en lo que declara el navegador y la falta de límites de longitud en tres campos de texto. El capítulo 7 las evalúa desde el punto de vista de la seguridad.

7. Seguridad

7.1 Enfoque

La seguridad se trató como un requisito transversal desde el diseño, y no como una fase final. Algunas decisiones de los capítulos anteriores responden directamente a ese objetivo: la autorización en los servicios y no en la interfaz, la validación repetida en el servidor, la identidad del autor tomada siempre de la sesión y la ausencia de SQL escrito a mano. Este capítulo reúne esas medidas, evalúa su efectividad y expone lo que falta.

Para la evaluación se realizó una revisión estática de caja blanca del código, la configuración y las pruebas, tomando como marco las diez categorías del OWASP Top 10 en su edición de 2021 (OWASP Foundation, 2021). Se eligió por ser la de uso más extendido; el proyecto OWASP ha publicado una edición posterior (2025), cuya correspondencia de categorías no se ha contrastado y por la que la clasificación debe leerse con esa salvedad. Cada hallazgo se puntúa con una escala común de probabilidad (P) e impacto (I) de 1 a 5, y su riesgo es el producto $R = P \times I$, con cinco bandas: crítico (20 a 25), alto (15 a 19), medio (8 a 14), bajo (3 a 7) e informativo (1 a 2). El informe completo, con las fichas de cada hallazgo, la evidencia con archivo y línea y las recomendaciones, se incluye como anexo B. Aquí se resumen sus resultados y se discuten sus implicaciones.

El capítulo distingue en todo momento tres estados: lo que está implementado, lo que está pendiente y el riesgo residual que queda mientras tanto. La revisión es estática y no incluye pruebas de intrusión sobre el sistema desplegado, limitación que se detalla en el apartado 7.9.

7.2 Contexto de amenaza

La aplicación maneja datos de bajo valor económico y no procesa pagos, pero sí datos personales: nombre, correo y hash de contraseña de cada cuenta, y las contribuciones de cada persona. En una plataforma sobre accesibilidad, además, el perfil de contribuciones puede revelar información sobre las necesidades de quien la usa, por lo que no es un dato trivial.

Las amenazas razonables para un servicio público de este tipo son el abuso automatizado de los formularios de acceso, la toma de cuentas por contraseñas débiles o robadas, el acceso a funciones de otro usuario o de administración, la subida de contenido inadecuado y la explotación de componentes vulnerables. No se contempló un atacante con recursos extraordinarios ni la seguridad de la infraestructura de los proveedores (Vercel, Neon, Cloudinary), que se asume responsabilidad de estos.

Las superficies expuestas son cinco: los formularios de registro e inicio de sesión, las Server Actions invocables con una sesión válida, la subida de archivos, la consulta saliente a Nominatim y la cookie de sesión.

7.3 Controles implementados

7.3.1 Autenticación y contraseñas

Las contraseñas se almacenan únicamente como hash con bcrypt y un coste de 12, y la contraseña en claro nunca llega a la capa de datos. El coste es un compromiso entre la resistencia a un ataque de diccionario sobre una base filtrada y el tiempo de cómputo por inicio de sesión. La política exige al menos 12 caracteres con mayúscula, minúscula y número; la longitud, y no la complejidad, es el factor que más dificulta el ataque de fuerza bruta, y por eso fue la regla que se fijó más alta. El inicio de sesión responde con un único mensaje ante credenciales incorrectas, sin indicar si el fallo es del correo o de la contraseña. El cambio de contraseña exige conocer la actual.

7.3.2 Sesión

La sesión es un token firmado con una clave secreta del servidor, transportado en una cookie. Auth.js emite por defecto cookies con los atributos de sesión habituales (no accesibles desde JavaScript, restringidas al mismo sitio y marcadas como seguras en conexiones cifradas); la comprobación de estos atributos sobre el entorno desplegado queda para una prueba dinámica. El token contiene el identificador, el nombre, el correo y el rol, pero no la contraseña ni el hash.

7.3.3 Autorización

La autorización es el control más sólido del sistema, y por esa razón la categoría de control de acceso roto del OWASP se valora con exposición baja. Tres propiedades la sustentan, descritas en el apartado 4.8: la regla es única y está en un solo fichero, se evalúa en los servicios contra el propietario persistido en la base de datos, y el actor procede de la sesión del servidor y nunca de datos enviados por el cliente. El equipo de Next.js recomienda precisamente este modelo para aplicaciones nuevas: concentrar el acceso a datos en una capa en la que cada función recibe al usuario actual y comprueba qué puede ver, y tratar los argumentos de las Server Actions como entrada no confiable que debe verificarse (Markbåge, 2023). Las 13 acciones con datos de usuario vuelven a comprobar la sesión, de modo que la protección no depende solo del layout de las páginas.

7.3.4 Validación de entrada e inyección

Toda entrada se valida en el servidor con esquemas de Zod, incluidos el tipo, el formato y los rangos. El acceso a datos se hace exclusivamente con Prisma, que parametriza las consultas, y no hay en el código ninguna consulta escrita a mano ni se interpola texto del usuario en una consulta, lo que elimina la inyección de SQL como vector. La única petición saliente a un servicio externo con datos del usuario es la consulta a Nominatim, que se envía a una dirección fija y el usuario solo aporta el texto de la consulta, de modo que queda descartada la falsificación de peticiones del servidor en su forma habitual.

7.3.5 Peticiones falsificadas y scripts en el navegador

Las Server Actions de Next.js solo se invocan mediante POST y el framework compara la cabecera de origen con la del servidor, rechazando las peticiones que proceden de otro sitio. No usan tokens anti-CSRF, de modo que la defensa depende de ese mecanismo y de las cookies restringidas al mismo sitio (Markbåge, 2023). Esa descripción está documentada para la versión 14 del framework; no se ha comprobado de forma dinámica en la versión 15 utilizada. Frente a la inyección de scripts, React escapa por defecto el contenido que se muestra, y el código no utiliza el mecanismo que desactiva ese escape, por lo que los comentarios, nombres y descripciones introducidos por los usuarios se muestran como texto. El mismo documento advierte de que, al no usar tokens anti-CSRF, el saneamiento del contenido mostrado resulta decisivo, lo que da más peso a la carencia de cabeceras que se describe más adelante.

7.3.6 Secretos y servicios externos

Las claves de Cloudinary, la clave de firma de sesiones y la cadena de conexión a la base de datos se leen de variables de entorno en el servidor y no están en el repositorio; el fichero de entorno está excluido del control de versiones. Las imágenes remotas que la aplicación acepta mostrar se limitan al dominio de Cloudinary.

7.3.7 Errores y mensajes

Las acciones devuelven mensajes genéricos al usuario, sin trazas ni detalles internos, y traducen los errores esperados a mensajes comprensibles. Next.js sustituye además el mensaje de los errores del servidor por un identificador en producción.

7.3.8 Pruebas de seguridad

La batería de pruebas incluye casos de permisos, de acceso sin sesión y de rechazo de operaciones sobre recursos ajenos en los servicios y acciones, y comprobaciones del cálculo y verificación de hashes. No incluye pruebas dinámicas de seguridad ni casos específicos para la limitación de intentos o las cabeceras, que no existen todavía, y las pruebas de extremo a extremo no cubren el acceso autenticado (capítulo 10).

7.4 Evaluación según el OWASP Top 10

La tabla 7.1 resume la exposición en cada categoría. El nivel se asigna según la presencia de hallazgos y su riesgo.

Tabla 7.1. *Exposición por categoría del OWASP Top 10 (2021).*

Categoría	Exposición	Hallazgos	Comentario
A01 Control de acceso roto	Baja	H4	Protección en layout y acciones; autorización por propietario desde los servicios. El riesgo es de regresión.

Categoría	Exposición	Hallazgos	Comentario
A02 Fallos criptográficos	Baja	H3	Hash con bcrypt y sesión firmada; falta forzar HTTPS con la cabecera correspondiente.
A03 Inyección	Baja	H7, H8	Zod y Prisma sin SQL crudo; faltan longitudes máximas y comprobación de la firma de los archivos.
A04 Diseño inseguro	Baja	H5, H7, H8	Arquitectura sólida con carencias puntuales.
A05 Configuración de seguridad incorrecta	Media	H3, H4, H9	Cabeceras ausentes, middleware sin protección y errores no homogéneos.
A06 Componentes vulnerables	Media	H10	18 vulnerabilidades en el análisis de dependencias, en su mayoría en herramientas de desarrollo.
A07 Fallos de identificación y autenticación	Media	H1, H2, H5, H6	Sin límite de intentos, sesión de 30 días y rol copiado en el token.
A08 Fallos de integridad	Baja	H8, H10	Pipeline sin controles; subidas validadas por el tipo declarado.
A09 Fallos de registro y monitorización	Baja	H9, H11	Sin registro estructurado de eventos.
A10 Falsificación de peticiones del servidor	No aplica	—	Única petición saliente a una dirección fija.

La exposición media se concentra en tres categorías que dependen de configuración, proceso y automatización más que del diseño. Es coherente con un proyecto que ha dedicado el esfuerzo a la lógica de negocio y no ha pasado todavía por una fase de endurecimiento.

7.5 Hallazgos

La revisión identificó doce hallazgos: ninguno crítico ni alto, cuatro de severidad media, siete bajos y uno informativo. La puntuación máxima es 12 sobre 25. Ninguno permite por sí solo comprometer el sistema en el alcance revisado; son carencias de endurecimiento que habría que cerrar antes de abrir el servicio a un público amplio. La tabla 7.2 los recoge con su estado en la versión entregada.

Tabla 7.2. *Hallazgos de la revisión de seguridad.*

ID	Hallazgo	P × I	Severidad	Estado
H1	Ausencia de limitación de intentos en acceso y registro	4 × 3 = 12	Media	Pendiente
H3	Ausencia de cabeceras HTTP de seguridad	4 × 3 = 12	Media	Pendiente
H2	Duración de sesión de 30 días sin control de revocación	3 × 3 = 9	Media	Pendiente

ID	Hallazgo	P × I	Severidad	Estado
H4	Middleware sin protección de rutas	2 × 4 = 8	Media	Pendiente
H5	Recuperación de contraseña no implementada	3 × 2 = 6	Baja	Pendiente
H6	Enumeración de cuentas en el registro y por tiempos de respuesta	3 × 2 = 6	Baja	Pendiente
H8	Validación de archivos basada en el tipo declarado	2 × 3 = 6	Baja	Pendiente
H9	Gestión de errores no homogénea	3 × 2 = 6	Baja	Pendiente
H10	Controles de seguridad ausentes en el pipeline y en las dependencias	2 × 3 = 6	Baja	Pendiente
H7	Longitudes máximas no acotadas en varios campos	2 × 2 = 4	Baja	Pendiente
H11	Sin registro de eventos de seguridad	2 × 2 = 4	Baja	Pendiente
H12	Observaciones generales de madurez	1 × 1 = 1	Informativa	—

7.5.1 Hallazgos de severidad media

H1. Limitación de intentos. Ni el inicio de sesión ni el registro limitan cuántas peticiones puede enviar un mismo origen o una misma cuenta. Cada intento ejecuta la comparación de bcrypt, de modo que además de permitir fuerza bruta y relleno de credenciales, el acceso es un amplificador de denegación de servicio, y las altas masivas llenarían la base de datos. La política de contraseñas reduce la probabilidad de éxito del ataque, pero no el abuso del servicio. La solución requiere un almacén compartido y no la memoria de un proceso, porque en un despliegue sin servidor cada petición puede ejecutarse en una instancia distinta. Es el hallazgo de mayor puntuación junto con H3.

H3. Cabeceras de seguridad. La aplicación no envía la política de seguridad de contenido, la que fuerza HTTPS, la que impide la incrustación en marcos, ni las de tipo de contenido, referencia y permisos. No son vulnerabilidades por sí mismas, sino una segunda línea de defensa: con una política de contenido, un fallo de inyección futuro tendría un alcance mucho menor, y sin la restricción de incrustación la aplicación podría mostrarse dentro de una página ajena para engañar a un usuario y hacerle pulsar «Eliminar». Dado que Server Actions no usa tokens contra peticiones falsificadas, esta capa adicional cobra más relevancia. Su corrección es de bajo esfuerzo y el mejor coste-beneficio de la revisión. Al tratarse de una aplicación de accesibilidad, cualquier política de contenido ha de probarse con tecnologías de apoyo antes de volverla bloqueante.

H2. Sesión. La sesión usa el valor por defecto de la biblioteca, de 30 días de vigencia absoluta, sin caducidad por inactividad ni posibilidad de revocarla desde el servidor. El rol se copia al token al iniciar sesión y no se vuelve a leer, por lo que una degradación de privilegios o el

borrado de una cuenta no surte efecto hasta que el token caduca, y cambiar la contraseña no invalida los tokens ya emitidos. El riesgo es un multiplicador de otros ataques, porque un token robado sirve durante un mes. Se corrige acotando la vigencia, releyendo el rol cada cierto tiempo y añadiendo una versión de token por usuario.

H4. Middleware sin protección. El middleware solo refresca la sesión y deja pasar toda petición. Hoy el sistema está protegido por el layout y por las acciones, y no existe ninguna ruta abierta, pero el comportamiento es abierto por defecto: una ruta creada fuera del grupo protegido, o un cambio en el layout, dejarían contenido accesible sin que nada lo detectara. Con una probabilidad baja y un impacto alto, se clasifica como medio por su gravedad potencial y por contradecir una recomendación del propio framework, que aconseja que el middleware limite el acceso con una lista de rutas permitidas y no con una lista de excluidas (Markbåge, 2023).

7.5.2 Hallazgos de severidad baja e informativa

H5. Recuperación de contraseña. El flujo de recuperación está planificado y no implementado, de modo que quien olvide su contraseña pierde el acceso. Que esté pendiente es en sí un riesgo por la forma en que se acabe implementando, ya que los flujos de recuperación improvisados son una fuente conocida de tomas de cuenta.

H6. Enumeración de cuentas. El registro informa de que un correo ya existe y asocia el error al campo, lo que permite comprobar si una persona concreta tiene cuenta. Además, el inicio de sesión no ejecuta el cálculo del hash cuando el usuario no existe, por lo que su tiempo de respuesta delata la existencia de la cuenta, con una diferencia medible. Es una fuga de información acotada que adquiere relevancia en una plataforma sobre accesibilidad.

H7. Longitudes máximas. Los nombres están acotados a 100 caracteres, pero no la contraseña, el correo, la dirección, la descripción ni el comentario de la valoración. El único límite global es el del cuerpo de las acciones. Además, bcrypt solo considera los primeros 72 bytes de la contraseña. El efecto es sobre la disponibilidad: consumo de cómputo, memoria y almacenamiento.

H8. Archivos subidos. El tipo de archivo se valida con el tipo que declara el navegador, sin comprobar los primeros bytes del contenido ni aplicar una cuota por usuario. El riesgo es bajo porque el servidor nunca interpreta el archivo, sino que lo reenvía a Cloudinary, que solo acepta imágenes. El impacto se limita a contenido no deseado y consumo de la cuota del servicio externo.

H9. Errores. Los mensajes al usuario son correctos, pero el tratamiento es desigual: solo hay un registro de errores en todo el código, y los demás fallos se descartan sin dejar rastro. No hay

páginas de error propias para fallos inesperados de renderizado. Es una carencia de observabilidad más que de confidencialidad.

H11. Eventos de seguridad. No se registran inicios de sesión correctos ni fallidos, cambios de contraseña, accesos denegados ni borrados. No añade una vía de ataque, pero impide detectar un incidente y reconstruirlo después.

H12. Observaciones. Agrupa las observaciones que no son vulnerabilidades: la revisión es estática, no hay un documento de política de seguridad ni un modelo de amenazas, las pruebas carecen de casos negativos de seguridad, la biblioteca de autenticación está en versión beta y no se ha hecho una revisión del tratamiento de datos personales frente al reglamento europeo.

7.6 Dependencias y entrega continua

El análisis de dependencias realizado sobre el fichero de bloqueo informa de 18 vulnerabilidades: 2 críticas, 14 altas y 2 moderadas. La mayoría procede de herramientas de desarrollo y de prueba, como el ejecutor de pruebas y el generador de componentes de interfaz, que está declarado como dependencia de ejecución aunque se usa solo como utilidad de línea de comandos. De las que afectan al código que se ejecuta en producción, la única identificada es una dependencia anidada de Next.js con severidad moderada.

El pipeline de integración continua ejecuta el analizador estático, las pruebas y la compilación en cada cambio, pero no incluye análisis de dependencias, de secretos ni de código, no declara permisos mínimos, no fija las acciones por su huella y no ejecuta las pruebas de extremo a extremo ni las de accesibilidad. No hay un proceso automático que avise de vulnerabilidades nuevas.

Este hallazgo (H10) se mantiene como de severidad baja y no media, a pesar de que su corrección es aconsejable. El motivo es que las vulnerabilidades detectadas están en paquetes que no se ejecutan en el servicio desplegado, por lo que no existe una vía de explotación directa, y que la ausencia de controles automáticos es una carencia de proceso que por sí sola no incrementa el riesgo operativo del sistema. La severidad debería revisarse si alguna dependencia de ejecución, como Next.js o Auth.js, resultara afectada. Esta última sigue en versión beta, lo que añade un motivo para vigilar las actualizaciones.

7.7 Protección de datos personales

La aplicación trata el nombre y el correo de cada cuenta, y asocia a cada persona sus contribuciones. Incluye páginas de aviso legal, de política de privacidad y de política de cookies, pero no se ha realizado una revisión del tratamiento conforme al Reglamento General de Protección de Datos, que incluiría los tratamientos que implican a Cloudinary y a los servicios de OpenStreetMap, ni se ha evaluado si procede una evaluación de impacto. Se recoge

como acción pendiente (H12) y se retoma en el capítulo 13. Tampoco existe, por ahora, ninguna función para que una persona exporte o elimine sus datos desde la aplicación.

7.8 Riesgo residual

La tabla 7.3 compara el riesgo de cada escenario de amenaza en la versión entregada con el esperado si se aplicaran las recomendaciones de la revisión.

Tabla 7.3. *Riesgo residual por escenario de amenaza.*

Escenario	Riesgo actual	Riesgo tras remediar	Medidas
Fuerza bruta y relleno de credenciales	Medio	Bajo	H1, H6, H11
Secuestro o reutilización de sesión	Medio	Bajo	H2, H3, H4
Acceso sin autenticar por ruta sin protección	Medio	Bajo	H4 y pruebas de acceso
Explotación de inyección de scripts o de incrustación en marcos	Medio	Bajo	H3, H7
Compromiso por dependencia o pipeline	Bajo	Informativo	H10
Incidente no detectado	Bajo	Informativo	H9, H11

Las medidas correspondientes a los hallazgos de severidad media requieren, según la estimación de la revisión, entre tres y cuatro días de trabajo, sin rediseño de la arquitectura. Eso indica que la distancia entre el estado actual y uno apto para un despliegue con usuarios reales es de días de trabajo, y no de meses, porque las carencias están en la configuración y el proceso, y no en el diseño.

7.9 Limitaciones de la evaluación y síntesis

La evaluación tiene tres limitaciones. Es estática: no incluye pruebas de intrusión ni análisis dinámico sobre el sistema desplegado, que podrían revelar problemas de configuración de la infraestructura invisibles en el código, y por eso se propone una prueba dinámica y repetir la revisión después de las correcciones. No cubre la seguridad de los proveedores. Y las puntuaciones de probabilidad e impacto son un juicio del evaluador sobre una escala común, no una medición, de modo que dos revisores podrían discrepar en uno o dos puntos sin cambiar la conclusión de conjunto.

La conclusión es que el sistema presenta un nivel de seguridad aceptable para el alcance de un trabajo fin de máster: el diseño es correcto en lo que más importa, que es la separación entre validación, autorización y datos, y las carencias detectadas son de endurecimiento. «Aceptable» no equivale a «listo para producción abierta». Antes de un despliegue con usuarios reales deberían cerrarse los cuatro hallazgos de severidad media, activar un registro básico de eventos

y completar la recuperación de contraseña. El estado del despliegue actual y sus condiciones se describen en el capítulo 11.

8. Accesibilidad

8.1 Enfoque

Una aplicación cuyo objeto es documentar la accesibilidad de los lugares no puede ser inaccesible para las personas a las que se dirige. Por eso la accesibilidad de la propia interfaz se planteó como requisito desde el principio (RNF-A1 a RNF-A6, capítulo 3) y no como un complemento. Este capítulo describe las decisiones tomadas, mide el resultado y expone los defectos que esa medición ha encontrado.

El criterio de referencia son las Pautas de Accesibilidad para el Contenido Web (WCAG) 2.2 (W3C, 2023), que organizan los requisitos en criterios de éxito con tres niveles de conformidad (A, AA y AAA). Se toma como objetivo el nivel AA, que es el habitual en la normativa y en la práctica profesional. **El proyecto no declara conformidad con WCAG 2.2 AA.** Declararla exigiría una evaluación completa con tecnologías de apoyo reales y con personas usuarias, que no se ha hecho. Lo que se presenta es una evaluación parcial, con dos instrumentos y sus límites:

- Una auditoría automática con axe-core (Deque Systems, s. f.), un motor de reglas que detecta una parte de los fallos de accesibilidad. Las propias herramientas de este tipo reconocen que solo identifican una fracción de los problemas posibles, de modo que un resultado sin violaciones no equivale a conformidad.
- Una revisión manual de los criterios que la herramienta no cubre: teclado, foco, reflujo a 320 píxeles, estructura de encabezados, tamaño de los objetivos táctiles y comportamiento del mapa.

Las mediciones se realizaron sobre una copia de la aplicación con una base de datos de prueba que contiene establecimientos de demostración, cuatro cuentas de distinto rol, valoraciones y fotografías, y con un navegador Chromium automatizado en dos ventanas de visualización: escritorio (1280 × 800 px) y móvil (375 × 812 px). La copia es funcionalmente equivalente a la aplicación entregada. La salvedad es que los servicios externos (teselas de OpenStreetMap, Cloudinary) no son accesibles desde el entorno de medición y se sustituyeron por datos locales, lo que afecta a los resultados en el mapa (apartado 8.7).

8.2 Decisiones de diseño

Las medidas de accesibilidad que incorpora la interfaz son estas, ordenadas por el principio de WCAG al que sirven.

Perceptible. Cada página declara el idioma del documento como español (`lang="es"`), lo que permite a los lectores de pantalla elegir la voz adecuada (criterio 3.1.1). Las imágenes de fotografías llevan texto alternativo. La información de la valoración no depende del color: cada puntuación se muestra con su valor numérico y su etiqueta verbal («3 — Buena»), y la media

aparece como número. El mapa no es la única vía a los datos de ubicación: la ficha de cada establecimiento muestra la dirección, el municipio y la provincia en texto, y el mapa global incluye un resumen textual del número de establecimientos representados. El componente de formularios usa texto de ayuda y mensajes de error visibles, no solo cambios de color.

Operable. El primer elemento enfocable de cada página es un enlace «Saltar al contenido» que lleva al contenedor principal (criterio 2.4.1). Los controles son elementos nativos (enlaces, botones, campos y botones de opción) o componentes de la biblioteca de interfaz con soporte de teclado, y el indicador de foco es visible en todos los controles interactivos que se comprobaron. El mapa tiene desactivado el zoom con la rueda del ratón para que no capture el desplazamiento de la página, y ofrece botones de acercar y alejar. Cada página tiene un título propio con el formato «Sección | Accesibilidad 360» (criterio 2.4.2) y un único encabezado de primer nivel.

Comprensible. Los campos de los formularios tienen etiqueta asociada y, donde procede, el atributo `autocomplete` correcto (nombre, correo, contraseña nueva y actual), lo que cubre el criterio 1.3.5. Cuando un formulario se envía con datos no válidos, el foco se traslada al primer campo con error, que queda marcado con `aria-invalid` y enlazado con su mensaje mediante `aria-describedby`; los mensajes tienen el rol `alert`, de modo que los lectores de pantalla los anuncian (criterios 3.3.1 y 3.3.3). En el formulario de valoración, la escala de 0 a 5 se explica en una leyenda y cada botón de opción lleva una etiqueta accesible que incluye el valor y su significado, de forma que el lector no anuncia un número suelto.

Robusto. Se usan puntos de referencia (*landmarks*) de HTML: cabecera, navegación principal con nombre, contenido principal, pie y navegación legal con nombre. Las páginas autenticadas añaden la navegación de accesos rápidos del panel. La estructura de encabezados es jerárquica y sin saltos en todas las páginas revisadas (tabla 8.2).

8.3 Resultados de la auditoría automática

Se ejecutó axe-core con las etiquetas de WCAG 2.0, 2.1 y 2.2 de niveles A y AA sobre 17 estados de página en cada una de las dos ventanas de visualización: las páginas públicas, la página inexistente (404), el panel, el listado, el listado con búsqueda, el alta, el mapa, el perfil, la ficha de un establecimiento, su edición, la administración y el registro con errores de validación mostrados. La tabla 8.1 resume el resultado.

Tabla 8.1. *Resultado de axe-core por estado de página (idéntico en escritorio y en móvil).*

Estado de página	Violaciones	Observaciones
Inicio, acceso, registro, registro con errores, aviso legal, privacidad, cookies, 404	0	
Listado, listado con búsqueda, alta, edición, perfil	0	

Estado de página	Violaciones	Observaciones
Mapa	0	4 comprobaciones de contraste no concluyentes (5 en móvil), por las teselas sustituidas
Panel	1 regla (1 nodo)	Contraste insuficiente de texto secundario sobre fondo gris
Ficha de establecimiento	3 reglas (9 nodos)	Contraste insuficiente de los botones de acción destructiva (2 nodos) y estructura no válida de la lista de definiciones (7 nodos)
Administración	1 regla (10 nodos)	Contraste insuficiente de los botones de acción destructiva

Los datos completos de la medición figuran en el anexo F. Catorce de los 17 estados no presentan violaciones en ninguna de las dos ventanas. Los tres restantes presentan violaciones de nivel «serio»: la regla de contraste de color en el panel, en la ficha y en la administración, y además dos reglas de estructura de la lista de definiciones en la ficha. Se analizan en el apartado 8.5. Los resultados son los mismos en escritorio y en móvil, lo que indica que el diseño adaptable no introduce fallos adicionales detectables por la herramienta.

8.4 Revisión manual

La tabla 8.2 recoge las comprobaciones manuales y su resultado. Las que aparecen como «Defecto» se desarrollan en el apartado 8.5.

Tabla 8.2. *Comprobaciones manuales de criterios que axe-core no cubre.*

Comprobación	Criterio WCAG 2.2	Resultado
Idioma del documento	3.1.1 (A)	Correcto: `lang="es"`
Título de página único y descriptivo	2.4.2 (A)	Correcto en las 12 páginas revisadas; la ficha usa un título genérico («Ficha de establecimiento») y no el nombre del lugar
Un solo `h1` y jerarquía de encabezados	1.3.1, 2.4.6	Correcto
Enlace de salto al contenido	2.4.1 (A)	Visible al recibir foco; el foco no se traslada al destino (defecto menor)
Etiquetas y `autocomplete` en formularios	1.3.1, 1.3.5, 3.3.2	Correcto
Foco en el primer campo erróneo, error asociado y anunciado	3.3.1	Correcto
Indicador de foco visible	2.4.7 (AA)	Correcto en enlaces, botones, campos y opciones; en botones y áreas de texto se dibuja con sombra y no con contorno

Comprobación	Criterio WCAG 2.2	Resultado
Escalado: el visor permite ampliar	1.4.4 (AA)	Correcto: la etiqueta `viewport` no bloquea el zoom
Reflujo a 320 px sin desplazamiento horizontal	1.4.10 (AA)	Defecto en la página de inicio; 1 px de exceso en listado y mapa
Uso del mapa solo con teclado	2.1.1 (A)	Defecto: los marcadores no reciben foco
Estructura de la lista de datos de la ficha	1.3.1 (A)	Defecto: estructura no válida
Alternativa textual de las fotografías	1.1.1 (A)	Limitada: el texto describe el establecimiento, no la imagen
Tamaño mínimo de los objetivos	2.5.8 (AA)	Enlaces del pie y del listado de 19 px de alto; botones de opción de 13 px; véase 8.5

8.5 Defectos detectados

Se detectaron siete defectos. Se ordenan por la gravedad que tiene para quien los sufre, no por el resultado de la herramienta.

1. Los marcadores del mapa no son accesibles con teclado (criterio 2.1.1, nivel A). Los marcadores son círculos vectoriales (SVG) de Leaflet que no reciben foco: en la comprobación, la secuencia de tabulación sobre la página del mapa recorre el contenedor del mapa, los botones de zoom y los enlaces de atribución, pero ninguno de los seis marcadores presentes. En consecuencia, una persona que navega solo con teclado no puede abrir la ventana de un marcador ni llegar desde ahí al enlace «Ver ficha». Esto contradice lo que afirma el requisito RNF-A5. Tampoco hay en la página del mapa una lista textual de los establecimientos que sirva de alternativa. Existe, sin embargo, un camino equivalente: el listado de establecimientos, con búsqueda y filtros, es plenamente navegable y lleva a las mismas fichas. Por eso el defecto limita la comodidad pero no bloquea el acceso a la información. Es el defecto de mayor gravedad conceptual, porque el mapa es una de las funciones centrales del producto. La corrección consiste en dotar a cada marcador de `tabindex` y rol de botón, o en ofrecer bajo el mapa una lista de los lugares con los mismos enlaces; la segunda opción es la más robusta, porque beneficia además a quien usa lector de pantalla.

2. Contraste insuficiente en los botones de acción destructiva (criterio 1.4.3, nivel AA). Los botones «Eliminar», en la ficha y en la administración, usan texto rojo (#e7000b) sobre un fondo rojo claro (#fde6e7), con una relación de contraste de 4,0:1 a 14 px, por debajo del mínimo de 4,5:1. Afecta a 2 elementos en la ficha y a 10 en la administración. Se corrige oscureciendo el texto o aclarando el fondo de la variante.

3. Contraste insuficiente del texto secundario en el panel (criterio 1.4.3, nivel AA). El color de texto secundario (#737373) sobre el gris de fondo de las tarjetas (#f5f5f5) da 4,34:1. Es un fallo por un margen muy pequeño, que se resuelve con un gris algo más oscuro para el texto secundario. Como ese color se usa en toda la interfaz, conviene revisar su uso sobre cualquier fondo gris.

4. Estructura no válida de la lista de datos de la ficha (criterio 1.3.1, nivel A). El bloque de datos del establecimiento (dirección, categoría, etc.) usa una lista de definiciones en la que los pares de término y descripción están envueltos en contenedores que añaden un icono. Esa agrupación no es la estructura permitida de HTML y puede hacer que los lectores de pantalla no anuncien la relación entre término y descripción. La herramienta automática la detecta con dos reglas (la lista contiene elementos no permitidos y los términos y descripciones quedan fuera de una lista válida), que afectan a siete nodos de la ficha. Se corrige agrupando cada par en un único contenedor admitido y dejando el icono decorativo fuera del flujo accesible.

5. Reflujo a 320 px (criterio 1.4.10, nivel AA). Con una anchura de 320 px, la página de inicio desborda en horizontal (el documento mide 369 px), y el listado y el mapa lo hacen por 1 px. Aunque el desbordamiento de 1 px apenas es perceptible, el de la página de inicio obliga a desplazarse en las dos direcciones, cosa que el criterio quiere evitar para quien amplía el texto. No se ha identificado el elemento causante; la hipótesis más probable, por el patrón habitual, es un contenido de la portada sin ajuste de ancho.

6. Texto alternativo de las fotografías insuficiente (criterio 1.1.1, nivel A). Todas las fotografías de un establecimiento llevan como alternativa «Fotografía de {nombre}», sin distinguir una de otra. Para una persona ciega es información útil pero repetida, porque una ficha con cinco fotos anuncia cinco veces lo mismo. El modelo de datos tiene un campo `caption` pensado para esto, que la interfaz no usa. Esta carencia es relevante en una aplicación de accesibilidad, porque las fotografías suelen mostrar justo los accesos y los obstáculos.

7. El enlace de salto no traslada el foco (criterio 2.4.1, nivel A). Al activar «Saltar al contenido», la página se desplaza al contenedor principal, pero el elemento activo sigue siendo el cuerpo del documento. En la mayoría de los navegadores modernos el siguiente tabulador continúa desde el destino, por lo que el efecto práctico suele ser correcto; no se verificó esa conducta en otros navegadores ni con lectores de pantalla. La corrección habitual es dar al contenedor un `tabindex="-1"` y llevarle el foco.

Fuera de estos siete defectos hay dos observaciones de menor peso. La primera es el tamaño de los objetivos de entrada (criterio 2.5.8, nivel AA, que pide al menos 24 × 24 px salvo excepciones): los enlaces del pie y las tarjetas de los listados miden 19 px de alto, y los botones de opción de la valoración miden 13 px. En estos últimos, la etiqueta que los envuelve amplía el área activable, y no se midió su tamaño real, por lo que no se puede afirmar que incumplan el

criterio. La segunda es que el desplazamiento suave que se aplica al llevar al usuario al primer criterio sin puntuar no consulta la preferencia del sistema de reducir el movimiento (criterio 2.3.3, nivel AAA).

8.6 Cobertura de las pruebas automáticas

La suite de extremo a extremo incluye una comprobación con axe-core, pero solo sobre la página de inicio y la de aviso legal (capítulo 10). Esto significa que ninguno de los defectos de la tabla 8.1 podía ser detectado por la integración continua, que además no ejecuta las pruebas de extremo a extremo. El requisito RNF-A6, que pedía una comprobación automática de accesibilidad, está cumplido solo parcialmente: existe, pero con un alcance mínimo. La medición realizada para este capítulo muestra que ampliar esa cobertura a las páginas principales es barato y habría detectado los tres defectos que la herramienta puede identificar (los dos de contraste, 2 y 3, y la estructura de la lista, 4).

8.7 Límites de la evaluación

Esta evaluación tiene límites que condicionan la lectura de los resultados:

- **No se probó con lectores de pantalla.** NVDA, VoiceOver y TalkBack no estaban disponibles en el entorno de medición. Las afirmaciones sobre cómo se anuncian los mensajes de error, la escala de valoración o la leyenda se basan en el marcado y en las pautas, no en la escucha real. Es la limitación más importante.
- **No hubo personas usuarias.** No se hicieron pruebas con personas con discapacidad. En un proyecto sobre accesibilidad, esa validación es la que más aportaría, y queda como trabajo futuro (capítulo 13).
- **El mapa se midió con teselas sustituidas.** Sin acceso a los servidores de OpenStreetMap, las comprobaciones de contraste del mapa quedaron como no concluyentes. Tampoco se evaluó la legibilidad real del mapa base.
- **Las fuentes pueden diferir.** El entorno de medición no pudo descargar la tipografía de la interfaz y usó una sustituta, lo que puede alterar ligeramente las alturas y los desbordamientos. El defecto 5 debería confirmarse en un navegador real.
- **La muestra de páginas es la de los datos de demostración.** Estados no representados, como un establecimiento con muchas fotografías o con nombres muy largos, pueden presentar otros problemas de reflujo.

8.8 Síntesis y mejoras propuestas

La interfaz parte de una base correcta: semántica, formularios bien etiquetados, gestión de errores con anuncio y foco, idioma declarado y 14 de 17 estados de página sin violaciones

automáticas. Pero la medición no respalda una declaración de conformidad AA: hay dos fallos de contraste (defectos 2 y 3), un problema de reflujo (5) y, sobre todo, un mapa que no se puede utilizar solo con teclado (1) en un producto cuya finalidad es la accesibilidad. Es un resultado esperable en un primer desarrollo, y la ventaja de haberlo medido es que cada defecto queda localizado y con una corrección de poco esfuerzo.

La tabla 8.3 ordena las mejoras por impacto y esfuerzo estimado.

Tabla 8.3. *Mejoras de accesibilidad propuestas.*

Prioridad	Mejora	Defecto	Esfuerzo estimado
1	Lista textual de establecimientos bajo el mapa, con enlaces a las fichas, o marcadores enfocables	1	Medio
2	Corregir los contrastes de la variante destructiva y del texto secundario	2, 3	Bajo
3	Corregir la estructura de la lista de datos de la ficha	4	Bajo
4	Eliminar el desbordamiento horizontal a 320 px	5	Bajo
5	Texto alternativo propio de cada fotografía (campo `caption` en el formulario de subida)	6	Medio
6	Trasladar el foco al destino del enlace de salto	7	Bajo
7	Ampliar la prueba de axe-core en CI a las páginas principales, en ambas ventanas	Todos	Bajo
8	Pruebas con lectores de pantalla y con personas usuarias	Límites	Alto

Las mejoras 2, 3, 4, 6 y 7 son de esfuerzo bajo y se pueden abordar en conjunto en pocos días de trabajo; las dos primeras son además las que más reducen la distancia a una declaración de conformidad. El capítulo 12 retoma estos resultados al valorar el cumplimiento de los objetivos del proyecto.

9. Uso de la inteligencia artificial en el desarrollo

9.1 Alcance y principio rector

La aplicación no incorpora inteligencia artificial como funcionalidad: ningún módulo del producto utiliza un modelo de lenguaje ni de aprendizaje automático, y no hay llamadas a servicios de IA en el código ni en las dependencias. La inteligencia artificial se empleó únicamente como herramienta de apoyo durante el desarrollo, y este capítulo describe cómo, con qué límites y con qué efectos.

El principio que gobernó el trabajo es que la IA es un asistente y no sustituye el criterio del desarrollador. Las decisiones técnicas, la validación de cada cambio, la revisión del código, las pruebas, las correcciones y la aceptación final fueron manuales. Esto no es una declaración de intenciones: se tradujo en un procedimiento escrito que las herramientas debían seguir y en comprobaciones que el autor ejecutó él mismo, y que se describen en los apartados siguientes.

La misma norma se aplica a esta memoria. Su redacción se ha apoyado en un asistente de inteligencia artificial, que ha colaborado en la estructura, el análisis del repositorio, la elaboración de las mediciones de los capítulos 7 y 8 y la redacción de los borradores. El autor ha revisado los textos, ha decidido su contenido y responde de él. Las cifras que se citan proceden del código, de las mediciones descritas o de las fuentes de la bibliografía.

9.2 Herramientas utilizadas

Se utilizaron tres herramientas, que se eligieron según disponibilidad y según el tipo de tarea.

Tabla 9.1. *Herramientas de IA utilizadas.*

Herramienta	Modelo	Forma de uso
ChatGPT	GPT-5.5	Conversación en la interfaz web
Claude	Opus o Sonnet, según disponibilidad	Conversación y trabajo sobre el código
OpenCode	Muse Spark 1.3, en su versión gratuita	Agente de programación en terminal sobre el repositorio

OpenCode es un agente de programación de código abierto que se ejecuta en la terminal y trabaja sobre los archivos del proyecto, y Muse Spark es un modelo de la división de superinteligencia de Meta. La combinación de varias herramientas respondió a razones prácticas, como los límites de uso de las versiones gratuitas y la disponibilidad de cada modelo, y no a un diseño experimental. Eso tiene una consecuencia metodológica que se discute en el apartado 9.8: no se puede comparar el rendimiento de unas herramientas con el de otras.

9.3 Tareas en las que se empleó

La IA se utilizó en ocho tipos de tarea, que la tabla 9.2 relaciona con el capítulo de esta memoria donde se ven sus resultados.

Tabla 9.2. *Tareas asistidas por IA y su resultado.*

Tarea	Resultado	Capítulo
Generación inicial de código	Primeras versiones de servicios, acciones, esquemas y componentes	6
Revisión de la arquitectura	Contraste de la estructura por módulos y de la dirección de las dependencias	4
Redacción de especificaciones	Catorce especificaciones funcionales y técnicas	3
Generación de pruebas	Pruebas unitarias, de componentes y de extremo a extremo	10
Revisión de seguridad	Detección de puntos débiles para la auditoría	7
Documentación técnica	Documentos de arquitectura, guías y decisiones	4
Refactorización	Reorganización de código tras una especificación o un error	6
Propuestas de mejora	Alternativas para valorar, que el autor aceptó o descartó	13

Que una tarea aparezca en la tabla no significa que su resultado se aceptara tal cual. Significa que la IA produjo un borrador, una propuesta o una revisión, y que la responsabilidad de decidir qué parte se incorporaba fue del autor.

9.4 Método de trabajo: contexto escrito y desarrollo guiado por especificaciones

El mayor riesgo de usar un asistente de código sin reglas es que cada sesión parta de cero y produzca soluciones distintas para el mismo problema. Para reducirlo, el proyecto incluye un conjunto de documentos que se entregaban a las herramientas como contexto y que funcionan como un contrato de trabajo:

- **Un archivo de instrucciones para agentes** (`AGENTS.md`), que fija el orden de consulta de las fuentes, el flujo de trabajo obligatorio y una lista de prohibiciones. Entre ellas, no inventar requisitos, no usar `any`, no añadir dependencias innecesarias, no eliminar funcionalidad sin autorización y no aplicar soluciones temporales sin señalarlas. El archivo de instrucciones propio de Claude remite a este.
- **Catorce especificaciones** (anexo C) en el directorio `.specs`, repartidas en cuatro de núcleo (producto, modelo de dominio, arquitectura del sistema e infraestructura) y diez de funcionalidades (autenticación, usuarios, establecimientos, valoraciones, fotografías, geocodificación, búsqueda, edición y borrado, mapa y panel). En conjunto suman unas 3.400 líneas.

- **Documentos de ingeniería** en `.ai`: arquitectura, guías de código, principios de ingeniería, definición de terminado, flujo de desarrollo y registro de decisiones.

El flujo que se exigía a la IA seguía siete fases: comprender la especificación, analizar el código existente, presentar un plan con los archivos afectados y los riesgos, implementar con cambios pequeños, escribir las pruebas, validar la compilación, el tipado y el analizador estático, y actualizar la documentación. La regla principal era no implementar nada sin una especificación aprobada, y la definición de terminado exigía además que el cambio compilara, pasara las pruebas y cumpliera los criterios de aceptación. Al cerrar cada tarea, la herramienta debía responder con una estructura fija: resumen, archivos modificados, pruebas, riesgos y siguiente paso recomendado.

Este diseño coincide, en lo esencial, con lo que la práctica llama desarrollo guiado por especificaciones: el requisito se escribe antes que el código y es la referencia con la que se juzga lo que la IA produce. Su valor en este proyecto fue doble. Redujo la variabilidad entre sesiones y herramientas, porque todas partían del mismo texto, y dio al autor un criterio objetivo para aceptar o rechazar una propuesta: si no cumplía la especificación, volvía a revisión.

También tuvo costes, y conviene decirlo. Mantener las especificaciones al día en un proyecto que avanzó deprisa exigía disciplina, y el propio método no llegó a fijar plantillas de petición reutilizables: cada petición a las herramientas se redactó en el momento.

9.5 Control humano

La aceptación del trabajo generado por la IA pasó por varios filtros, que son los que sostienen la afirmación de que la responsabilidad final fue del autor.

Revisión del código. Cada cambio se revisó antes de incorporarlo. El flujo de trabajo pedía cambios pequeños porque son los que hacen viable una revisión real: un cambio de pocas líneas se puede leer entero, uno de varios cientos solo se puede aprobar por confianza. El historial muestra que no siempre se cumplió: junto a correcciones de una docena de archivos hay incorporaciones de funcionalidad completa que superan con mucho esa medida, como la creación del proyecto o la del sistema de valoraciones. En esos casos la revisión descansó más en las pruebas y en las comprobaciones manuales que en la lectura línea a línea.

Verificación automática. La compilación, el tipado estricto, el analizador estático y las 192 pruebas actúan como un segundo control que no depende del juicio de nadie. Una propuesta de la IA que rompía alguna de estas comprobaciones no avanzaba. El capítulo 10 describe esta batería.

Pruebas manuales. El autor probó a mano las funciones de la aplicación en el navegador: el registro y la autenticación, el alta, la edición y el borrado de establecimientos, la subida, el

borrado y la selección de fotografías, la creación y el borrado de valoraciones, los filtros y las búsquedas, el panel, la administración, el perfil, el mapa, las páginas legales y el comportamiento adaptable a distintos tamaños. Además realizó una comprobación visual al terminar cada microfase de interfaz. Este tipo de comprobación detecta cosas que ninguna prueba automática del proyecto cubre, y el capítulo 10 la trata con más detalle.

Decisiones de diseño. Las decisiones con consecuencias, como la política de borrado en cascada, el uso de Cloudinary o la regla única para las Server Actions, están registradas como decisiones de arquitectura con su contexto y sus consecuencias. Que una decisión esté escrita permite revisarla, y varias de ellas incluyen una revisión prevista (por ejemplo, la política de borrado, a revisar antes de una versión 1.0).

9.6 Casos en que la IA no bastó

Un capítulo que solo cuente aciertos no informa. En este proyecto hay evidencia de cuatro situaciones en las que el resultado de la IA, o el trabajo con ella, necesitó corrección humana.

Un error que solo aparecía en producción. Un archivo de acciones de servidor exportaba, además de funciones, un objeto con el estado inicial de un formulario. En desarrollo funcionaba, y en Vercel fallaba con un error de objeto inesperado. La causa es una regla de Next.js (un módulo con la directiva `"use server"` solo puede exportar funciones asíncronas) que el código generado incumplía sin que la compilación ni las pruebas locales lo detectaran. Se detectó al desplegar, y la respuesta fue doble: corregir el caso y fijar una norma para todo el proyecto en el registro de decisiones, que desde entonces se hace respetar a las herramientas por escrito. Es un ejemplo de aprendizaje convertido en regla, y también de que los asistentes producen código plausible que no se ha ejecutado en las condiciones reales.

Cinco correcciones consecutivas en la subida de fotografías. El módulo de fotografías necesitó cinco commits seguidos de reparación. El registro de decisiones documenta que uno de los fallos de este módulo fue el error de exportación descrito arriba. El autor señala que la causa final fue distinta: faltaban en la configuración del entorno las credenciales del servicio de almacenamiento, la clave y el secreto de la API, que solo él podía proporcionar. Este episodio ilustra un límite de cualquier asistente: ante un fallo, intenta repararlo en el código, porque es lo que puede ver, aunque la causa esté en la configuración. Cinco intentos sobre el código para un problema que no estaba en él son un coste real en tiempo y en historial, y la lección práctica es diagnosticar primero dónde está el fallo antes de pedir una corrección.

Carencias de seguridad. La revisión del capítulo 7 encontró cuatro hallazgos de severidad media (limitación de intentos, cabeceras de seguridad, configuración de sesión y protección en el middleware) en código que se había escrito con apoyo de IA. Esas medidas no se mencionan en las especificaciones y la IA, que trabajaba contra ellas, no las incorporó por iniciativa propia.

Un asistente cumple lo que se le pide con bastante fidelidad y tiende a no añadir lo que nadie le pidió, aunque sea necesario. Es una razón para incluir requisitos de seguridad explícitos en las especificaciones y no confiar en que aparezcan.

Defectos de accesibilidad. La evaluación del capítulo 8 encontró siete defectos, entre ellos marcadores del mapa inaccesibles con teclado y contrastes insuficientes. La comprobación automática de accesibilidad del proyecto solo cubría la página de inicio y la de aviso legal, de modo que ni los asistentes ni la integración continua podían detectarlos. De nuevo, lo que no se mide no se corrige.

9.7 Riesgos de usar IA en el desarrollo y cómo se trataron

La tabla 9.3 resume los riesgos principales y las medidas aplicadas, con una valoración de cuánto cubre cada una.

Tabla 9.3. *Riesgos del uso de IA y medidas aplicadas.*

Riesgo	Medida aplicada	Valoración
Código plausible pero incorrecto	Revisión manual, tipado estricto, pruebas, análisis estático	Eficaz con límites: no detectó el error de exportación hasta el despliegue
Inconsistencia entre sesiones y herramientas	Especificaciones y guías como contexto común	Eficaz
Dependencias innecesarias o inseguras	Regla de justificar cada dependencia; auditoría posterior	Parcial: la auditoría de dependencias no es automática (capítulo 7)
Omisión de requisitos no escritos, como seguridad	Auditoría independiente tras la implementación	Detectó los huecos, pero tarde
Fuga de secretos hacia las herramientas	Credenciales en variables de entorno y archivo de ejemplo con valores de relleno	Preventiva; depende de la disciplina del autor
Dependencia excesiva del asistente	Aceptación manual y registro de decisiones	Difícil de medir; se ha tratado como principio
Falta de trazabilidad del origen del código	Ninguna sistemática	Carencia reconocida (apartado 9.8)

9.8 Limitaciones

Este capítulo tiene límites que conviene declarar:

- **No hay un registro sistemático de las conversaciones.** El proyecto no conservó las peticiones ni las respuestas de las herramientas. Por eso no se puede cuantificar qué porcentaje del código procede de la IA ni qué proporción de las propuestas se aceptó, y esta memoria no ofrece cifras al respecto. Lo que sí se puede afirmar son las reglas con que se trabajó y los resultados verificables.

- **No hay comparación entre herramientas.** Se usaron según disponibilidad y para tareas distintas, de modo que no cabe concluir cuál rindió mejor.
- **Las versiones de los modelos cambian.** Los nombres de modelo de la tabla 9.1 identifican lo que se usó, pero el comportamiento de estos servicios evoluciona y los resultados no son reproducibles exactamente.
- **El historial de commits no distingue autoría.** Los cambios no llevan marca de qué herramienta intervino, y todos figuran bajo la cuenta del autor, lo cual es coherente con su responsabilidad, pero no informa sobre el proceso.

9.9 Reflexión y recomendaciones

Del trabajo con estas herramientas se extraen cinco conclusiones, que la memoria sostiene con los casos anteriores.

1. **El contexto escrito rinde más que la elección del modelo.** Con especificaciones y reglas claras, herramientas distintas producían resultados compatibles. Sin ellas, la variabilidad habría sido el problema principal.
2. **Los cambios pequeños hacen posible la revisión.** Es la condición para que el control humano sea real y no formal, y cuando se incumplió la verificación recayó en las pruebas.
3. **La IA no sustituye a la verificación.** Los fallos que se encontraron aparecieron al ejecutar el código en condiciones reales, al auditarlo o al medirlo, y no al leer la propuesta.
4. **Lo que no se especifica no se obtiene.** Seguridad y accesibilidad quedaron por debajo de lo deseable porque no estaban escritas como requisitos verificables.
5. **Ante un fallo, hay que diagnosticar antes de pedir una corrección.** El episodio de las fotografías costó cinco intentos por lo contrario.

Como recomendación para un proyecto posterior, conviene guardar un registro de las peticiones relevantes, añadir a las especificaciones requisitos explícitos de seguridad y accesibilidad con su prueba automática asociada, y mantener la regla de que ninguna propuesta se acepta sin haberla ejecutado.

10. Pruebas

10.1 Estrategia

La estrategia de pruebas combina cuatro niveles. En la base están las pruebas automáticas rápidas y aisladas (esquemas, servicios, acciones y componentes), que se ejecutan en cada cambio. Encima hay un pequeño conjunto de pruebas de extremo a extremo sobre un navegador real. A ambos se suma una comprobación manual de las funciones en el navegador, hecha por el autor al terminar cada fase, y, por último, las evaluaciones específicas de seguridad y de accesibilidad que se describen en los capítulos 7 y 8.

El criterio que guió la escritura de pruebas fue el que fijaba el flujo de trabajo del proyecto (capítulo 9): toda funcionalidad nueva incluye sus pruebas y no se da por terminada si no pasan. En la práctica, eso concentró el esfuerzo en la lógica de negocio y en la autorización, que es donde la arquitectura por capas (capítulo 4) la hace más fácil de probar, y dejó con menos cobertura la interfaz. La contrapartida de esa decisión se cuantifica en el apartado 10.3.

10.2 Herramientas y ejecución

Las pruebas unitarias y de componentes usan Vitest con el entorno de navegador simulado jsdom y Testing Library; las de extremo a extremo, Playwright sobre Chromium con la integración de axe-core para accesibilidad. Los comandos de ejecución son `npm test` (Vitest) y `npm run test:e2e` (Playwright).

Dos mecanismos automáticos protegen cada cambio. Antes de cada commit, un gancho local (Husky con lint-staged) ejecuta el analizador estático y el formateador sobre los archivos modificados. En el repositorio remoto, un flujo de integración continua en GitHub Actions ejecuta en cada incorporación a la rama principal y en cada solicitud de cambios, sobre Node 22, la instalación limpia de dependencias, el analizador estático, las pruebas de Vitest y la compilación de producción. Dos límites importantes: el gancho local no ejecuta las pruebas, y la integración continua no ejecuta las de extremo a extremo.

10.3 Pruebas automáticas

10.3.1 Inventario

La suite de Vitest consta de 192 pruebas en 46 archivos, y su ejecución completa pasa. En la medición hecha para esta memoria tardó 57 segundos, de los cuales la mayor parte es la preparación del entorno simulado y no la ejecución de las pruebas, que suman unos 3 segundos. La tabla 10.1 las distribuye por capa.

Tabla 10.1. *Pruebas de Vitest por capa.*

Capa	Archivos	Pruebas	Qué se comprueba
Esquemas de validación	7	34	Valores válidos y límites rechazados: longitudes, formatos, rangos de puntuación, contraseñas
Servicios	14	83	Reglas de negocio, errores tipados, cálculo de medias y resúmenes, búsqueda, geocodificación
Acciones de servidor	13	47	Sesión obligatoria, validación, traducción de errores de servicio a mensajes, resultado devuelto
Componentes	6	9	Formulario de valoración, tarjetas, avisos y enlace de vuelta
Utilidades, permisos y sesión	6	19	Reglas de propiedad y rol, protección de páginas, configuración de autenticación
Total	**46**	**192**	

Las pruebas se nombran con el identificador de la especificación que verifican (por ejemplo, «SPEC-080» para edición y borrado), lo que da una trazabilidad básica entre requisito y prueba: 41 de los 46 archivos citan al menos una especificación.

10.3.2 Técnica

Los servicios y las acciones se prueban de forma aislada. El cliente de la base de datos se sustituye por un doble (*mock*) en los 24 archivos de prueba que lo necesitan, y en las acciones se sustituye también la función que devuelve la sesión. Así, una prueba de acción construye una sesión con un rol determinado, llama a la acción con datos válidos o inválidos y comprueba el resultado y las llamadas que se han hecho al servicio. La geocodificación se prueba sustituyendo la función de red, con casos de respuesta correcta, vacía, con error HTTP, con coordenadas no numéricas y con caída de la red.

Este enfoque tiene ventajas claras (rapidez, determinismo, ausencia de infraestructura) y una limitación que debe quedar a la vista: **ninguna prueba se ejecuta contra una base de datos real**. Lo que se verifica es que el servicio llama a la base de datos del modo esperado, no que la base de datos responda como se espera. Quedan fuera de la red de seguridad las restricciones definidas en el esquema, como la unicidad de una valoración por persona y establecimiento, las políticas de borrado en cascada o las consultas con agregados, que son precisamente las que más dependen del motor.

Entre las pruebas de componentes hay una que nació de un error real y conviene destacar como práctica: una prueba de reproducción del formulario de valoración. El navegador entrega los valores de los botones de opción como texto, y el esquema de validación debe convertirlos en

números (con una coerción explícita), de modo que la puntuación llegue como número y «No aplicable» como nulo. La prueba recrea la situación y comprueba esos dos casos, de modo que el error no puede reaparecer sin que falle.

Otro detalle del entorno de pruebas es que el simulador de navegador no implementa la lectura de archivos como lo hace un navegador, por lo que las pruebas de fotografías usan una utilidad que añade esa función a los archivos de prueba. Está documentada como de uso exclusivo en pruebas.

10.3.3 Cobertura

Se midió la cobertura con el proveedor V8 de Vitest sobre una copia de la aplicación, porque el proyecto no tiene configurada la medición ni establece un umbral mínimo. Los resultados globales son 39,93 % de sentencias, 83,78 % de ramas y 85,51 % de funciones.

La diferencia entre las sentencias y las otras dos cifras parece una contradicción. Se explica, con toda probabilidad, porque la herramienta cuenta como sentencias todo el código de los archivos que ninguna prueba carga, sobre todo páginas y componentes de interfaz, y en cambio no incluye sus ramas ni sus funciones. La tabla 10.2 desglosa las sentencias por tipo de archivo y lo hace visible.

Tabla 10.2. *Cobertura de sentencias por tipo de archivo.*

Tipo de archivo	Archivos	Sentencias	Cobertura
Esquemas de validación	8	131	100 %
Servicios	13	1.034	90,8 %
Carpetas de acciones	31	530	68,3 %
Componentes de módulos	30	1.578	17,4 %
Componentes de interfaz base	17	435	44,4 %
Utilidades y configuración (lib)	10	94	37,2 %
Páginas de la aplicación	18	898	0 %
Script de carga de datos iniciales	1	140	0 %

La lectura es la que se esperaba por la estrategia: la lógica de negocio y la validación están bien cubiertas, y lo que está por debajo es la capa de presentación. Esto importa porque varios defectos de los capítulos 7 y 8 están justo ahí. Una cifra agregada de 40 % es engañosa en las dos direcciones: infravalora la cobertura de lo que más pesa en la corrección de los datos y sobrevalora la confianza que se puede tener en la interfaz.

Entre los servicios, solo dos quedan sin cubrir o casi: el de almacenamiento de fotografías (23 %), que es la frontera con el servicio externo y por eso se sustituye en las pruebas, y el servicio de criterios, que no tiene pruebas propias.

10.4 Pruebas de extremo a extremo

La suite de Playwright contiene siete pruebas en tres archivos, todas sobre páginas públicas: que la portada muestra el título y que sus botones llevan a iniciar sesión y a crear cuenta, que las tres páginas legales se muestran con su título, que el pie las enlaza y que ofrecen volver a la aplicación, y dos comprobaciones de accesibilidad con axe-core sobre la portada y sobre el aviso legal. Sirven como prueba de humo de que la aplicación arranca y se renderiza, pero no ejercitan ningún flujo con sesión: ni el registro, ni la creación de un establecimiento, ni la valoración, ni la subida de fotografías. Estos flujos, que son el núcleo del producto, solo se verificaron a mano.

10.5 Pruebas manuales

El autor probó a mano las funciones de la aplicación en el navegador, en el orden en que se iban desarrollando y con una revisión visual al terminar cada microfase de interfaz. Las funciones comprobadas fueron las siguientes:

Tabla 10.3. *Funciones comprobadas manualmente.*

Función	Comprobaciones
Registro y autenticación	Alta de cuenta, inicio y cierre de sesión, rechazo de credenciales incorrectas
Establecimientos	Alta, edición, borrado y consulta de la ficha
Fotografías	Subida, eliminación y selección de la principal
Valoraciones	Creación y eliminación
Búsqueda	Búsqueda por texto y filtros
Panel y administración	Resúmenes del panel y gestión de categorías
Perfil	Consulta y cambio de nombre y de contraseña
Mapa	Visualización de la ubicación y del mapa global
Páginas legales	Contenido y enlaces
Adaptación	Comportamiento en distintos tamaños de pantalla

Estas comprobaciones son valiosas, porque cubren justo lo que las pruebas automáticas no alcanzan, pero tienen dos límites que hay que declarar. No se conservó un protocolo escrito con casos y resultados esperados, por lo que no son repetibles de forma sistemática ni se pueden aportar como evidencia detallada. Y la revisión visual la hizo una sola persona, el propio desarrollador, que es la peor situada para ver los problemas de uso que ha dejado de percibir.

10.6 Incidencias detectadas

La tabla 10.4 recoge las incidencias de las que hay constancia en el historial del repositorio o en el registro de decisiones, y cómo se detectaron.

Tabla 10.4. *Incidencias significativas y su detección.*

Incidencia	Cómo se detectó	Resolución
Valores de puntuación recibidos como texto desde el formulario	No consta; se conserva su prueba de reproducción	Coerción en el esquema de validación y prueba de reproducción
Exportación no permitida en un archivo de acciones, que fallaba solo en producción	Despliegue en Vercel	Corrección y regla única para las acciones de servidor (ADR-005)
Fallos al subir fotografías, que exigieron cinco commits de reparación	Uso de la función	Entre las causas, la falta de la clave y el secreto del servicio de almacenamiento en la configuración del entorno
Cuatro hallazgos de seguridad de severidad media	Revisión de seguridad (capítulo 7)	Pendientes; propuestos para su corrección
Siete defectos de accesibilidad	Auditoría con axe-core y revisión manual (capítulo 8)	Pendientes; propuestos para su corrección

Dos de las cinco incidencias (la exportación y la subida de fotografías) se detectaron al desplegar y no antes. Ambas son de las que una prueba automática contra el entorno real habría revelado antes. Las otras tres se encontraron por revisión posterior y no por la batería de pruebas. Es una señal del alcance real de ésta: protege contra regresiones en la lógica, no contra fallos de configuración, de integración con servicios externos ni de interfaz.

10.7 Análisis y mejoras propuestas

El conjunto de pruebas es sólido donde se centró el esfuerzo (reglas de negocio, validación, permisos) y deficiente donde no se centró (interfaz, integración con la base de datos y con servicios externos, flujos completos de usuario). El objetivo de calidad previsto en la introducción, una suite que incluya pruebas de integración y de extremo a extremo ejecutables en la integración continua, se cumple solo en parte: las pruebas de integración propiamente dichas no existen, y las de extremo a extremo existen pero no se ejecutan en el flujo automático ni cubren lo esencial.

La tabla 10.5 ordena las mejoras por la relación entre beneficio y esfuerzo.

Tabla 10.5. *Mejoras propuestas en las pruebas.*

Prioridad	Mejora	Qué resuelve	Esfuerzo
1	Ejecutar las pruebas de extremo a extremo en la integración continua	Regresiones en las páginas servidas	Bajo
2	Ampliar la prueba de accesibilidad a las páginas principales	Defectos del capítulo 8	Bajo
3	Pruebas de integración contra una base PostgreSQL efímera	Restricciones, cascadas y agregados	Medio

Prioridad	Mejora	Qué resuelve	Esfuerzo
4	Flujos de extremo a extremo con sesión: registro, alta, valoración	Núcleo del producto	Medio
5	Configurar cobertura con umbrales por capa, en lugar de un único global	Evitar la regresión de lo que ya está cubierto	Bajo
6	Pruebas de los componentes de formulario y del mapa	Presentación	Medio
7	Pruebas de la limitación de intentos y de las cabeceras, cuando se implementen	Hallazgos del capítulo 7	Bajo
8	Añadir análisis de dependencias a la integración continua	Hallazgo H10	Bajo

Las dos primeras mejoras son de las de menor esfuerzo y reducirían de forma apreciable la distancia entre la intención y lo que la integración continua verifica. La tercera es la que más cambiaría la confianza en la capa de datos.

11. Despliegue

11.1 Entorno de producción

La aplicación está desplegada y accesible públicamente en https://accesibilidad-360.vercel.app. Se ejecuta sobre tres servicios gestionados, sin servidores propios:

- **Vercel** aloja la aplicación Next.js. Sirve las páginas y ejecuta las acciones de servidor y el resto del código de servidor como funciones sin estado.
- **Neon** aloja la base de datos, que es PostgreSQL gestionado. No se ha verificado la versión concreta del motor en producción, por lo que no se cita.
- **Cloudinary** almacena las fotografías y las sirve desde su red de distribución, según la decisión ADR-004 del capítulo 4.

La elección de un entorno gestionado y sin sistema de archivos persistente condicionó el diseño: es la razón de que las fotografías no se guarden en el servidor (el sistema de archivos de Vercel es efímero) y de que el estado de sesión viaje en un token firmado, sin tabla de sesiones (capítulo 7). A cambio, el despliegue no exige administrar infraestructura. El almacenamiento de fotografías usa el plan gratuito de Cloudinary (ADR-004), con los límites de uso que ello implica.

Se comprobó desde el exterior que el despliegue responde y se comporta como se espera: la página de inicio se sirve con su contenido y sus enlaces, la página de acceso se muestra con sus dos campos, y al pedir el panel sin sesión se obtiene la pantalla de inicio de sesión, lo que concuerda con la protección de las páginas privadas descrita en el capítulo 4. No se realizaron pruebas de carga ni se inspeccionaron las cabeceras de respuesta del despliegue, que el entorno de verificación no permitía.

11.2 Configuración

La configuración se entrega al despliegue mediante variables de entorno, sin valores en el código. Son cinco, y no hay otras en producción.

Tabla 11.1. *Variables de entorno de producción.*

Variable	Uso	Observación
`DATABASE_URL`	Cadena de conexión a la base de datos de Neon	Leída por el cliente de Prisma
`AUTH_SECRET`	Clave con la que Auth.js firma los tokens de sesión	Un valor distinto en cada entorno; su exposición permitiría falsificar sesiones
`CLOUDINARY_CLOUD_NAME`	Identificador de la cuenta de almacenamiento	Se usa también para construir las direcciones de las imágenes

Variable	Uso	Observación
`CLOUDINARY_API_KEY`	Credencial de la API de Cloudinary	Solo en el servidor
`CLOUDINARY_API_SECRET`	Secreto de la API de Cloudinary	Solo en el servidor; nunca llega al navegador

El archivo `.env` real está excluido del repositorio y un archivo de ejemplo (`.env.example`) documenta los nombres con valores de relleno. La subida de fotografías se hace siempre desde el servidor, de modo que el secreto no se expone al cliente. Como se indicó en el capítulo 9, la ausencia de la clave y del secreto en la configuración fue una de las causas de los fallos que hubo al subir fotografías al principio.

Otros aspectos de configuración relevantes para producción son que la imagen remota de Cloudinary está autorizada de forma explícita en la configuración de Next.js, que el tamaño máximo del cuerpo de las acciones de servidor se fija en 5 MB, y que la instalación de dependencias genera el cliente de Prisma automáticamente (`postinstall`), requisito para que la compilación en Vercel funcione.

11.3 Proceso de despliegue

El proceso se resume en cinco pasos, que la figura 11.1 representa.

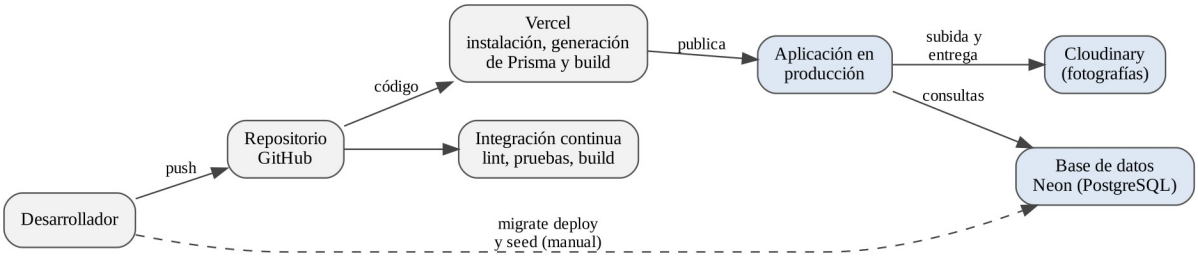


Figura 11.1. Flujo de despliegue y servicios de producción.

1. El código se incorpora al repositorio de GitHub. La integración continua ejecuta el análisis estático, las pruebas y la compilación (capítulo 10).
2. Vercel obtiene el código, instala las dependencias (lo que genera el cliente de Prisma) y ejecuta la compilación de producción de Next.js.
3. El resultado se publica como una nueva versión inmutable de la aplicación, con las variables de entorno configuradas en el proyecto de Vercel.
4. Si la versión incluye cambios en el esquema de la base de datos, las migraciones se aplican a Neon con el comando `prisma migrate deploy`, que ejecuta las pendientes en orden y registra cuáles se han aplicado.
5. Los datos de referencia (10 categorías, 8 criterios, 52 provincias y 8.192 municipios del INE) se cargan con el comando `prisma db seed`.

Hay una característica de este proceso que se debe señalar: **las migraciones no forman parte de la compilación**. El comando de compilación es el estándar de Next.js y no ejecuta `migrate deploy`. Esto es una decisión razonable (aplicar migraciones desde el arranque de una función sin estado es arriesgado), pero deja dos consecuencias. Un cambio de esquema exige un paso manual que puede olvidarse, y en ese caso la aplicación desplegada quedaría ejecutando código que espera columnas que no existen. Y no hay un mecanismo automático que verifique, antes de publicar, que el esquema de la base de datos y el código son compatibles.

El disparador del despliegue en Vercel (automático al incorporar cambios a la rama principal o lanzado manualmente) no se ha documentado en este trabajo, y por eso la figura y la descripción lo presentan de forma genérica.

11.4 Administración y operación

La aplicación no tiene un panel de operación. Las tareas de administración se resuelven así:

- **Asignación de administradores.** Por ahora, el rol de administrador se asigna directamente en la base de datos, sin interfaz. Una consecuencia favorable es que no existe ninguna ruta de la aplicación que permita elevar un rol. La contrapartida es que cada alta de administrador exige acceso a la base de datos.
- **Gestión de categorías y moderación.** Se hace desde la sección de administración de la aplicación, que solo está disponible para ese rol.
- **Copias de seguridad y recuperación.** El proyecto no define ni documenta una política propia. La capacidad de recuperación depende de lo que ofrezca el plan de Neon contratado, que no se ha verificado.
- **Registro y monitorización.** Se dispone de los registros que ofrece Vercel para cada función, pero la aplicación no registra eventos de seguridad ni de negocio (hallazgo H11 del capítulo 7) ni tiene alertas.

11.5 Riesgos y limitaciones del despliegue

Límite de tamaño de las peticiones. La aplicación permite fotografías de hasta 5 MB y configura ese mismo límite para el cuerpo de las acciones de servidor. Según la documentación de Vercel, el tamaño máximo de la petición o de la respuesta de una función es de 4,5 MB, y si se supera se devuelve un error 413 (Vercel, 2026). Si ese límite se aplica al despliegue, una fotografía de entre 4,5 y 5 MB, que la aplicación declara admisible, sería rechazada por la plataforma antes de llegar al código y la persona vería un fallo sin explicación. No se ha reproducido esto en producción, de modo que se registra como un riesgo a verificar y no como un defecto confirmado. La corrección sería alinear el límite de la aplicación con el de la

plataforma o, mejor, subir las fotografías directamente desde el navegador a Cloudinary con una firma emitida por el servidor.

Conexiones a la base de datos. El cliente de Prisma se instancia una vez por proceso. En un entorno de funciones sin estado cada instancia abre sus propias conexiones, y un servicio de base de datos con un número limitado de conexiones puede agotarlas con tráfico moderado. Neon ofrece un modo de conexiones agrupadas para este caso. No se ha verificado si la cadena de conexión de producción usa ese modo, y para el volumen de uso previsto no es un problema, pero es un punto a comprobar antes de un uso real.

Rendimiento de la primera carga. Los listados y el mapa leen todos los registros y el formulario envía al navegador las 8.192 localidades (capítulo 6), lo cual es aceptable con pocos datos, pero crece con el uso. Además, las funciones de Vercel pueden tener un arranque en frío que añade latencia a la primera petición tras un periodo de inactividad.

Dependencia de servicios externos. El funcionamiento completo depende de cuatro servicios de terceros: Vercel, Neon, Cloudinary y los servicios de OpenStreetMap (teselas y geocodificación). Un fallo o un cambio de condiciones en cualquiera degrada la aplicación (por ejemplo, sin Cloudinary no se pueden subir fotografías, y la geocodificación tiene un límite de tiempo de 5 segundos). Las condiciones de los planes gratuitos pueden cambiar sin que el proyecto lo controle.

Seguridad en producción. Los hallazgos del capítulo 7 (limitación de intentos, cabeceras, configuración de sesión y protección en el middleware) afectan al despliegue público. Mientras no se corrijan, el sistema es apto para su uso como prototipo de un trabajo académico y no para uso abierto con usuarios reales.

11.6 Mejoras propuestas

Tabla 11.2. *Mejoras propuestas en el despliegue.*

Prioridad	Mejora	Motivo	Esfuerzo
1	Verificar y alinear el límite de tamaño de las fotografías con el de la plataforma	Riesgo de fallo con archivos de 4,5 a 5 MB	Bajo
2	Aplicar `migrate deploy` en el proceso de publicación, con un paso previo de comprobación	Evitar desajustes entre código y esquema	Medio
3	Añadir las cabeceras de seguridad y la limitación de intentos	Hallazgos H1 y H3	Medio
4	Entorno de vista previa con base de datos propia	Probar antes de publicar	Medio
5	Comprobar el modo de conexión de Neon y definir una política de copias	Resiliencia	Bajo
6	Registro de eventos y alertas básicas	Detección de incidentes (H11)	Medio

Prioridad	Mejora	Motivo	Esfuerzo
7	Revisión del tratamiento de datos conforme al RGPD	Hallazgo H12	Medio

La mayoría son de esfuerzo bajo o medio. El resultado esperable es que el despliegue actual, que es correcto como prototipo, pase a ser apto para un uso real con usuarios.

12. Resultados y discusión

12.1 Qué se ha obtenido

El resultado del trabajo es una aplicación web en producción (https://accesibilidad-360.vercel.app) que cubre el alcance del producto mínimo viable definido en el capítulo 1: registro y acceso de personas usuarias, alta, edición y borrado de establecimientos, valoración por criterio, fotografías, búsqueda y filtrado, mapa por establecimiento y mapa global, panel de resumen, perfil y administración de categorías. El producto comprende 9 entidades de dominio con 8 migraciones, 16 acciones de servidor y unas 6.800 líneas de código TypeScript de aplicación, acompañadas de unas 3.400 líneas de pruebas (192 casos) y de 14 especificaciones escritas antes de implementar.

Este capítulo contrasta ese resultado con lo que se pretendía. Lo hace en tres niveles: los objetivos específicos y el general del capítulo 1, los requisitos del capítulo 3 y la aportación que el capítulo 2 declaraba. Las deficiencias se dan por conocidas y se remiten a los capítulos donde se analizan, sin repetirlas.

12.2 Cumplimiento de los objetivos específicos

La tabla 12.1 valora cada objetivo con tres niveles: cumplido, cumplido con reservas (el criterio se alcanza, pero hay una carencia relevante) y parcial (el criterio no se alcanza por completo). La valoración se hace contra el criterio de cumplimiento que cada objetivo tenía en la tabla 1.1, no contra un criterio más cómodo.

Tabla 12.1. *Valoración de los objetivos específicos.*

Id.	Objetivo	Valoración	Evidencia y reservas
OE1	Modelo de datos relacional	Cumplido con reservas	Esquema migrado y poblado con 52 provincias, 8.192 municipios, 10 categorías y 8 criterios; claves ajenas y unicidades garantizadas por la base de datos (cap. 5). Reserva: los rangos de las puntuaciones no tienen restricciones en la base y dependen de la validación de la aplicación
OE2	Identificación y control de acceso	Cumplido con reservas	Autorización en los servicios, contra el propietario persistido, con pruebas de casos permitidos y denegados (cap. 4, 7 y 10). Reservas: la protección de rutas en el middleware está desactivada y depende de una única comprobación en el diseño de las páginas privadas (H4), y no hay limitación de intentos (H1)

Id.	Objetivo	Valoración	Evidencia y reservas
OE3	Valoración por ocho criterios	Cumplido	Escala de 0 a 5, «No aplicable» en dos criterios, una valoración por persona y establecimiento, media que excluye los no aplicables (cap. 5 y 6)
OE4	Gestión de fotografías	Cumplido con reservas	Subida, selección de la principal y borrado, con almacenamiento externo y validación de formato, peso y dimensiones (cap. 6). Reservas: el formato se verifica solo por el tipo que declara el navegador (H8) y el límite de 5 MB podría chocar con el de la plataforma (cap. 11)
OE5	Geocodificación y mapas	Cumplido con reservas	Mapa por establecimiento y mapa global operativos; sin coordenadas, el establecimiento se crea igualmente (cap. 6). Reserva: los marcadores no son accesibles con teclado (cap. 8)
OE6	Búsqueda y filtrado	Cumplido	Filtros por nombre, categoría, provincia y municipio, con orden por fecha, nombre o valoración y estado en la URL (cap. 6). El ordenamiento por valoración se hace en memoria, lo que limita la escalabilidad (cap. 13)
OE7	Aplicación accesible	Parcial	Base semántica correcta y 14 de 17 estados de página sin violaciones automáticas, pero siete defectos medidos (cap. 8), uno de ellos en un requisito de navegación por teclado, y comprobación automática solo en dos páginas
OE8	Calidad con pruebas e integración continua	Parcial	192 pruebas que pasan, y flujo de integración continua con análisis estático, pruebas y compilación (cap. 10). No hay pruebas de integración contra base de datos, las de extremo a extremo son siete, solo sobre páginas públicas, y no se ejecutan en el flujo automático
OE9	Despliegue en producción	Cumplido	Aplicación pública con base de datos y almacenamiento de imágenes gestionados (cap. 11)
OE10	Análisis crítico de la IA	Cumplido con reservas	Flujo de trabajo, tareas asistidas y control humano descritos, con casos de fallo (cap. 9). Reserva: sin registro de las conversaciones no se puede cuantificar la contribución de cada herramienta

El balance es de tres objetivos cumplidos, cinco cumplidos con reservas y dos parciales: de los diez, ninguno queda sin cumplir, pero solo tres se alcanzan sin matices. Hay un patrón claro en

las reservas. Las funciones del producto (OE3 a OE6, OE9) se cumplen; lo que falla o queda corto es lo que sostiene su calidad: accesibilidad medida (OE7), pruebas que cubran los flujos reales (OE8) y endurecimiento de seguridad (OE2).

12.3 Cumplimiento de los requisitos

Los 23 requisitos funcionales del capítulo 3 están implementados. En los 36 no funcionales el resultado es más desigual, y la tabla 12.2 lo resume por categoría.

Tabla 12.2. *Estado de los requisitos no funcionales por categoría.*

Categoría	Cubiertos	Parciales	No cubiertos	Total
Seguridad	3	2	3	8
Rendimiento	4	0	1	5
Accesibilidad	4	2	0	6
Mantenibilidad	6	0	1	7
Escalabilidad	3	0	2	5
Experiencia de usuario	5	0	0	5
Total	**25**	**4**	**7**	**36**

Lo cubierto es la parte estructural (organización del código, validación, autorización, interfaz en español, confirmación de acciones destructivas). Lo no cubierto es de tres tipos: endurecimiento de seguridad (limitación de intentos, cabeceras, registro de eventos), medición y escalabilidad (objetivos de tiempo de respuesta, paginación, ordenación en la base de datos) y gobernanza de la calidad (umbrales de cobertura exigidos). Ninguno está en la funcionalidad del producto.

12.4 Valoración del objetivo general

El objetivo general era diseñar, implementar y desplegar una plataforma colaborativa que registrara establecimientos, valorara su accesibilidad, compartiera fotografías y los localizara en un mapa, aplicando prácticas que garantizaran su mantenibilidad, su seguridad y su propia accesibilidad.

La primera parte, el producto, se ha logrado: se diseñó, se implementó y se desplegó, y las cuatro funciones nombradas están operativas. La segunda parte, las garantías de calidad, se ha logrado de forma desigual. La **mantenibilidad** es el atributo mejor sostenido: la arquitectura por módulos, la restricción del acceso a la base de datos a diez archivos de servicio, el tipado estricto, las especificaciones y las decisiones registradas son los resultados de más peso del trabajo. La **seguridad** tiene un diseño correcto en lo estructural y carencias de endurecimiento: cuatro hallazgos de severidad media, ninguno crítico ni alto, y un riesgo residual que hace al sistema apto como prototipo y no para uso abierto (capítulo 7). La **accesibilidad** no puede

declararse conforme a ningún nivel, y los defectos medidos incluyen alguno que afecta a la función central del mapa (capítulo 8).

Una valoración honesta es que el objetivo general se cumple en lo que dice «diseñar, implementar y desplegar», y se cumple solo en parte en lo que dice «garantizar». La palabra «garantizar» era la más ambiciosa del enunciado y el trabajo no la sostiene del todo: lo que se puede afirmar es que se aplicaron prácticas que favorecen esas cualidades y que se midieron sus resultados, no que las cualidades estén garantizadas.

12.5 Contraste con la aportación declarada

El capítulo 2 afirmaba cuatro aportaciones: un modelo de valoración por ocho criterios, una valoración por persona y establecimiento, la fotografía como parte de la ficha y una geografía administrativa normalizada. Las cuatro están implementadas y se comportan como se diseñaron.

También declaraba limitaciones frente a las plataformas existentes, y el trabajo las confirma sin matizarlas: la plataforma depende de que las personas aporten datos, no ofrece interfaz de programación ni exportación, exige iniciar sesión para consultar y carece de moderación. Un resultado que el capítulo 2 no podía anticipar es que la aplicación, aun siendo una herramienta de accesibilidad, presenta ella misma barreras para el uso solo con teclado, que es el tipo de problema que la plataforma pretende ayudar a detectar en otros sitios. No invalida la aportación, pero la matiza: la coherencia entre lo que el producto promete y lo que el producto es sigue pendiente de cerrarse.

12.6 Qué funcionó y qué no

Del conjunto de resultados se desprenden cuatro valoraciones sobre el proceso.

Funcionó el desarrollo guiado por especificaciones. Escribir primero el requisito produjo un producto coherente en sus reglas (permisos, valoración, borrado) y redujo la variabilidad entre sesiones de trabajo y herramientas (capítulo 9).

Funcionó la separación en capas. La autorización y las reglas de negocio están en los servicios y son lo mejor cubierto por las pruebas (90,8 % de las sentencias de los servicios), de modo que el núcleo del producto es el más fiable.

No funcionó la verificación de lo que se ve y se configura. Los defectos se concentran en la presentación (accesibilidad), en la configuración (credenciales, exportaciones, límites de plataforma) y en la integración con servicios externos. Es justo lo que las pruebas automáticas no cubren (capítulo 10), y se encontró tarde, al desplegar o al auditar.

No funcionó el tratamiento de los requisitos no funcionales como requisitos medibles. Seguridad y accesibilidad estaban expresadas como buenas prácticas, sin criterio de aceptación comprobable, y por eso se quedaron cortas. Lo que no tiene una prueba asociada se descuida sin que nadie lo note.

12.7 Limitaciones de la evaluación

La valoración de este capítulo tiene límites propios, que se suman a los de cada capítulo anterior.

- **No ha habido personas usuarias.** No se evaluó la usabilidad ni la utilidad de la información con personas con discapacidad ni con ninguna otra. El resultado dice si el producto cumple lo que se diseñó, no si resuelve el problema para quien lo tiene.
- **No hay medición de rendimiento ni de carga.** Los requisitos de rendimiento y escalabilidad se valoran por el análisis del código, y el comportamiento con muchos datos o muchos usuarios no se ha medido.
- **La evaluación la hace el propio autor.** Los criterios y las mediciones son del mismo equipo que desarrolló el producto, con la parcialidad que eso implica. La revisión de seguridad y la de accesibilidad son estáticas o parciales, y no sustituyen una auditoría independiente.
- **Los datos son de prueba.** El producto no tiene aún una base de aportaciones reales con la que valorar la utilidad de la información ni la calidad de las valoraciones.

12.8 Síntesis

El trabajo entrega un producto funcional y desplegado, con una base de ingeniería sólida en la lógica y la organización, y deja identificadas, medidas y priorizadas sus deficiencias en seguridad, accesibilidad, pruebas y despliegue. Que esas deficiencias estén medidas y no ocultas es, en sí, un resultado: permite al lector decidir cuánto confiar en el producto y a quien continúe el trabajo saber por dónde empezar. El capítulo 13 las ordena como trabajo futuro y el capítulo 14 extrae las conclusiones.

13. Trabajo futuro

13.1 Criterio de ordenación

Los capítulos anteriores han ido dejando mejoras propuestas. Este capítulo no las repite una a una: las reúne en una hoja de ruta única, las ordena y añade las líneas de evolución del producto y de investigación que el trabajo no pudo abordar. Cada acción lleva un identificador (T1 a T21) y el capítulo de origen, donde está su justificación.

El orden sigue una regla simple. Primero, lo que hay que cerrar antes de abrir la aplicación a personas reales, que es lo que separa un prototipo de un servicio. Después, lo que consolida la base técnica para que crezca sin degradarse. Más tarde, la evolución funcional, y por último, la validación con personas, que debería acompañar a todas las fases pero que solo da todo su valor cuando el producto está cerrado. El esfuerzo se expresa como bajo (horas a pocos días), medio (una a dos semanas) y alto (más de dos semanas), estimaciones relativas del autor, salvo donde se cita otra fuente.

13.2 Fase 1: cerrar las deficiencias de uso real

Esta fase reúne lo que debería estar resuelto antes de un uso abierto. La revisión de seguridad estimó entre tres y cuatro días para los cuatro hallazgos de severidad media, sin cambios de arquitectura (capítulo 7), y la mayoría de las acciones restantes son de esfuerzo bajo, de modo que la fase completa es abordable en pocas semanas.

Tabla 13.1. *Fase 1: cierre de deficiencias.*

Id.	Acción	Origen	Esfuerzo
T1	Limitación de intentos de acceso y registro, cabeceras de seguridad, configuración explícita de la sesión y protección de rutas en el middleware (H1 a H4)	Cap. 7	Medio
T2	Recuperación de contraseña por correo, homogeneización de las respuestas de registro, longitudes máximas, comprobación de la firma de los archivos y errores homogéneos (H5 a H9)	Cap. 7	Medio
T3	Corregir los siete defectos de accesibilidad: lista textual o marcadores enfocables en el mapa, contrastes, estructura de la ficha, reflujo a 320 px, texto alternativo propio de cada fotografía y foco del enlace de salto	Cap. 8	Medio
T4	Resolver el límite de tamaño de las fotografías frente al de la plataforma, preferentemente subiéndolas desde el navegador a Cloudinary con firma del servidor	Cap. 11	Medio

Id.	Acción	Origen	Esfuerzo
T5	Ampliar el flujo de integración continua: pruebas de extremo a extremo, análisis de dependencias, permisos mínimos y acciones fijadas; aplicar las migraciones en la publicación con una comprobación previa	Cap. 7, 10 y 11	Bajo
T6	Registro estructurado de eventos de seguridad, con alertas básicas (H11)	Cap. 7 y 11	Medio
T7	Revisión del tratamiento de datos personales según el RGPD, y funciones de exportación y eliminación de la propia cuenta, incluidas las fotografías en Cloudinary	Cap. 5 y 7	Medio

Dos observaciones sobre esta fase. La primera es que T3 y T5 se refuerzan entre sí: ampliar la prueba de accesibilidad automática a las páginas principales impediría que los defectos de contraste reaparezcan. La segunda es que T7 contiene una decisión de diseño pendiente, la de qué ocurre con las valoraciones y los establecimientos de una cuenta eliminada. La alternativa de conservarlos de forma anónima exige sustituir la política de borrado en cascada por la de poner a nulo (capítulo 5), y esa decisión tiene consecuencias sobre la moderación y la fiabilidad de los datos, por lo que debería tomarse antes de implementar la eliminación.

13.3 Fase 2: consolidar la base técnica

Una vez cerradas las deficiencias, la segunda fase hace que el producto pueda crecer sin que se degrade ni se rompa en silencio.

Tabla 13.2. *Fase 2: consolidación técnica.*

Id.	Acción	Origen	Esfuerzo
T8	Pruebas de integración contra una base PostgreSQL efímera y flujos de extremo a extremo con sesión (registro, alta, valoración, subida), con umbrales de cobertura por capa	Cap. 10	Medio
T9	Integridad en la base de datos: restricción de rango de las puntuaciones, índice parcial único para la fotografía principal, comprobación atómica del límite de fotografías e índices de consulta para los filtros y la ordenación	Cap. 5	Bajo
T10	Escalabilidad del listado: paginación, ordenación por valoración en la base de datos y media materializada o almacenada, con sincronización al crear y eliminar valoraciones	Cap. 3, 5 y 6	Medio
T11	Carga bajo demanda de los municipios y consulta de los establecimientos del mapa por la zona visible	Cap. 6 y 11	Medio

Id.	Acción	Origen	Esfuerzo
T12	Definir objetivos de rendimiento (tiempos de respuesta, métricas de carga de la interfaz), medirlos y vigilarlos	Cap. 3	Medio

T9 y T10 son de especial interés porque se apoyan: sin índices, la paginación y la ordenación en la base de datos no darían la mejora esperada, y sin ellas las pruebas de T8 no tendrían nada que medir. Una consideración sobre T10: la media materializada devuelve la consistencia a la que renunció el diseño actual (capítulo 5), de modo que solo conviene si el volumen la justifica, y debería acompañarse de una prueba que compare la media almacenada con la calculada.

13.4 Fase 3: evolución funcional

Estas líneas amplían lo que el producto hace. Son las que el capítulo 2 identificó como carencias frente a las soluciones existentes y las que el alcance dejó fuera por decisión.

Tabla 13.3. *Fase 3: evolución funcional.*

Id.	Acción	Origen	Esfuerzo
T13	Consulta pública de establecimientos, valoraciones y mapa sin iniciar sesión, manteniendo el registro para aportar	Cap. 2 y 4	Medio
T14	Moderación: activar el rol de moderador, permitir denunciar contenidos y ocultar o retirar aportaciones	Cap. 2 y 3	Alto
T15	Edición de las valoraciones propias con historial, y ranking que exija un mínimo de valoraciones para evitar que un único voto sitúe un lugar en primera posición	Cap. 3 y 6	Medio
T16	Gestión de roles desde la administración, en lugar de asignarlos en la base de datos	Cap. 11	Bajo
T17	Interfaz de programación pública y exportación de datos, con licencia abierta, y posible intercambio con OpenStreetMap	Cap. 2	Alto
T18	Importación de un directorio inicial de lugares para reducir el arranque en frío	Cap. 2	Alto
T19	Líneas previstas en el alcance original: favoritos, internacionalización, aplicación móvil o instalable, y rutas accesibles	Cap. 2 y 3	Alto

La acción T13 contradice en parte una decisión del producto mínimo viable (exigir sesión para consultar), y su coste real no está en la interfaz sino en la protección: una consulta pública necesita limitación de intentos y de peticiones (T1), por lo que depende de la fase 1. La T14 es la de mayor alcance, porque la fiabilidad de una plataforma colaborativa depende en última

instancia de cómo se corrigen los errores y los abusos, y por eso debería ir antes que la T17: abrir los datos a terceros sin moderación expone a esos terceros a información sin verificar.

13.5 Fase 4: validación e investigación

Esta fase responde a la limitación que más veces ha aparecido en la memoria: ninguna decisión se ha contrastado con las personas a las que se dirige.

Tabla 13.4. *Fase 4: validación e investigación.*

Id.	Acción	Origen	Esfuerzo
T20	Pruebas con lectores de pantalla (NVDA, VoiceOver, TalkBack) y con personas usuarias con discapacidad, con tareas definidas y registro de resultados	Cap. 8 y 10	Medio
T21	Estudio de la escala de valoración: fiabilidad entre evaluadores (si dos personas puntúan lo mismo de la misma manera), contraste con protocolos normalizados de evaluación y piloto con una asociación o comunidad real	Cap. 1 y 12	Alto

T20 debería hacerse en cuanto se hayan corregido los defectos de T3, para no emplear a las personas participantes en encontrar problemas conocidos. T21 es el trabajo de mayor valor científico del conjunto: determinará si lo que la plataforma recoge, testimonios con una escala de 0 a 5, es lo bastante consistente como para servir de información a quien decide si ir a un local. La limitación epistemológica del capítulo 1 sigue sin resolverse hasta que exista esa evidencia.

Hay además una línea exploratoria que merece mención sin comprometerse: el uso de IA como función del producto, por ejemplo para sugerir descripciones de las fotografías y ayudar a rellenar el texto alternativo. Podría reducir el defecto 6 del capítulo 8, pero introduce riesgos de exactitud (una descripción errónea de un acceso es peor que ninguna), de privacidad (se enviarían imágenes a un tercero) y de responsabilidad, que habría que evaluar antes. Se menciona como posibilidad que debería estudiarse, no como recomendación.

13.6 Dependencias y camino mínimo

El camino mínimo para pasar de prototipo a servicio es T1, T2, T3, T4, T5 y T7. Con T6 se podría operar con garantías, y con T8 a T12 crecer sin deteriorar la base. El resto son decisiones de producto que dependen de qué comunidad use la aplicación.

Las dependencias principales son pocas: T13 requiere T1, T17 aconseja T14, T20 requiere T3, T10 se apoya en T9 y T8 se beneficia de que existan las correcciones de T1 y T3 para poder

fijarlas con pruebas. Con estas relaciones, el orden por fases es también un orden de ejecución razonable.

14. Conclusiones

14.1 Síntesis del trabajo

El trabajo partía de un problema concreto: quien tiene que decidir si puede acceder a un establecimiento suele carecer de información fiable sobre sus barreras, y las soluciones existentes la ofrecen de forma incompleta, con un modelo poco detallado o con una cobertura limitada (capítulos 1 y 2). Para abordarlo se diseñó, implementó y desplegó Accesibilidad 360, una plataforma web colaborativa en la que las personas registran establecimientos, los valoran con ocho criterios de accesibilidad, aportan fotografías y los localizan en un mapa.

La aplicación se construyó sobre una arquitectura de módulos con la autorización y las reglas de negocio en una capa de servicios (capítulo 4), un modelo de datos relacional con geografía administrativa normalizada (capítulo 5) y un proceso guiado por especificaciones escritas antes del código (capítulo 9). Se evaluó con tres instrumentos propios: una revisión de seguridad, una medición de accesibilidad y una batería de 192 pruebas automáticas (capítulos 7, 8 y 10). Está desplegada en producción sobre servicios gestionados (capítulo 11).

14.2 Conclusiones

El producto cumple su función. La aplicación hace lo que se propuso. Las cuatro funciones del objetivo general (registrar, valorar, compartir fotografías y localizar) están operativas en un entorno público, y de los diez objetivos específicos ninguno queda sin cumplir. La valoración detallada, con sus reservas, está en el capítulo 12 y no se repite aquí.

La arquitectura es el resultado más sólido. Concentrar la autorización y la validación en los servicios, limitar el acceso a la base de datos a unos pocos archivos y registrar las decisiones con su justificación dio un código coherente y fácil de probar. La capa de servicios es la mejor cubierta por las pruebas. Es la conclusión más transferible del proyecto: una separación de responsabilidades clara es lo que hace que el resto de la calidad se pueda medir y mejorar.

La calidad se midió y no quedó garantizada. Seguridad, accesibilidad y pruebas tienen carencias identificadas: cuatro hallazgos de seguridad de severidad media, siete defectos de accesibilidad y unas pruebas que no cubren la interfaz, la base de datos real ni los flujos de usuario. Ninguna de estas carencias está en el diseño de fondo, y todas tienen una corrección acotada y priorizada en el capítulo 13. La conclusión no es que el producto sea malo, sino que está en el estado propio de un prototipo bien construido y no en el de un servicio listo para uso abierto.

Un producto de accesibilidad debe medirse con sus propias reglas. El hallazgo más instructivo del trabajo es que una aplicación pensada para documentar barreras tenía ella misma barreras, y que solo se vieron al medir con herramientas y revisión manual. Haber declarado la

accesibilidad como buena práctica, sin un criterio de aceptación comprobable en cada página, fue la causa. Lo mismo ocurrió con la seguridad. Un requisito que no tiene una prueba asociada tiende a quedar sin cumplir sin que nadie lo advierta.

La IA acelera el trabajo y exige método. Las herramientas de IA aportaron velocidad en la generación de código, pruebas, documentación y revisión, y su aprovechamiento dependió de un contexto escrito y de un control humano real: especificaciones, normas, revisión del código, pruebas y aceptación manual. Los fallos que aparecieron (un error que solo se manifestaba en producción, cinco intentos de reparación sobre un problema que estaba en la configuración, la ausencia de medidas que nadie había pedido) fueron los de un asistente que hace lo que se le pide y no lo que nadie especificó. La IA no sustituye el criterio del desarrollador, y la responsabilidad del resultado sigue siendo suya.

Lo que no se verificó es tan importante como lo que sí. El trabajo no incluye pruebas con personas usuarias, ni con lectores de pantalla, ni mediciones de rendimiento o de carga, y la evaluación la ha hecho el propio autor. Por eso las conclusiones sobre accesibilidad, usabilidad y utilidad de la información son provisionales. Que estas limitaciones estén declaradas en cada capítulo y recogidas como trabajo futuro forma parte del resultado.

14.3 Aportaciones

Las aportaciones del trabajo son de tres tipos.

Al producto: un modelo de valoración por ocho criterios con opción «No aplicable», una valoración por persona y establecimiento, la fotografía como evidencia asociada y una geografía administrativa normalizada con los datos del INE, que se combinan en un producto reproducible (capítulo 2).

Al método: una forma de trabajar con IA basada en especificaciones previas, normas escritas y un flujo de siete fases, junto con una descripción crítica de dónde funcionó y dónde no (capítulo 9). El procedimiento es aplicable a otros proyectos y sus resultados, sus limitaciones incluidas, se documentan con evidencia.

A la evaluación: un análisis de seguridad con puntuación comparable de cada hallazgo, una medición de accesibilidad contra WCAG 2.2 con instrumentos y límites declarados, y un conjunto de acciones priorizadas para cerrar las deficiencias. Pueden reutilizarse como plantilla para revisar proyectos de alcance similar.

14.4 Lecciones aprendidas

De la experiencia del proyecto se extraen cinco lecciones prácticas.

1. **Escribir los requisitos no funcionales como criterios comprobables.** «Buenas prácticas de accesibilidad» o «seguridad» no son verificables, y lo que no se verifica se descuida. Cada uno debe llevar una prueba o una medición asociada desde el principio.
2. **Probar en las condiciones reales cuanto antes.** Los defectos de configuración, de integración y de plataforma aparecieron al desplegar. Un despliegue temprano, aunque sea provisional, los habría adelantado.
3. **Diagnosticar antes de corregir.** Ante un fallo conviene averiguar dónde está, en el código, en la configuración o en la plataforma, antes de pedir o aplicar una reparación.
4. **Mantener los cambios pequeños.** Es lo que hace posible una revisión real, y cuando no se cumplió, la verificación recayó en las pruebas.
5. **Medir con instrumentos independientes del que escribe el código.** Las herramientas automáticas, aun con sus límites, encontraron lo que la revisión del propio autor había pasado por alto.

14.5 Cierre

Accesibilidad 360 es una aplicación funcional, desplegada y documentada, construida con una base de ingeniería sólida, y cuyas deficiencias están medidas y ordenadas. Su siguiente paso natural no es añadir funciones, sino cerrar esas deficiencias, validar el producto con las personas a las que va dirigido y comprobar si la información que recoge es lo bastante fiable como para servir a quien decide si puede entrar en un lugar. Mientras esa validación no exista, el producto debe entenderse como un complemento colaborativo de las fuentes oficiales y no como un sustituto. Esa es la condición con la que el trabajo se entrega y la que determina por dónde debería continuar.

Bibliografía

Chomsky, M. (2021, 20 de mayo). *How anyone can make Maps more accessible*. Google. https://blog.google/products/maps/how-anyone-can-make-maps-more-accessible/

Dale, B. (2013, 1 de noviembre). AXS Map crowdsources accessibility information for disabled patrons. *Technical.ly*. https://technical.ly/diversity-equity-inclusion/axs-map-disability

datos.gob.es. (2019, 9 de enero). *Mapcesible*. https://datos.gob.es/es/aplicaciones/mapcesible

Deque Systems. (s. f.). *axe-core: Accessibility engine for automated web UI testing* [Software]. GitHub. https://github.com/dequelabs/axe-core

Instituto Nacional de Estadística. (2022, 19 de abril). *Encuesta de Discapacidad, Autonomía personal y situaciones de Dependencia (EDAD) 2020* [Nota de prensa]. https://ine.es/prensa/edad_2020_p.pdf

Jefatura del Estado. (2008, 21 de abril). Instrumento de Ratificación de la Convención sobre los derechos de las personas con discapacidad, hecho en Nueva York el 13 de diciembre de 2006. *Boletín Oficial del Estado*, (96), 20648-20659. https://www.boe.es/buscar/doc.php?id=BOE-A-2008-6963

Markbåge, S. (2023, 23 de octubre). *How to think about security in Next.js*. Next.js Blog. https://nextjs.org/blog/security-nextjs-server-components-actions

Mobasheri, A., Deister, J., & Dieterich, H. (2017). Wheelmap: The wheelchair accessibility crowdsourcing platform. *Open Geospatial Data, Software and Standards, 2*, Artículo 27. https://doi.org/10.1186/s40965-017-0040-5

Nemani, A. (2012, 18 de mayo). Crowdsourcing accessibility maps with AXSMap. *GovLoop*. https://www.govloop.com/community/blog/crowdsourcing-accessibility-maps-with-axsmap

OpenStreetMap Wiki. (s. f.-a). *Key:wheelchair*. https://wiki.openstreetmap.org/wiki/Key:wheelchair

OpenStreetMap Wiki. (s. f.-b). *Wheelmap*. https://wiki.openstreetmap.org/wiki/Wheelmap

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/2021/A00_2021_Introduction/

Real Decreto Legislativo 1/2013, de 29 de noviembre, por el que se aprueba el Texto Refundido de la Ley General de derechos de las personas con discapacidad y de su inclusión social. (2013). *Boletín Oficial del Estado*, (289). https://boe.gob.es/boe/dias/2013/12/03/pdfs/BOE-A-2013-12632.pdf

Vercel. (2026, 24 de agosto). *Vercel Functions limits*. https://vercel.com/docs/functions/limitations

World Wide Web Consortium. (2023, 5 de octubre). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/

Zero Project. (2018, 31 de enero). *The playful way to create accessibility maps* [Ficha de proyecto: AXS Map]. https://zeroproject.org/view/project/037e35d6-9a17-eb11-a813-0022489b3a6d

Anexos

Los anexos se entregan como documentos independientes, junto con esta memoria, para no alterar su extensión. La tabla A.1 los relaciona con el capítulo que los cita y con el nombre del archivo.

Tabla A.1. Relación de anexos.

Anexo	Contenido	Capítulos	Archivo
A	Manual de usuario: guía paso a paso para personas usuarias y administradoras, con capturas	6 y 11	`Anexo_A_Manual_de_usuario.docx`
B	Informe de auditoría de seguridad (versión 2): metodología, doce hallazgos con evidencia y recomendaciones, y correspondencia con OWASP Top 10:2021	7, 9 y 12	`Anexo_B_Informe_de_auditoria_de_seguridad.docx`
C	Especificaciones del proyecto: las catorce especificaciones de núcleo y de funcionalidades que guiaron el desarrollo	3, 4 y 9	`Anexo_C_Especificaciones.docx`
D	Modelo de datos: diagrama entidad-relación, relación de migraciones y esquema de Prisma íntegro	5	`Anexo_D_Modelo_de_datos.docx`
E	Capturas de pantalla de las pantallas principales, en escritorio y en móvil	6	`Anexo_E_Capturas_de_pantalla.docx`
F	Resultados de la medición de accesibilidad: datos de axe-core y de las comprobaciones manuales	8	`Anexo_F_Medicion_de_accesibilidad.docx`

Dos precisiones sobre su procedencia. Los anexos A y E se elaboraron sobre una copia de la aplicación con una base de datos de demostración, de modo que muestran datos ficticios y no los del entorno de producción. El anexo C reproduce las especificaciones tal como se redactaron durante el desarrollo, y cuando alguna difiere de la versión entregada prevalece el código, descrito en los capítulos 3 a 6.